LAPORAN PRAKTIKUM NATURAL LANGUAGE PROCESSING

Modul 1

Eka Yuliana Rizky

240306011

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	1
Topik	Eksplorasi Corpus __ komentar TikTok mengenai Koperasi Desa Merah Putih
Repository	https://github.com/lian180706/NATURAL-LANGUAGE-PROCESSING
Tanggal	-

A. Tujuan

1. Membaca dan memuat dataset komentar TikTok dalam format CSV menggunakan Python.
2. Memahami struktur, isi, dan karakteristik dataset komentar TikTok mengenai Koperasi Desa Merah Putih.
3. Melakukan pemeriksaan dan pembersihan data dari nilai kosong serta komentar yang duplikat.
4. Menghitung statistik deskriptif teks berupa jumlah karakter, jumlah kata, dan jumlah kalimat.
5. Memvisualisasikan distribusi panjang komentar untuk membantu memahami karakteristik corpus teks.

B. Requirement

1. Sistem Operasi yang digunakan : Microsoft Windows 11 Pro
2. Browser : Versi 153.0.8010.50 (Build Resmi) (64 bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Natural Language Processing

Natural Language Processing (NLP) merupakan bidang dalam kecerdasan buatan yang berfokus pada kemampuan komputer untuk memproses, menganalisis, dan memahami bahasa manusia. Bahasa manusia memiliki struktur yang kompleks karena dapat mengandung variasi kata, konteks, singkatan, kesalahan penulisan, tanda baca, dan bentuk bahasa informal. Oleh karena itu, data bahasa perlu diolah terlebih dahulu agar dapat dianalisis menggunakan komputer. NLP digunakan dalam berbagai aplikasi, seperti penerjemahan bahasa, pencarian informasi, analisis sentimen, klasifikasi teks, chatbot, dan peringkasan dokumen. Jurafsky dan Martin menjelaskan NLP sebagai bidang yang berkaitan dengan pemrosesan bahasa dan teks oleh komputer, termasuk analisis kata, struktur bahasa, serta pemodelan bahasa.

Dalam praktikum ini, NLP diterapkan pada data komentar TikTok. Komentar yang awalnya berupa teks tidak terstruktur diubah menjadi data yang dapat dianalisis secara kuantitatif. Proses tersebut dilakukan dengan mengetahui jumlah dokumen, jumlah karakter, jumlah kata, dan jumlah kalimat. Hasil eksplorasi ini dapat menjadi dasar untuk proses NLP pada tahap berikutnya.

2. Corpus

Corpus merupakan kumpulan teks yang digunakan sebagai bahan penelitian atau analisis bahasa. Dalam NLP, corpus dapat berupa kumpulan berita, buku, percakapan, ulasan produk, komentar media sosial, maupun jenis teks lainnya. NLTK menjelaskan bahwa text corpus merupakan kumpulan teks yang terstruktur dan dapat digunakan untuk berbagai analisis bahasa.

Pada praktikum ini, corpus berupa kumpulan komentar TikTok yang berkaitan dengan Koperasi Desa Merah Putih. Setiap komentar dapat diperlakukan sebagai sebuah dokumen teks. Dengan demikian, satu baris data komentar dapat dianalisis berdasarkan karakteristik teksnya, seperti jumlah karakter, jumlah kata, dan jumlah kalimat. Penggunaan corpus yang cukup besar memungkinkan mahasiswa memperoleh gambaran mengenai karakteristik bahasa yang digunakan dalam data media sosial.

3. Eksplorasi Data

Eksplorasi data merupakan tahap awal yang dilakukan untuk memahami kondisi dataset sebelum masuk ke proses pengolahan NLP yang lebih lanjut. Tahap ini meliputi pemeriksaan jumlah baris dan kolom, nama kolom, tipe data, nilai kosong, serta contoh isi data. Eksplorasi penting dilakukan agar kolom yang digunakan dalam analisis dapat ditentukan dengan tepat.

Pada praktikum ini, dataset dibaca menggunakan library Pandas melalui fungsi read_csv(). Setelah dataset dimuat, struktur data diperiksa menggunakan head(), tail(), info(), shape, dan columns. Hasil pemeriksaan tersebut digunakan untuk mengetahui bahwa dataset memiliki beberapa kolom metadata dan satu kolom utama yang berisi teks komentar. Pendekatan seperti ini sesuai dengan penggunaan Python untuk mengeksplorasi dan menganalisis kumpulan teks sebagaimana diperkenalkan dalam buku NLTK.

4. Data Cleaning

Data cleaning merupakan proses mempersiapkan data agar dapat digunakan untuk analisis dengan mengurangi atau menangani data yang tidak sesuai. Pada data teks dari media sosial, masalah yang dapat ditemukan antara lain nilai kosong, data duplikat, kesalahan penulisan, singkatan, dan bentuk teks yang tidak konsisten.

Dalam praktikum ini, data cleaning difokuskan pada pemeriksaan nilai kosong dan data duplikat pada kolom komentar. Nilai kosong pada kolom teks tidak dapat digunakan untuk menghitung karakter, kata, atau kalimat sehingga perlu ditangani. Pandas menyediakan fungsi seperti isnull() untuk mendeteksi nilai yang hilang.

Selain itu, komentar yang memiliki teks sama dapat muncul lebih dari satu kali. Untuk mengetahui data duplikat digunakan fungsi duplicated(), sedangkan drop_duplicates() digunakan untuk menghapus data yang dianggap duplikat. Dokumentasi Pandas menjelaskan bahwa kedua fungsi tersebut dapat digunakan untuk mengidentifikasi dan menghapus baris duplikat berdasarkan kolom tertentu.

5. Statistik Deskriptif Teks

Statistik deskriptif teks digunakan untuk menggambarkan karakteristik corpus secara kuantitatif. Dalam praktikum ini, beberapa ukuran yang

digunakan adalah jumlah karakter, jumlah kata, jumlah kalimat, rata-rata jumlah kata, serta nilai minimum dan maksimum jumlah kata.

Jumlah karakter menunjukkan banyaknya karakter yang terdapat dalam sebuah komentar, sedangkan jumlah kata menunjukkan banyaknya kata yang terdapat dalam komentar. Dalam pengolahan teks, konsep token juga penting karena teks dapat dipecah menjadi unit-unit yang akan dianalisis. NLTK menjelaskan bahwa token dapat berupa rangkaian karakter yang diperlakukan sebagai satu unit, misalnya kata atau tanda tertentu.

Jumlah kalimat digunakan untuk mengetahui banyaknya kalimat yang terdapat dalam sebuah komentar. Ketiga ukuran tersebut dapat digunakan untuk membandingkan panjang dan karakteristik dokumen dalam corpus. Statistik seperti rata-rata, minimum, maksimum, dan ukuran kuartil juga membantu memberikan gambaran mengenai distribusi panjang teks.

6. Visualisasi Data

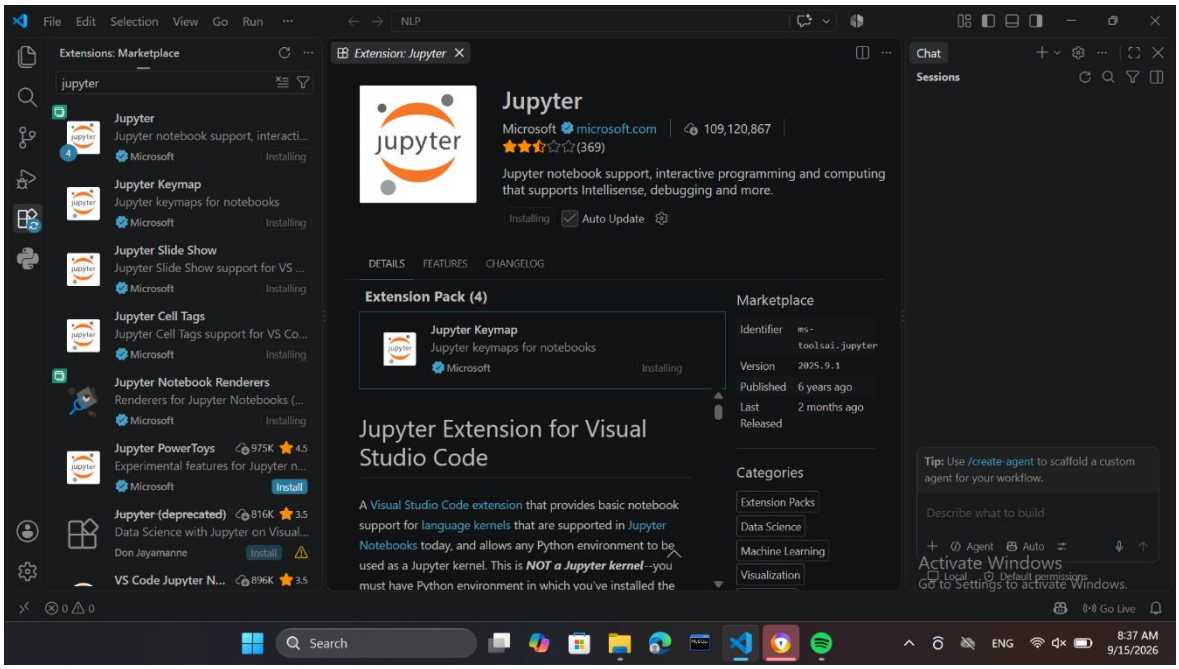
Visualisasi data digunakan untuk menyajikan hasil analisis dalam bentuk grafik sehingga pola pada dataset lebih mudah diamati. Salah satu visualisasi yang digunakan dalam praktikum adalah histogram distribusi jumlah kata. Histogram menunjukkan bagaimana jumlah kata tersebar pada dokumen dalam corpus.

Dengan melihat distribusi panjang dokumen, dapat diketahui apakah sebagian besar komentar memiliki jumlah kata yang relatif pendek atau terdapat banyak komentar dengan ukuran teks yang lebih panjang. Informasi tersebut dapat membantu memahami karakteristik corpus sebelum dilakukan proses NLP lanjutan. NLTK juga memperkenalkan penggunaan statistik sederhana dan distribusi frekuensi untuk melihat pola kemunculan unsur bahasa dalam teks.

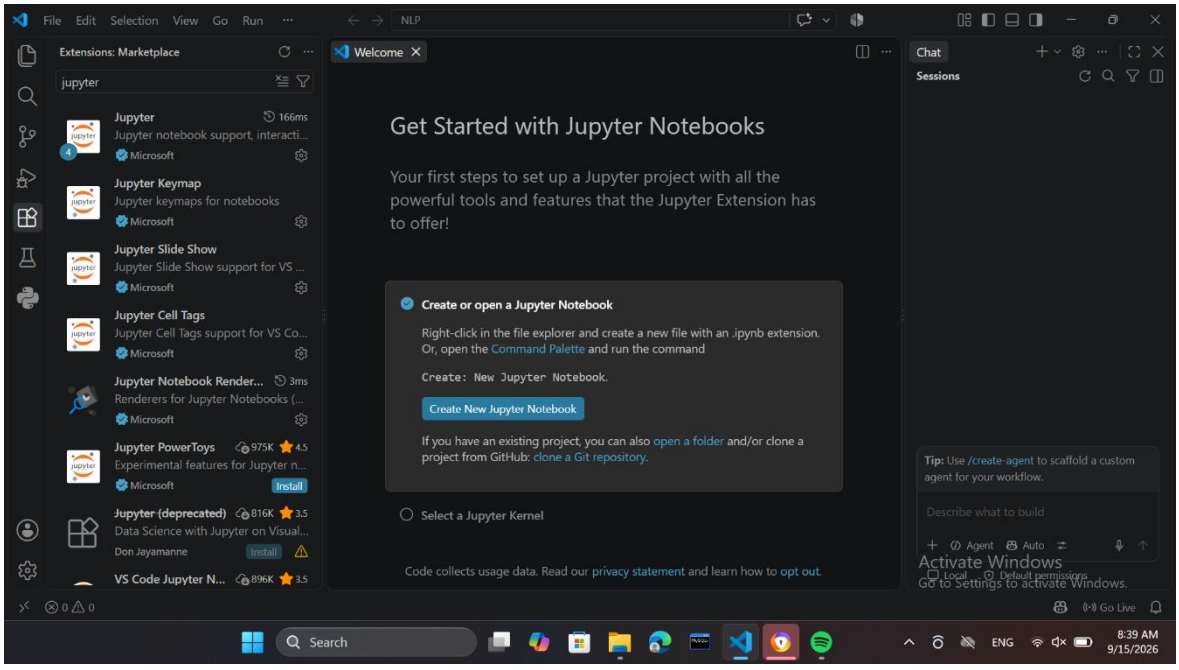
Berdasarkan teori tersebut, eksplorasi corpus merupakan tahap penting sebelum melakukan analisis NLP yang lebih lanjut. Pemeriksaan struktur data, pembersihan data, perhitungan statistik teks, dan visualisasi membantu memastikan bahwa data yang digunakan telah dipahami karakteristiknya dan siap digunakan untuk proses pengolahan berikutnya.

D. Implementasi

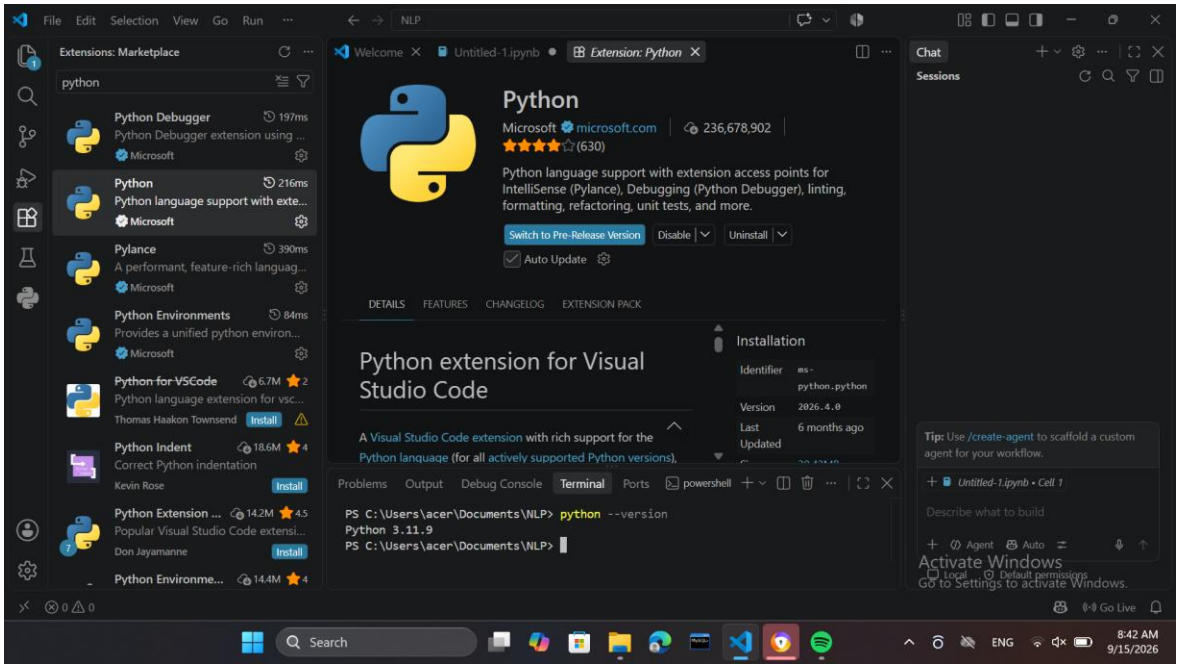
Persiapan Lingkungan dan Instalasi



Gambar Instalasi 1. Instalasi ekstensi Jupyter pada Visual Studio Code.



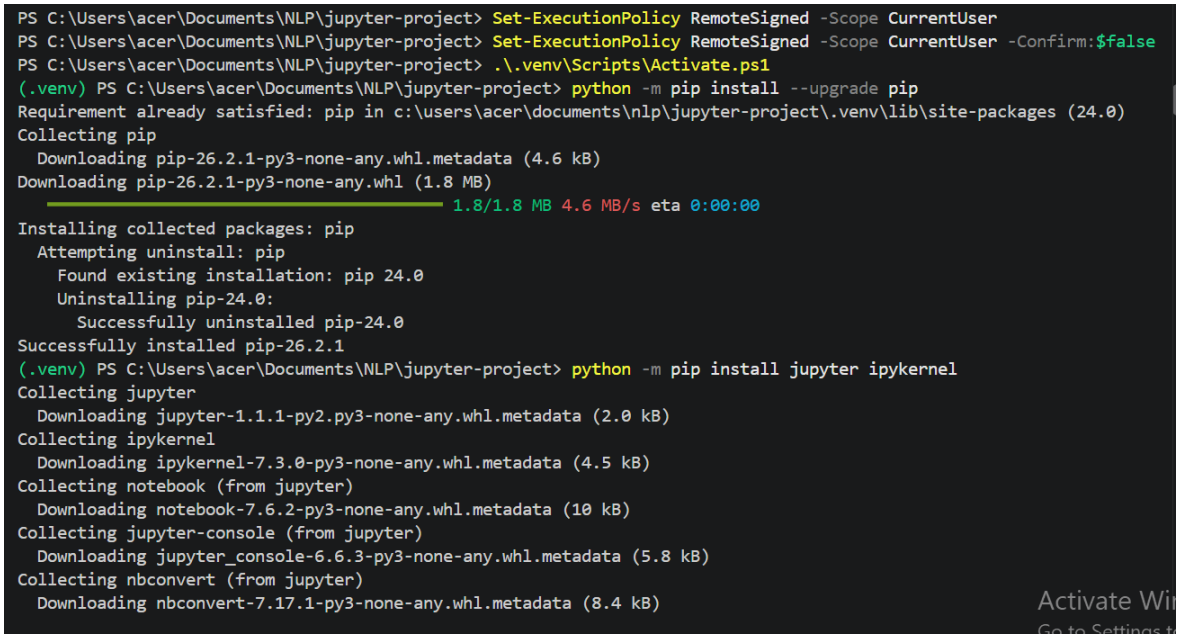
Gambar Instalasi 2. Tampilan awal Jupyter Notebook pada Visual Studio Code.



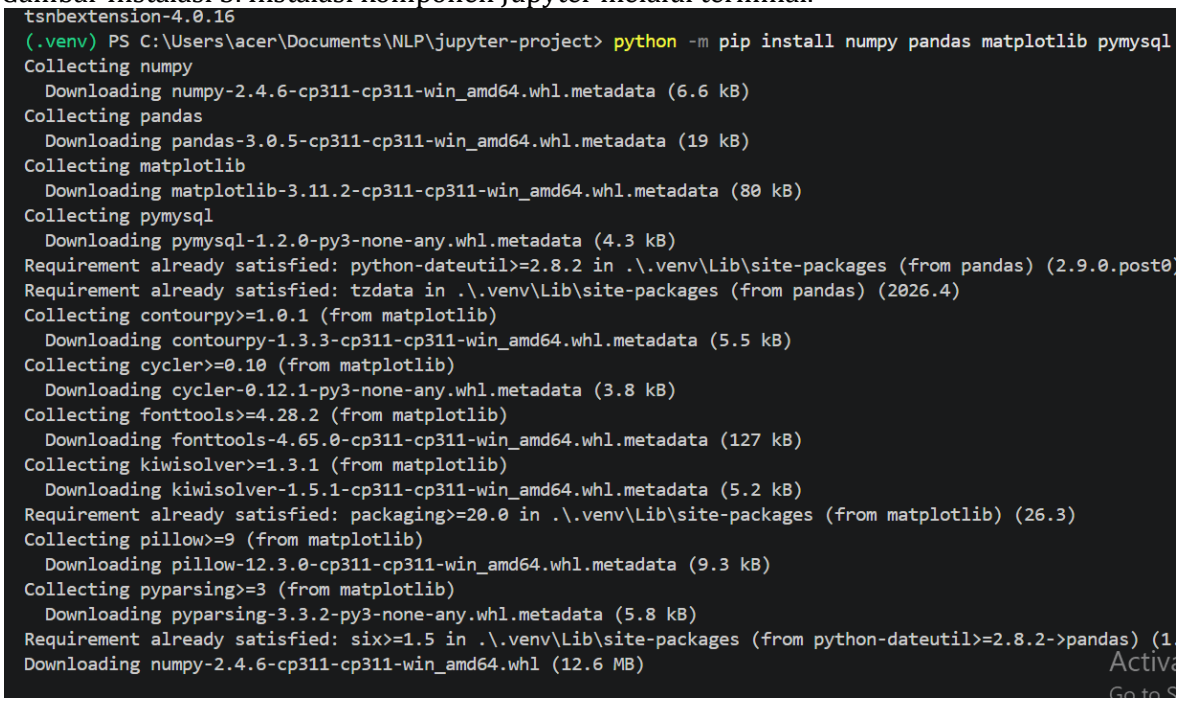
Gambar Instalasi 3. Instalasi ekstensi Python pada Visual Studio Code.

```
PS C:\Users\acer\Documents\NLP> python --version
Python 3.11.9
```

Gambar Instalasi 4. Pemeriksaan versi Python pada terminal.



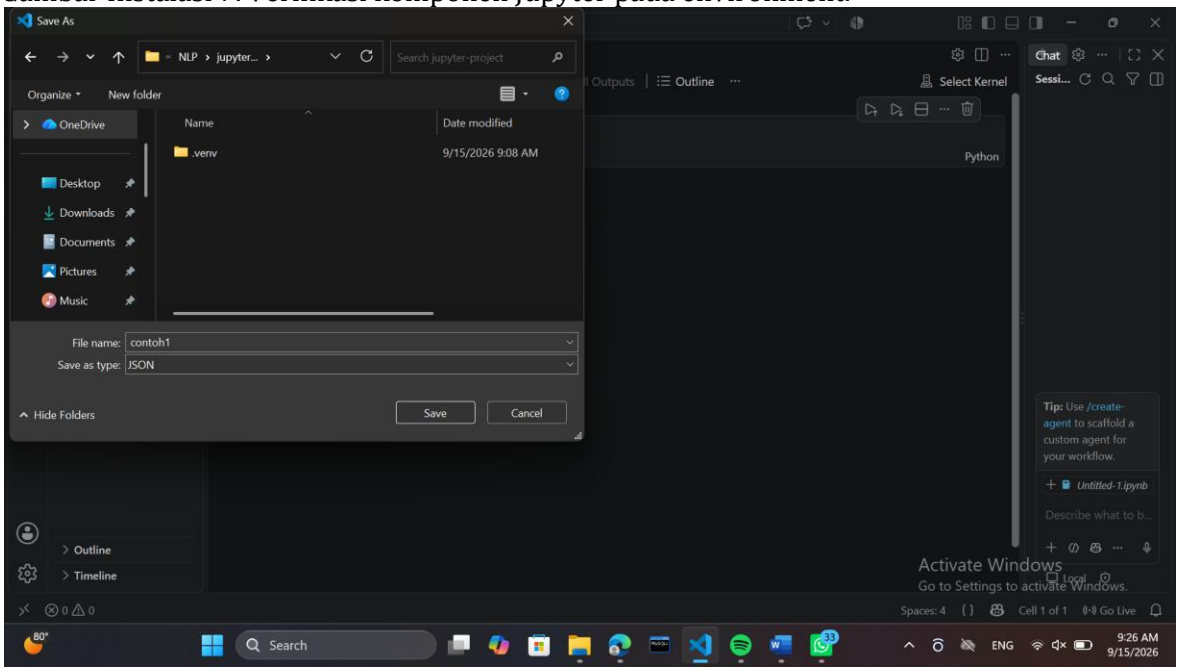
Gambar Instalasi 5. Instalasi komponen Jupyter melalui terminal.



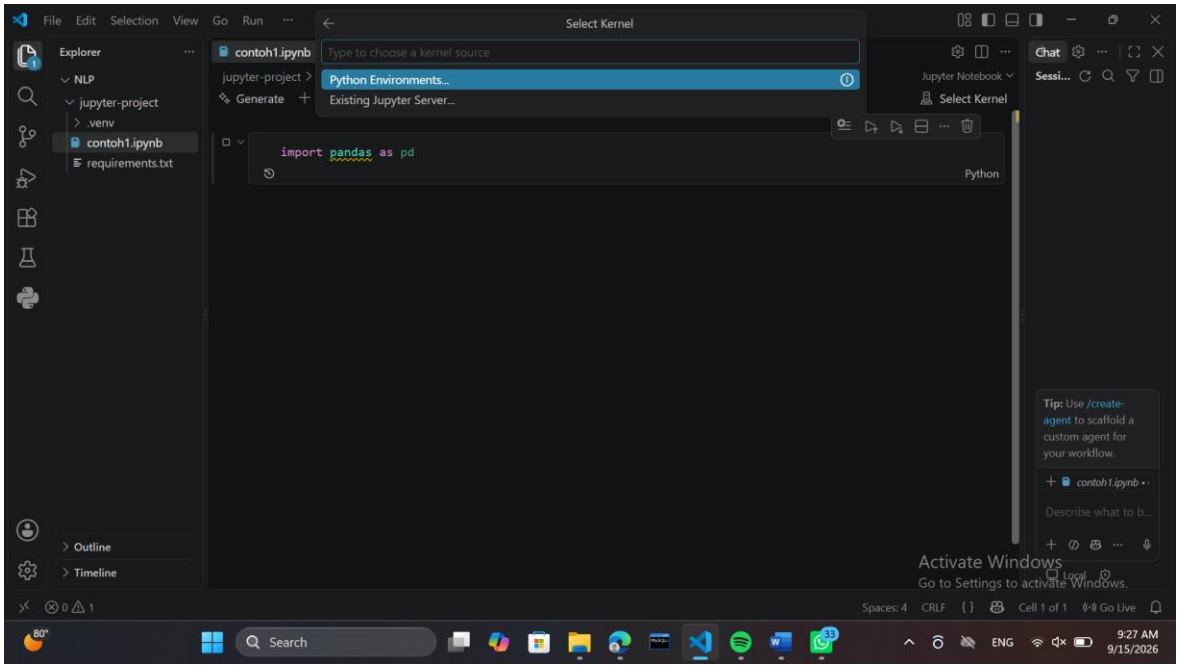
Gambar Instalasi 6. Instalasi pandas dan dependensi yang diperlukan.

```
(.venv) PS C:\Users\acer\Documents\NLP\jupyter-project> pip freeze > requirements.txt
(.venv) PS C:\Users\acer\Documents\NLP\jupyter-project> jupyter notebook
[I 2026-09-15 09:12:34.126 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-09-15 09:12:34.152 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-09-15 09:12:34.179 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-09-15 09:12:34.205 ServerApp] notebook | extension was successfully linked.
[I 2026-09-15 09:12:34.229 ServerApp] Writing Jupyter server cookie secret to C:\Users\acer\AppData\Roa
runtime\jupyter_cookie_secret
[I 2026-09-15 09:12:34.386 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-09-15 09:12:34.480 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-09-15 09:12:34.486 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-09-15 09:12:34.487 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-09-15 09:12:34.505 LabApp] JupyterLab extension loaded from C:\Users\acer\Documents\NLP\jupyter
v\Lib\site-packages\jupyterlab
[I 2026-09-15 09:12:34.505 LabApp] JupyterLab application directory is C:\Users\acer\Documents\NLP\jupy
venv\share\jupyter\lab
[I 2026-09-15 09:12:34.507 LabApp] Extension Manager is 'pypi'.
[I 2026-09-15 09:12:35.020 ServerApp] jupyterlab | extension was successfully loaded.
[I 2026-09-15 09:12:35.045 ServerApp] notebook | extension was successfully loaded.
[I 2026-09-15 09:12:35.048 ServerApp] Serving notebooks from local directory: C:\Users\acer\Documents\N
oject
[I 2026-09-15 09:12:35.049 ServerApp] Jupyter Server 2.21.0 is running at:
[I 2026-09-15 09:12:35.049 ServerApp] http://localhost:8888/tree?token=d1395f8bc81b307cd7d4a42abd8fd18a
a39
[I 2026-09-15 09:12:35.049 ServerApp]     http://127.0.0.1:8888/tree?token=d1395f8bc81b307cd7d4a42abd8f
ae49a39
[I 2026-09-15 09:12:35.049 ServerApp] Use Control-C to stop this server and shut down all kernels (twic
```

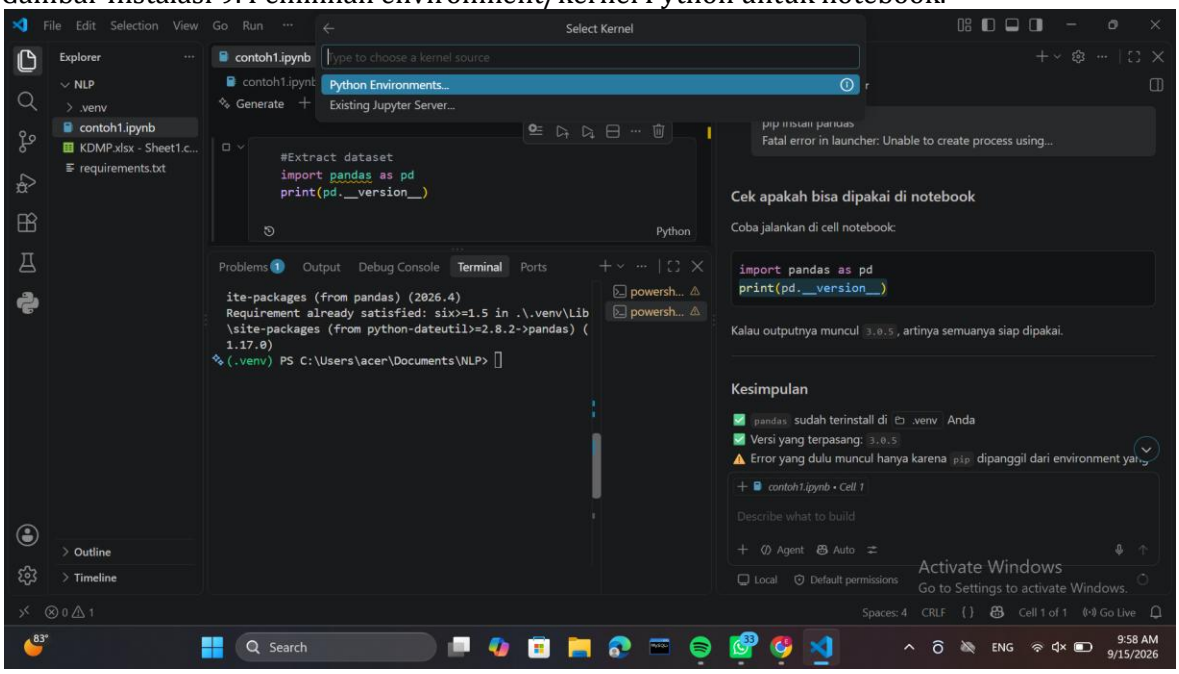
Gambar Instalasi 7. Verifikasi komponen Jupyter pada environment.



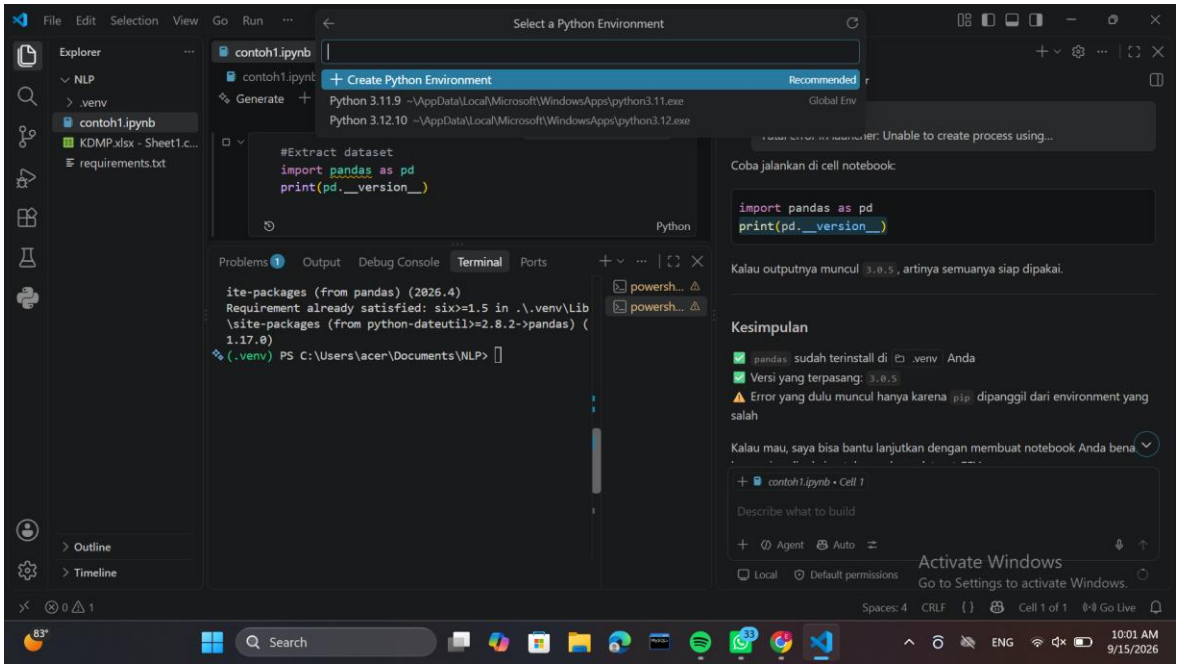
Gambar Instalasi 8. Proses penyimpanan notebook pada Visual Studio Code.



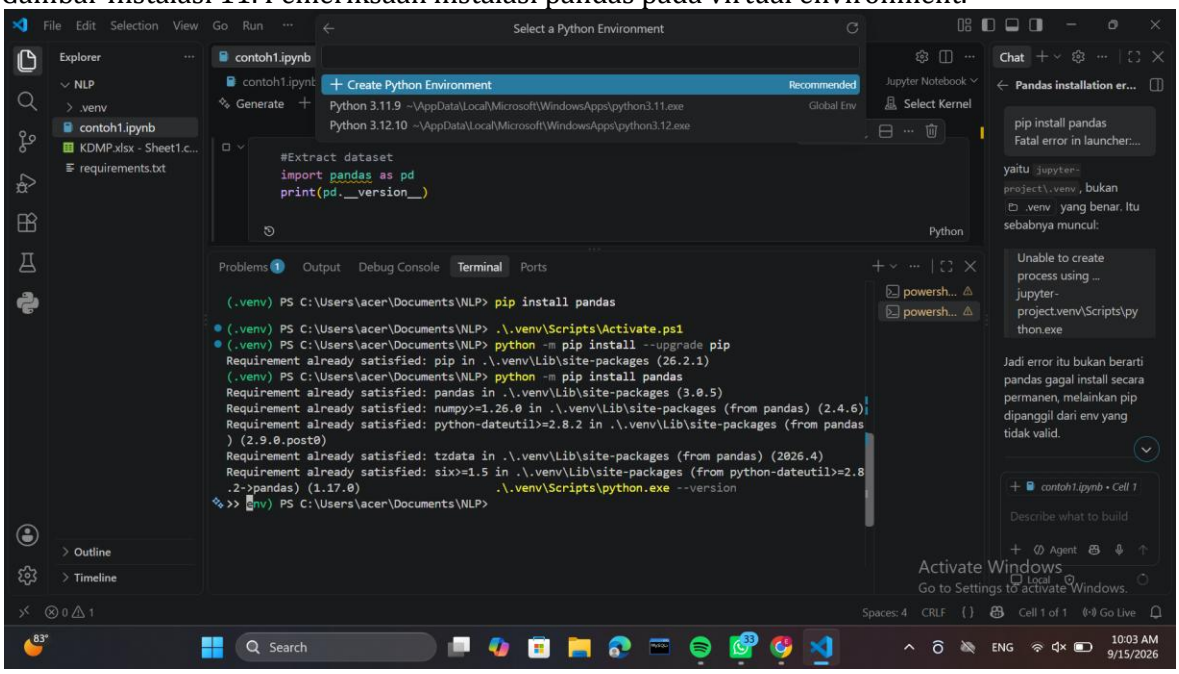
Gambar Instalasi 9. Pemilihan environment/kernel Python untuk notebook.



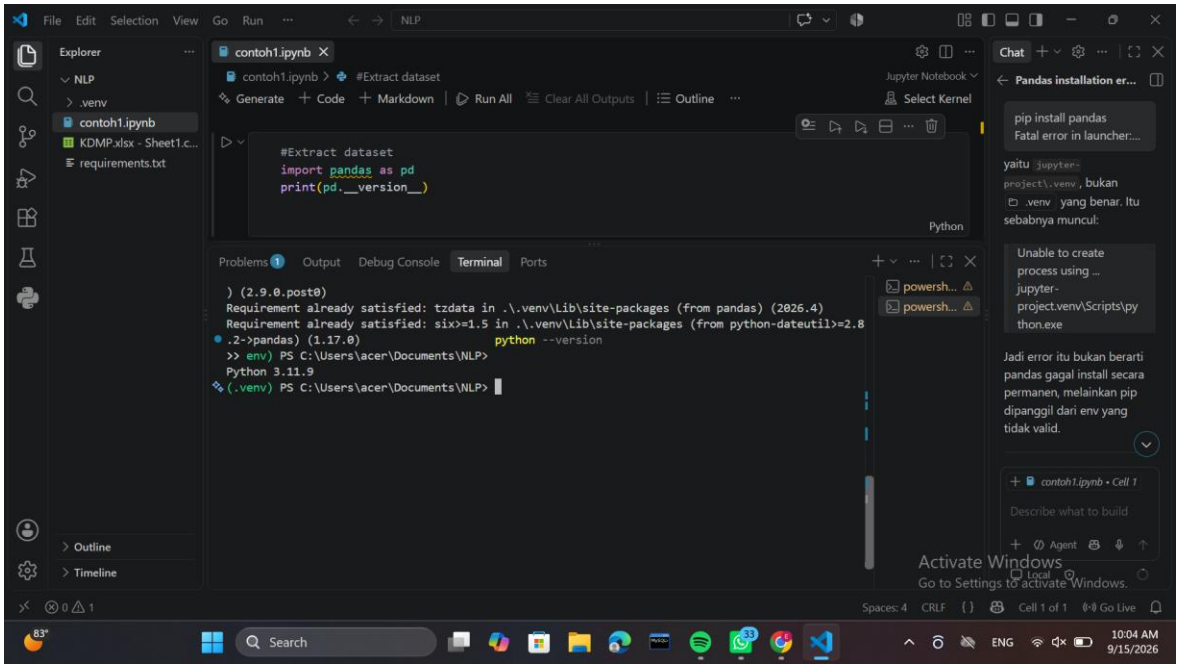
Gambar Instalasi 10. Pemilihan kernel Python pada notebook.



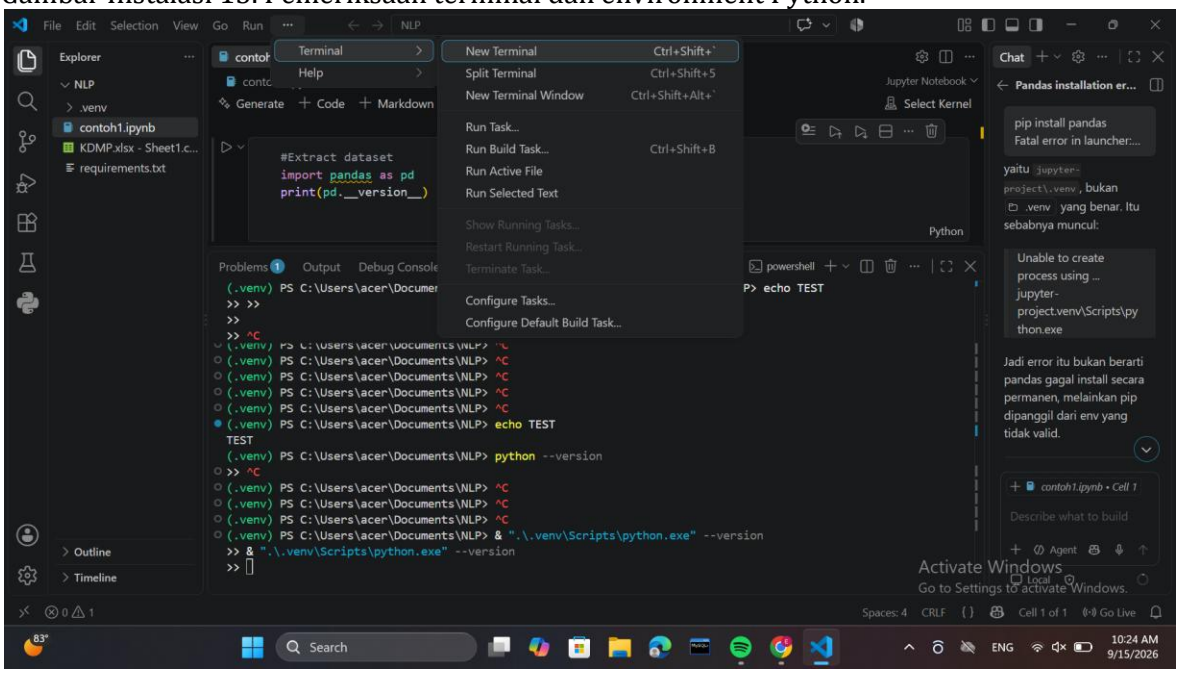
Gambar Instalasi 11. Pemeriksaan instalasi pandas pada virtual environment.



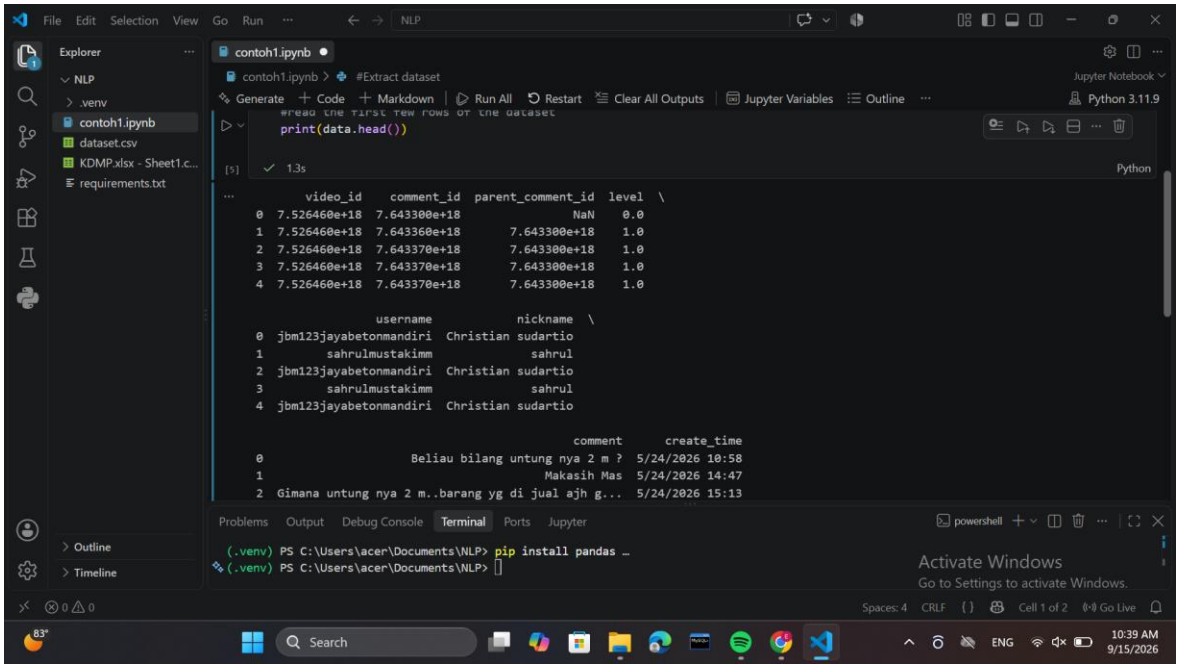
Gambar Instalasi 12. Verifikasi versi pandas pada virtual environment.



Gambar Instalasi 13. Pemeriksaan terminal dan environment Python.

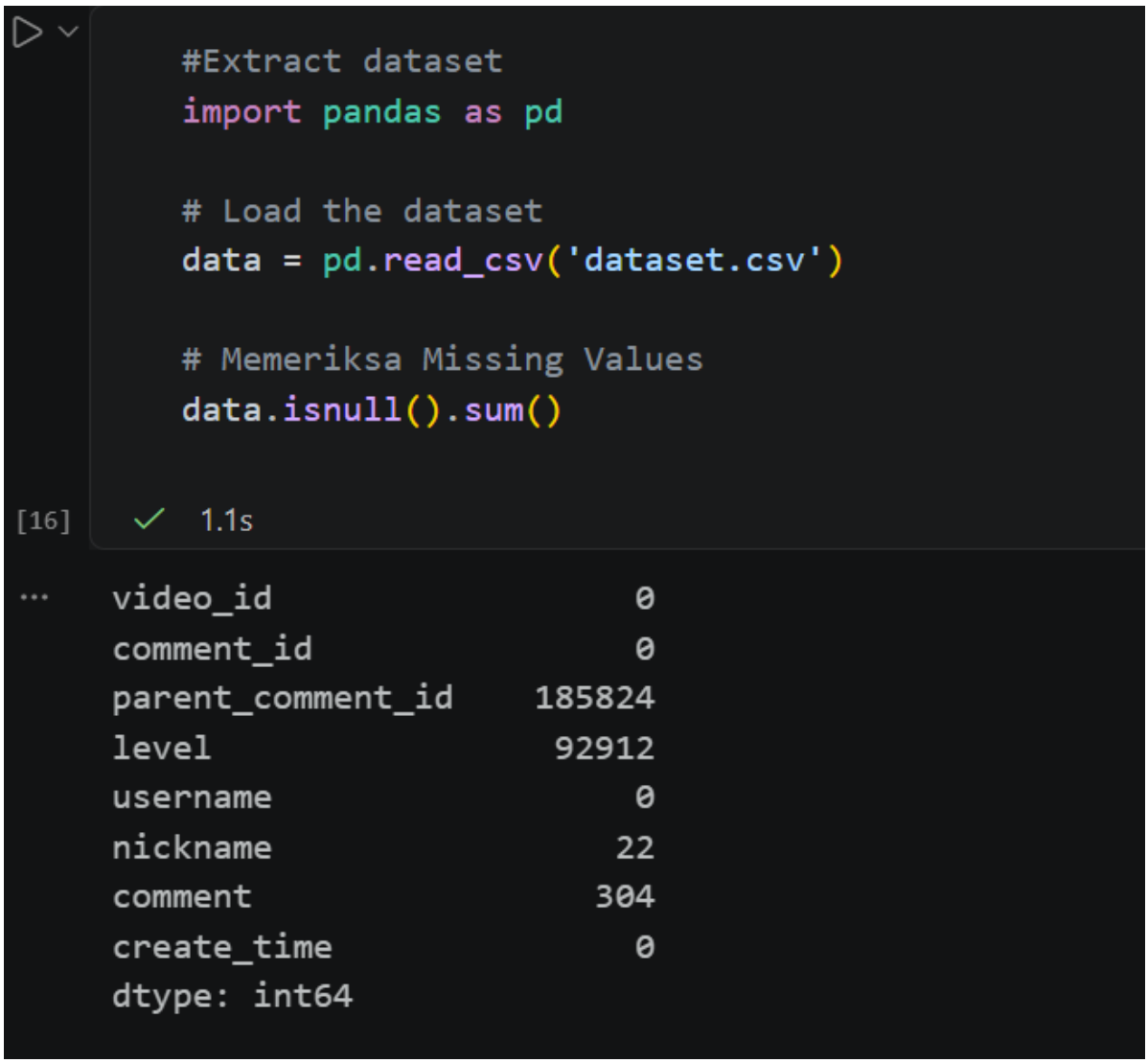


Gambar Instalasi 14. Proses menjalankan kode pada notebook.



Gambar Instalasi 15. Hasil eksekusi notebook dan pembacaan dataset.

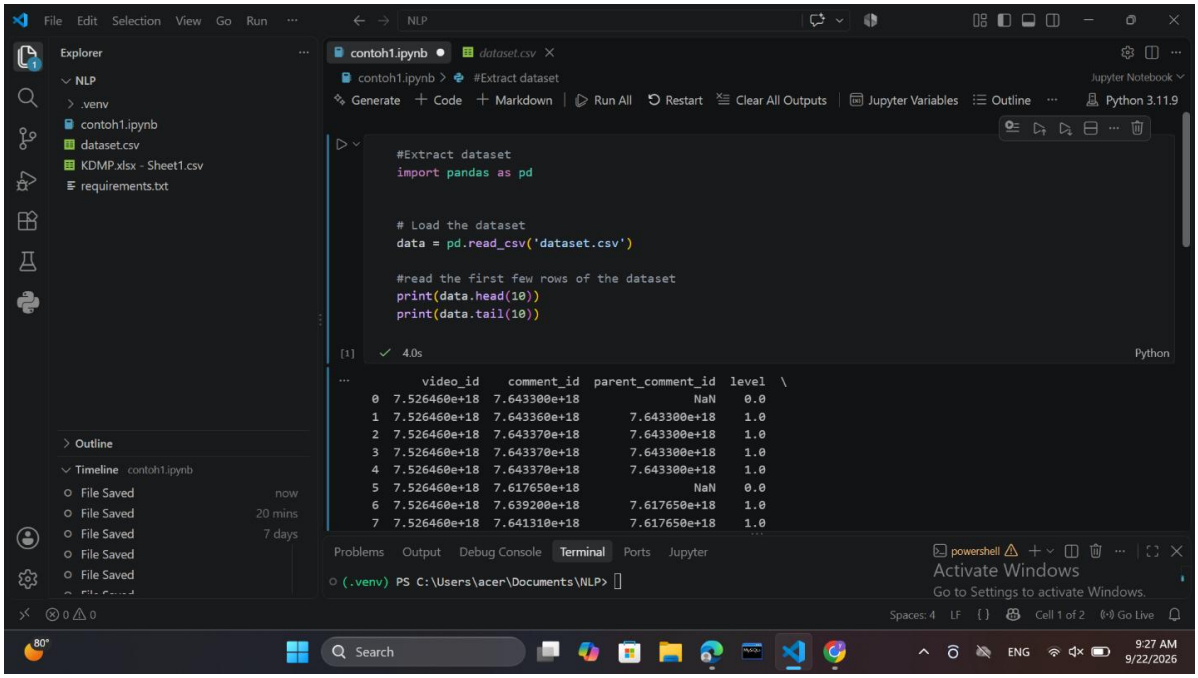
1. Pembacaan Dataset dan Pemeriksaan Awal



Gambar 1. Pembacaan Dataset dan Pemeriksaan Awal

Penjelasan: Tahap awal implementasi dilakukan dengan membaca dataset komentar TikTok menggunakan library Pandas. Setelah data dimuat, dilakukan pemeriksaan nilai kosong untuk memperoleh gambaran awal mengenai kelengkapan data pada setiap kolom.

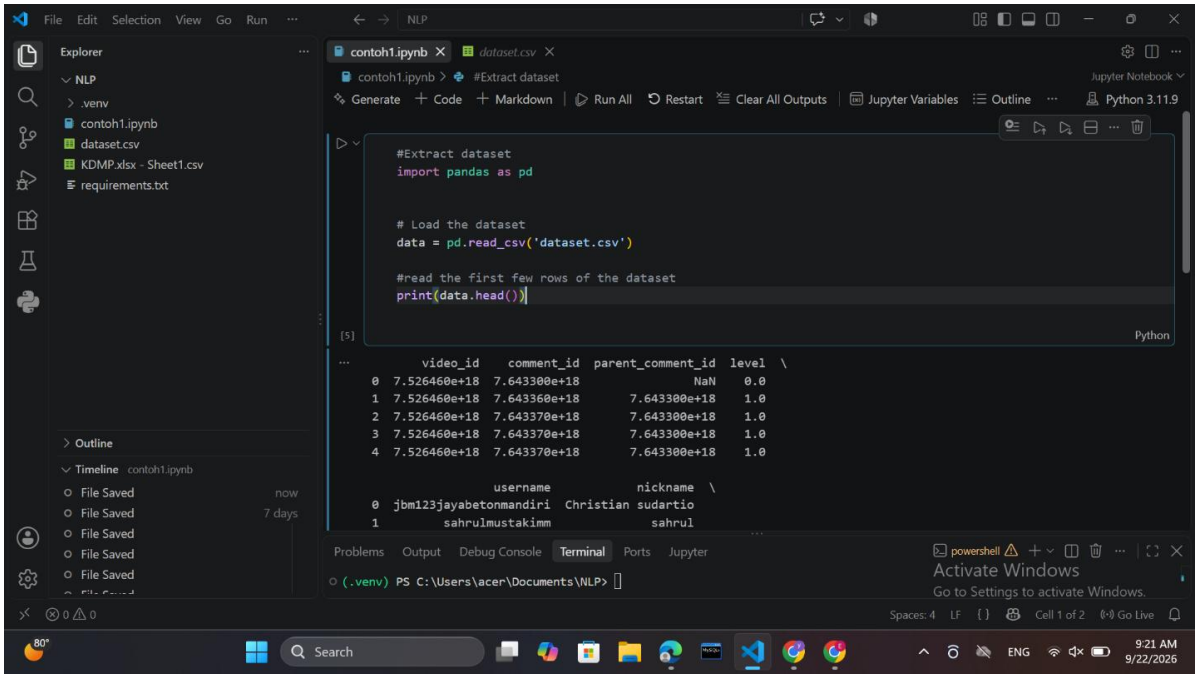
2. Melihat Data Awal



Gambar 2. Melihat Data Awal

Penjelasan: Data awal ditampilkan untuk mengamati bentuk dan isi dataset secara langsung. Pemeriksaan ini membantu mengidentifikasi struktur data serta mengetahui contoh isi pada beberapa baris pertama.

3. Pemeriksaan Contoh Data



Gambar 3. Pemeriksaan Contoh Data

Penjelasan: Pemeriksaan contoh data dilakukan untuk memastikan bahwa data telah terbaca dengan benar dan kolom-kolom yang tersedia dapat digunakan pada tahap eksplorasi berikutnya.

4. Informasi Struktur Dataset

```
# Informasi dataset
print(data.info())
```

```
[1]  ✓ 1.9s
```

```
<class 'pandas.DataFrame'>
RangeIndex: 220051 entries, 0 to 220050
Data columns (total 8 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   video_id           220051 non-null  float64
 1   comment_id         220051 non-null  float64
 2   parent_comment_id  34227 non-null   float64
 3   level              127139 non-null  float64
 4   username           220051 non-null  str
 5   nickname           220029 non-null  str
 6   comment            219747 non-null  str
 7   create_time        220051 non-null  str
dtypes: float64(4), str(4)
memory usage: 32.7 MB
None
```

Gambar 4. Informasi Struktur Dataset

Penjelasan: Fungsi data.info() digunakan untuk memperoleh informasi mengenai jumlah entri, nama kolom, jumlah nilai non-null, tipe data, dan penggunaan memori. Berdasarkan hasil yang ditampilkan, dataset terdiri atas 220.051 baris dan 8 kolom.

5. Pemeriksaan Informasi Dataset

```
    # Melihat nama kolom
    #print(data.columns)

    # Informasi dataset
    print(data.info())
```

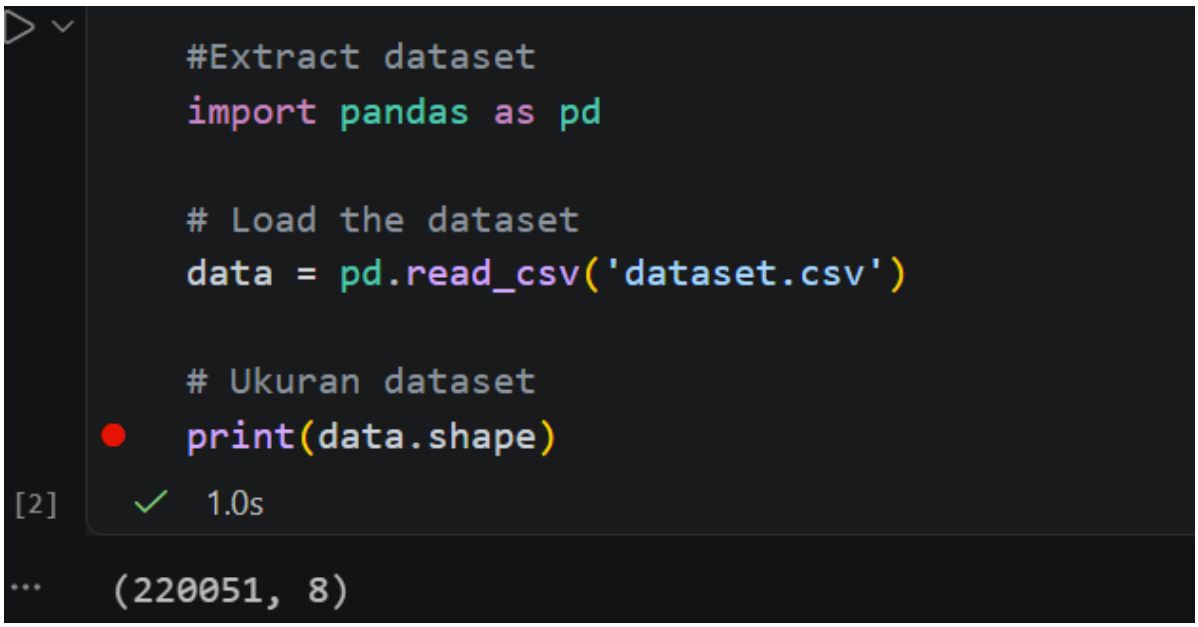
[4] ✓ 1.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 220051 entries, 0 to 220050
Data columns (total 8 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   video_id           220051 non-null  float64
 1   comment_id         220051 non-null  float64
 2   parent_comment_id  34227 non-null   float64
 3   level              127139 non-null  float64
 4   username           220051 non-null  str
 5   nickname           220029 non-null  str
 6   comment            219747 non-null  str
 7   create_time        220051 non-null  str
dtypes: float64(4), str(4)
memory usage: 32.7 MB
None
```

Gambar 5. Pemeriksaan Informasi Dataset

Penjelasan: Informasi dataset diperiksa kembali untuk memastikan struktur data yang digunakan dalam praktikum. Pemeriksaan ini mencakup jumlah baris, jumlah kolom, tipe data, dan ketersediaan nilai pada masing-masing kolom.

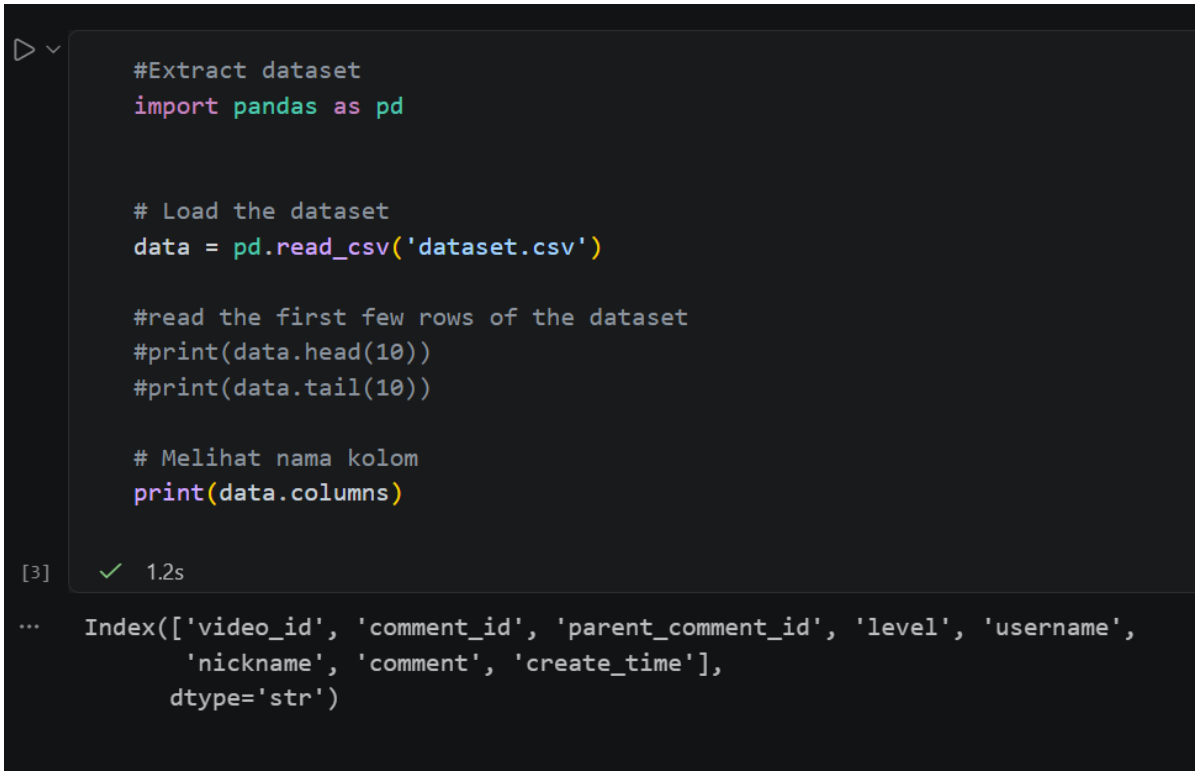
6. Ukuran Dataset



Gambar 6. Ukuran Dataset

Penjelasan: Ukuran dataset diperoleh menggunakan data.shape. Hasil (220051, 8) menunjukkan bahwa dataset terdiri atas 220.051 baris dan 8 kolom.

7. Nama Kolom Dataset

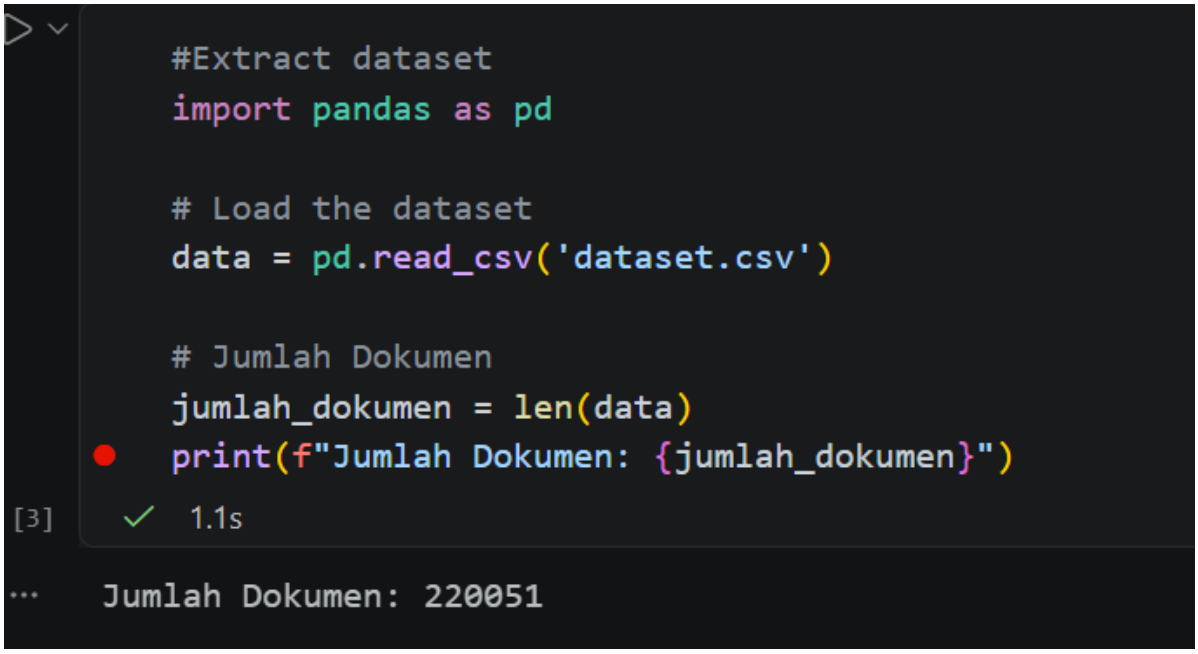


Gambar 7. Nama Kolom Dataset

Penjelasan: Nama kolom diperiksa menggunakan data.columns. Kolom yang tersedia meliputi video_id, comment_id, parent_comment_id, level, username, nickname,

comment, dan create_time. Kolom comment digunakan sebagai sumber utama teks komentar dalam analisis.

8. Jumlah Dokumen



Gambar 8. Jumlah Dokumen

Penjelasan: Jumlah dokumen dihitung menggunakan fungsi len(data). Hasil yang diperoleh adalah 220.051 dokumen. Dalam konteks praktikum ini, setiap baris komentar diperlakukan sebagai satu dokumen teks.

9. Pemilihan Kolom Teks



Gambar 9. Pemilihan Kolom Teks

Penjelasan: Kolom comment ditetapkan sebagai kolom teks yang akan dianalisis. Nilai pada kolom tersebut dikonversi menjadi tipe string dan dikumpulkan sebagai data teks untuk proses pengolahan berikutnya.

10. Pemeriksaan Contoh Teks

```
#Extract dataset
import pandas as pd

# Load the dataset
data = pd.read_csv('dataset.csv')

# Melihat Contoh Teks
for text in data_teks[:5]:
    print(text)
    print("-" * 80)
```

[15] ✓ 1.0s

```
Beliau bilang untung nya 2 m ?
--------------------------------------------------------------------------------
Makasih Mas
--------------------------------------------------------------------------------
Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁
--------------------------------------------------------------------------------
[Sticker] dibilang Makasih Mas udah 2M itu
--------------------------------------------------------------------------------
😳😳😳
--------------------------------------------------------------------------------
```

Gambar 10. Pemeriksaan Contoh Teks

Penjelasan: Beberapa contoh komentar ditampilkan untuk mengidentifikasi karakteristik awal corpus. Data menunjukkan penggunaan bahasa informal, variasi penulisan, serta kemungkinan adanya elemen nonteks seperti penanda stiker.

11. Pemeriksaan Missing Value pada Kolom Teks

```
#Extract dataset
import pandas as pd

# Load the dataset
data = pd.read_csv('dataset.csv')

# Menghitung Missing Value pada Kolom Teks
data[text_column].isnull().sum()
```

[17] ✓ 1.1s

```
np.int64(304)
```

Gambar 11. Pemeriksaan Missing Value pada Kolom Teks

Penjelasan: Pemeriksaan nilai kosong pada kolom comment menghasilkan 304 nilai yang tidak tersedia. Informasi ini menjadi dasar untuk melakukan tahap pembersihan data sebelum perhitungan statistik teks.

12. Penghapusan Missing Value

```
#Extract dataset
import pandas as pd

# Load the dataset
data = pd.read_csv('dataset.csv')

# Menghapus Data Kosong
data = data.dropna(subset=[text_column])
data[text_column].isnull().sum()
```

[18] ✓ 1.2s

```
np.int64(0)
```

Gambar 12. Penghapusan Missing Value

Penjelasan: Data yang memiliki nilai kosong pada kolom teks dihapus menggunakan fungsi dropna(). Setelah proses tersebut, hasil pemeriksaan kembali menunjukkan tidak terdapat nilai kosong pada kolom teks.

13. Pemeriksaan Data Duplikat

```
#Extract dataset
import pandas as pd

# Load the dataset
data = pd.read_csv('dataset.csv')

#Memeriksa Duplikasi
data.duplicated().sum()
data[text_column].duplicated().sum()
```

[19] ✓ 1.5s

```
np.int64(102236)
```

Gambar 13. Pemeriksaan Data Duplikat

Penjelasan: Duplikasi diperiksa menggunakan fungsi duplicated(), baik pada keseluruhan baris maupun secara khusus pada kolom teks. Hasil pemeriksaan

menunjukkan adanya data teks yang berulang sehingga diperlukan proses pembersihan duplikasi.

14. Penghapusan Data Duplikat

```
#Extract dataset
import pandas as pd

# Load the dataset
data = pd.read_csv('dataset.csv')

# Menghapus Duplikasi
data = data.drop_duplicates(subset=[text_column])
```

[21] ✓ 1.1s

Gambar 14. Penghapusan Data Duplikat

Penjelasan: Data duplikat pada kolom teks dihapus menggunakan fungsi drop_duplicates(). Tahap ini bertujuan mengurangi pengulangan komentar yang identik sehingga corpus lebih sesuai untuk analisis.

15. Perhitungan Jumlah Karakter

```
# Menghitung Jumlah Karakter
data['jumlah_karakter'] = data[text_column].str.len()
print("Jumlah karakter per baris:")
print(data["jumlah_karakter"])
```

[26] ✓ 1.1s

```
Jumlah karakter per baris:
0         30.0
1         11.0
2         57.0
3         42.0
4          3.0
          ...
127119    13.0
127120    34.0
127123    26.0
127125    73.0
134024     7.0
Name: jumlah_karakter, Length: 117815, dtype: float64
```

Gambar 15. Perhitungan Jumlah Karakter

Penjelasan: Jumlah karakter pada setiap komentar dihitung dan disimpan pada kolom jumlah_karakter. Nilai tersebut digunakan sebagai salah satu ukuran panjang teks dalam corpus.

16. Perhitungan Jumlah Kata

```
    # Menghitung Jumlah Kata
    data['jumlah_kata'] = data[text_column].str.split().str.len()
    print("Jumlah kata per baris:")
    print(data["jumlah_kata"])
[27]  ✓ 1.9s
...  Jumlah kata per baris:
    0          7.0
    1          2.0
    2         14.0
    3          7.0
    4          1.0
              ...
    220046     1.0
    220047     1.0
    220048     1.0
    220049     1.0
    220050     1.0
    Name: jumlah_kata, Length: 220051, dtype: float64
```

Gambar 16. Perhitungan Jumlah Kata

Penjelasan: Jumlah kata pada setiap komentar dihitung berdasarkan pemisahan teks menggunakan fungsi split(). Hasil perhitungan disimpan pada kolom jumlah_kata sebagai dasar analisis statistik panjang teks.

17. Statistik Jumlah Kata

```
#Statistik jumlah kata
print("Statistik Jumlah Kata:")
print(data["jumlah_kata"].describe())
```

```
[39]  ✓ 1.9s

Statistik Jumlah Kata:
count    220051.000000
mean          8.720842
std           9.963053
min           0.000000
25%           4.000000
50%           6.000000
75%          11.000000
max         706.000000
Name: jumlah_kata, dtype: float64
```

Gambar 17. Statistik Jumlah Kata

Penjelasan: Statistik deskriptif pada kolom jumlah_kata digunakan untuk melihat karakteristik distribusi panjang komentar. Informasi yang ditampilkan meliputi jumlah data, rata-rata, simpangan baku, nilai minimum, kuartil, dan nilai maksimum.

18. Tokenisasi Kalimat

```
nltk.download('punkt_tab')

from nltk.tokenize import sent_tokenize
text = "Saya belajar NLP. NLP sangat menarik."

sentences = sent_tokenize(text)

print(sentences)
```

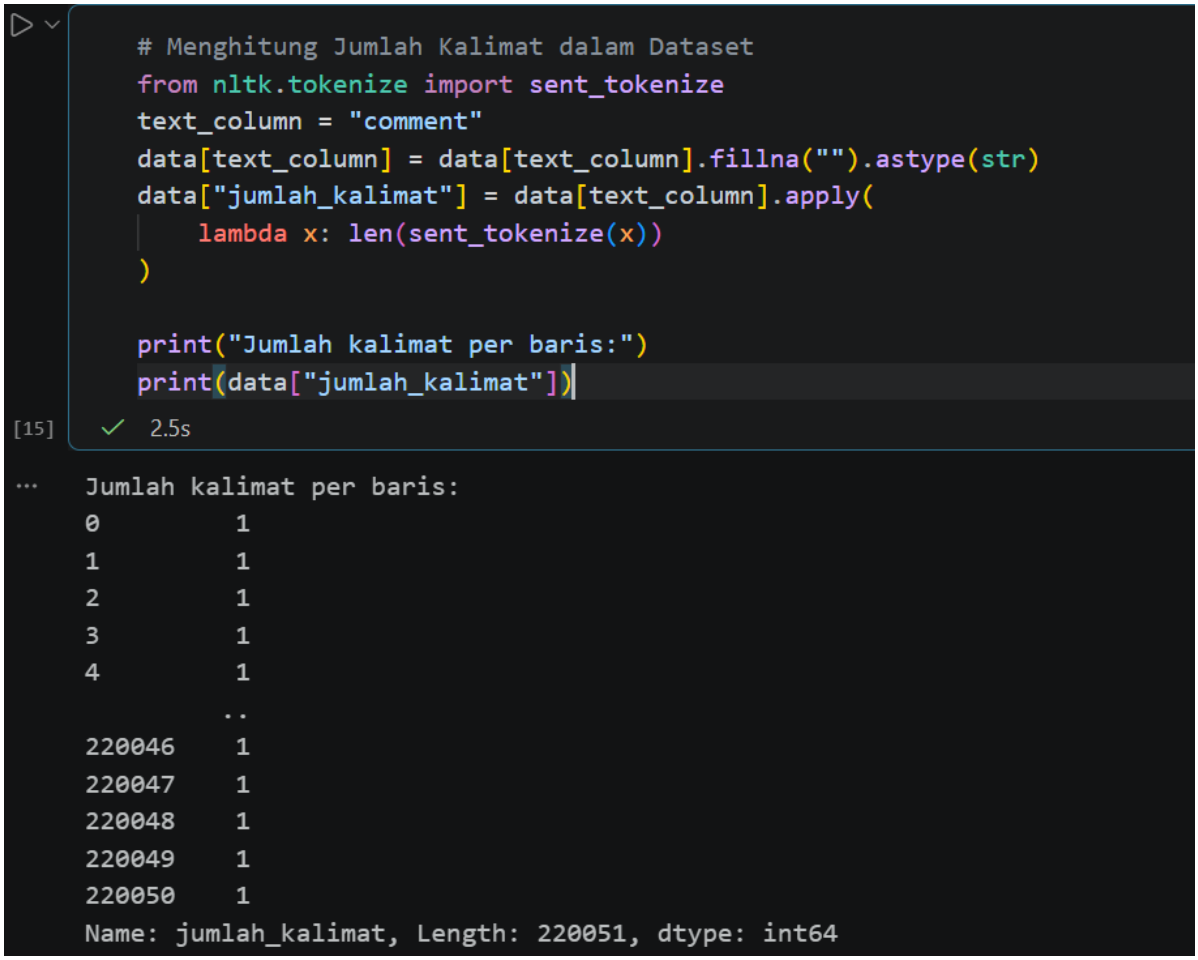
```
[3]  ✓ 2.5s

Requirement already satisfied: nltk in c:\users\acer\appdata\local\packages\pythonsoftwarefoundation.py
Requirement already satisfied: defusedxml in c:\users\acer\appdata\local\packages\pythonsoftwarefoundat
Requirement already satisfied: click in c:\users\acer\appdata\local\packages\pythonsoftwarefoundation.p
Requirement already satisfied: joblib in c:\users\acer\appdata\local\packages\pythonsoftwarefoundation.
Requirement already satisfied: regex>=2021.8.3 in c:\users\acer\appdata\local\packages\pythonsoftwarefo
Requirement already satisfied: tqdm in c:\users\acer\appdata\local\packages\pythonsoftwarefoundation.py
Requirement already satisfied: colorama in c:\users\acer\appdata\local\packages\pythonsoftwarefoundatio
Note: you may need to restart the kernel to use updated packages.
['Saya belajar NLP.', 'NLP sangat menarik.']
```

Gambar 18. Tokenisasi Kalimat

Penjelasan: Tokenisasi kalimat dilakukan menggunakan sent_tokenize dari NLTK. Proses ini memisahkan teks berdasarkan batas kalimat sehingga jumlah kalimat pada setiap dokumen dapat dihitung secara sistematis.

19. Perhitungan Jumlah Kalimat



```
# Menghitung Jumlah Kalimat dalam Dataset
from nltk.tokenize import sent_tokenize
text_column = "comment"
data[text_column] = data[text_column].fillna("").astype(str)
data["jumlah_kalimat"] = data[text_column].apply(
    lambda x: len(sent_tokenize(x))
)

print("Jumlah kalimat per baris:")
print(data["jumlah_kalimat"])
```

[15] ✓ 2.5s

```
Jumlah kalimat per baris:
0         1
1         1
2         1
3         1
4         1
         ..
220046    1
220047    1
220048    1
220049    1
220050    1
Name: jumlah_kalimat, Length: 220051, dtype: int64
```

Gambar 19. Perhitungan Jumlah Kalimat

Penjelasan: Jumlah kalimat pada setiap komentar dihitung dengan menerapkan sent_tokenize pada seluruh kolom teks. Hasilnya disimpan pada kolom jumlah_kalimat sebagai bagian dari statistik deskriptif corpus.

20. Statistik Corpus

```
# Melihat Statistik Corpus
print("Jumlah dokumen :", len(data))
print("Jumlah kata    :", data["jumlah_kata"].sum())
print("Jumlah kalimat :", data["jumlah_kalimat"].sum())
print("Jumlah karakter:", data["jumlah_karakter"].sum())
```

[20] ✓ 0.0s

```
Jumlah dokumen : 220051
Jumlah kata    : 1919030
Jumlah kalimat : 253903
Jumlah karakter: 11582153
```

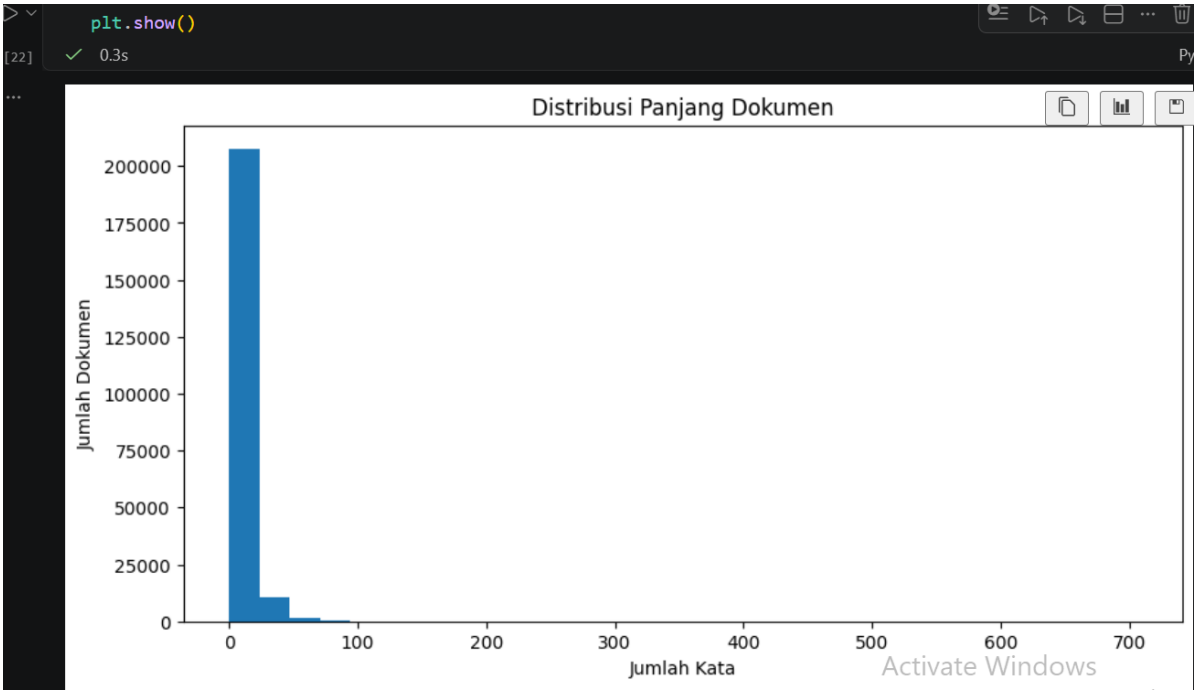
Gambar 20. Statistik Corpus

Penjelasan: Statistik corpus dihitung untuk memperoleh gambaran kuantitatif mengenai keseluruhan data. Hasil yang ditampilkan mencakup jumlah dokumen, jumlah kata, jumlah kalimat, dan jumlah karakter.

21. Distribusi Panjang Dokumen

```
plt.show()
```

[22] ✓ 0.3s



Gambar 21. Distribusi Panjang Dokumen

Penjelasan: Histogram digunakan untuk memvisualisasikan distribusi panjang dokumen berdasarkan jumlah kata. Visualisasi menunjukkan konsentrasi data pada komentar dengan jumlah kata relatif sedikit serta sebagian kecil komentar dengan panjang yang lebih besar.

E. Tugas Analisis

Dataset yang dianalisis merupakan kumpulan komentar TikTok mengenai Koperasi Desa Merah Putih.

1. Apa nama dataset?
   Dataset yang digunakan adalah dataset komentar TikTok mengenai Koperasi Desa Merah Putih.
2. Dataset berasal dari mana?
   Dataset berasal dari komentar pada platform TikTok yang berkaitan dengan topik Koperasi Desa Merah Putih. Dalam praktikum, setiap komentar diperlakukan sebagai satu dokumen teks.
3. Berapa jumlah dokumen?
   Jumlah dokumen adalah 220.051 dokumen.
4. Berapa jumlah kolom?
   Dataset memiliki 8 kolom, yaitu video_id, comment_id, parent_comment_id, level, username, nickname, comment, dan create_time.
5. Apa nama kolom teks?
   Kolom teks yang digunakan adalah comment, karena kolom tersebut berisi isi komentar yang menjadi objek utama analisis NLP.
6. Apakah terdapat missing value?
   Ya, terdapat missing value. Pada kolom comment terdapat 304 data kosong. Selain itu, beberapa kolom metadata juga memiliki nilai kosong. Pemeriksaan missing value merupakan bagian dari proses *data cleaning* yang dilakukan dalam praktikum.
7. Apakah terdapat data duplikat?
   Ya, terdapat data duplikat. Berdasarkan pemeriksaan pada kolom teks, ditemukan 102.236 data duplikat. Data tersebut kemudian perlu ditangani menggunakan drop_duplicates() agar komentar yang identik tidak dihitung berulang dalam analisis.
8. Berapa rata-rata jumlah kata per dokumen?
   Rata-rata jumlah kata adalah sekitar 8,72 kata per dokumen.
9. Berapa dokumen terpendek?
   Dokumen terpendek memiliki 1 kata.

10. Berapa dokumen terpanjang?

    Dokumen terpanjang memiliki sekitar 706 kata.

11. Apakah dataset memiliki label?

    Tidak terdapat kolom label yang digunakan dalam dataset. Kolom yang tersedia merupakan metadata dan kolom teks komentar. Oleh karena itu, dataset ini pada praktikum tersebut tidak digunakan sebagai dataset berlabel untuk klasifikasi.

12. Bagaimana distribusi labelnya?

    Karena tidak terdapat kolom label, distribusi label tidak dapat dihitung. Analisis yang dilakukan berfokus pada karakteristik corpus, seperti jumlah dokumen, karakter, kata, kalimat, dan distribusi panjang komentar.

13. Apa karakteristik bahasa yang ditemukan?

    Berdasarkan contoh komentar dan karakteristik corpus, bahasa yang ditemukan cenderung berupa bahasa informal media sosial. Terdapat variasi penulisan, penggunaan singkatan, kemungkinan kesalahan ejaan, tanda baca yang tidak selalu baku, serta penggunaan elemen seperti stiker. Karakteristik tersebut sesuai dengan sifat data teks media sosial yang umumnya tidak terstruktur.

Kesimpulan Analisis

Secara keseluruhan, corpus terdiri atas 220.051 dokumen dengan 8 kolom, dengan comment sebagai kolom utama untuk analisis teks. Corpus mengandung missing value dan duplikasi, sehingga diperlukan tahap *data cleaning* sebelum analisis lebih lanjut. Rata-rata panjang komentar relatif pendek, yaitu sekitar 8,72 kata per dokumen, dengan rentang dari 1 hingga 706 kata. Karakteristik bahasa menunjukkan pola bahasa informal yang umum ditemukan pada komentar media sosial. Hal ini sejalan dengan tujuan eksplorasi corpus, yaitu memahami struktur, kualitas, dan karakteristik data sebelum dilakukan pemrosesan NLP lanjutan.

F. Daftar Pustaka

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed., online manuscript). Stanford University.

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

Pandas Development Team. (2026). *pandas documentation: Working with missing data*. pandas.

Pandas Development Team. (2026). *pandas documentation: Indexing and selecting data*. pandas.

Modul	1 — Fundamental dan Text Processing
Topik	Tokenization
Repository	https://github.com/lian180706/NATURAL-LANGUAGE-PROCESSING
Tanggal	-

A. Tujuan

1. Memahami konsep token
2. Melakukan sentence segmentation
3. Melakukan word tokenization
4. Melakukan character tokenization
5. Memahami subword tokenization
6. Membandingkan tokenizer yang berbeda.

B. Requirement

1. Sistem Operasi yang digunakan : Microsoft Windows 11 Pro
2. Browser : Versi 153.0.8010.50 (Build Resmi) (64 bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Natural Language Processing (NLP)

   Natural Language Processing (NLP) merupakan salah satu bidang kecerdasan buatan (*Artificial Intelligence*) yang mempelajari bagaimana komputer dapat memproses dan menganalisis bahasa manusia. NLP digunakan dalam berbagai aplikasi, seperti analisis sentimen, penerjemahan otomatis, chatbot, dan klasifikasi teks. Dalam proses pengolahan bahasa alami, teks perlu dipersiapkan melalui beberapa tahapan agar dapat diproses oleh komputer. Salah satu tahapan dasar tersebut adalah tokenization (Bird, Klein, & Loper, 2009).

2. Pengertian Token dan Tokenization

   Token adalah unit teks yang dihasilkan melalui proses pemisahan teks. Token dapat berupa kata, karakter, tanda baca, atau bagian kata yang lebih kecil. Tokenization merupakan proses membagi teks menjadi unit-unit tersebut agar lebih mudah diproses dan dianalisis dalam sistem NLP.

   Sebagai contoh, kalimat "Saya belajar NLP." dapat dipecah menjadi token ["Saya", "belajar", "NLP", "."]. Pada contoh tersebut, tanda titik dipisahkan sebagai token tersendiri. Hasil tokenisasi dapat berbeda bergantung pada metode dan aturan tokenizer yang digunakan (NLTK Project, n.d.).

3. Jenis-Jenis Tokenization

   Tokenization dapat dilakukan pada beberapa tingkatan sesuai dengan tujuan pengolahan teks.

   a. Sentence Tokenization

      Sentence tokenization atau *sentence segmentation* merupakan proses membagi teks menjadi beberapa kalimat. Proses ini membantu memisahkan satu kalimat dari kalimat lainnya berdasarkan tanda baca dan aturan bahasa. Contohnya, teks "Saya belajar NLP. NLP sangat menarik." dapat dipisahkan menjadi dua kalimat.

   b. Word Tokenization

      Word tokenization merupakan proses membagi teks menjadi token berdasarkan kata dan aturan pemisahan tertentu. Dalam proses ini, tanda baca dapat dipisahkan dari kata yang menyertainya. Contohnya, kalimat

"Saya belajar NLP." dapat menghasilkan empat token, yaitu "Saya", "belajar", "NLP", dan ".".

c. Character Tokenization

Character tokenization merupakan proses membagi teks menjadi karakter individual. Misalnya, teks "NLP" menghasilkan tiga karakter, yaitu "N", "L", dan "P". Metode ini dapat digunakan dalam pengolahan bahasa pada tingkat karakter dan analisis pola penulisan.

d. Subword Tokenization

Subword tokenization merupakan proses membagi kata menjadi unit yang lebih kecil daripada kata utuh. Metode ini digunakan pada berbagai model bahasa modern untuk menangani kata yang jarang muncul dan variasi bentuk kata. Beberapa pendekatan subword tokenization yang dikenal adalah Byte Pair Encoding (BPE), WordPiece, dan SentencePiece (Hugging Face, n.d.).

4. Metode dan Library Tokenization

Dalam praktikum ini, digunakan bahasa pemrograman Python dengan library Natural Language Toolkit (NLTK). NLTK menyediakan fungsi sent_tokenize() untuk memisahkan teks menjadi kalimat dan word_tokenize() untuk melakukan tokenisasi teks menjadi unit-unit kata dan tanda baca sesuai aturan tokenizer.

Selain menggunakan NLTK, pemisahan teks juga dapat dilakukan menggunakan metode bawaan Python, yaitu split(). Metode ini memisahkan teks berdasarkan spasi atau karakter whitespace. Perbedaan aturan pemisahan menyebabkan hasil split() dapat berbeda dari word_tokenize(), khususnya pada teks yang mengandung tanda baca, angka, dan URL. Oleh karena itu, perbandingan kedua metode dapat membantu memahami karakteristik tokenisasi teks (NLTK Project, n.d.).

5. Tokenization pada Dataset Teks

Tokenisasi dataset merupakan proses menerapkan tokenisasi pada kumpulan dokumen teks. Setiap dokumen diubah menjadi daftar token sehingga dapat digunakan untuk analisis lebih lanjut. Dalam praktikum ini, library pandas digunakan untuk membaca dataset, menyimpan hasil tokenisasi, dan menghitung jumlah token pada setiap dokumen.

Jumlah token dapat digunakan untuk menggambarkan panjang dokumen berdasarkan aturan tokenizer yang digunakan. Hasilnya dapat dibandingkan dengan jumlah kata yang dihitung menggunakan metode split(). Perbedaan jumlah tersebut dapat terjadi karena kedua metode menerapkan aturan pemisahan teks yang berbeda.

6. Tantangan Tokenisasi Bahasa Indonesia

   Tokenisasi bahasa Indonesia memiliki beberapa tantangan, terutama ketika diterapkan pada teks informal dari media sosial atau ulasan pengguna. Tantangan tersebut meliputi variasi ejaan, penggunaan imbuhan, singkatan, campuran bahasa, angka, URL, dan tanda baca berulang.

   Contohnya, kata "tidak", "nggak", "gak", dan "ga" dapat memiliki makna serupa, tetapi bentuk penulisannya berbeda. Selain itu, singkatan seperti "yg" dan "dgn" tidak otomatis diubah menjadi "yang" dan "dengan" melalui proses tokenisasi biasa. Penyeragaman bentuk kata tersebut termasuk dalam proses normalisasi teks, yang berbeda dari tokenization.

   Oleh karena itu, pemilihan tokenizer perlu mempertimbangkan karakteristik bahasa dan jenis dataset yang digunakan agar hasil pemrosesan sesuai dengan kebutuhan analisis.

7. Hubungan Tokenization dengan Vocabulary

   Vocabulary merupakan kumpulan token yang dikenali oleh suatu model NLP. Setelah teks ditokenisasi, token dapat dipetakan menjadi ID numerik agar dapat digunakan sebagai masukan model. Isi dan ukuran vocabulary bergantung pada tokenizer dan metode pembentukan vocabulary yang digunakan.

   Pada tokenisasi berbasis kata, kata yang tidak terdapat dalam vocabulary dapat menimbulkan masalah *out-of-vocabulary*. Subword tokenization membantu mengurangi masalah tersebut dengan membagi kata menjadi unit yang lebih kecil. Pendekatan ini memungkinkan model menangani berbagai bentuk kata tanpa harus menyimpan setiap kata sebagai entri tersendiri dalam vocabulary (Hugging Face, n.d.).

D. Implementasi

```
import pandas as pd
import nltk
from pathlib import Path
from nltk.tokenize import sent_tokenize, word_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")

print("Library siap digunakan.")
```

[1] ✓ 2.6s

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\acer\AppData\Roaming\nltk_data...
Library siap digunakan.
[nltk_data]   Package punkt_tab is already up-to-date!
```

Gambar 1. Import Library dan Persiapan NLTK.

Program mengimpor pandas dan NLTK, modul Path, serta fungsi sent_tokenize() dan word_tokenize(). Resource punkt dan punkt_tab diperiksa/diunduh agar fungsi tokenisasi NLTK dapat digunakan. Output menunjukkan bahwa resource sudah tersedia dan library siap digunakan.

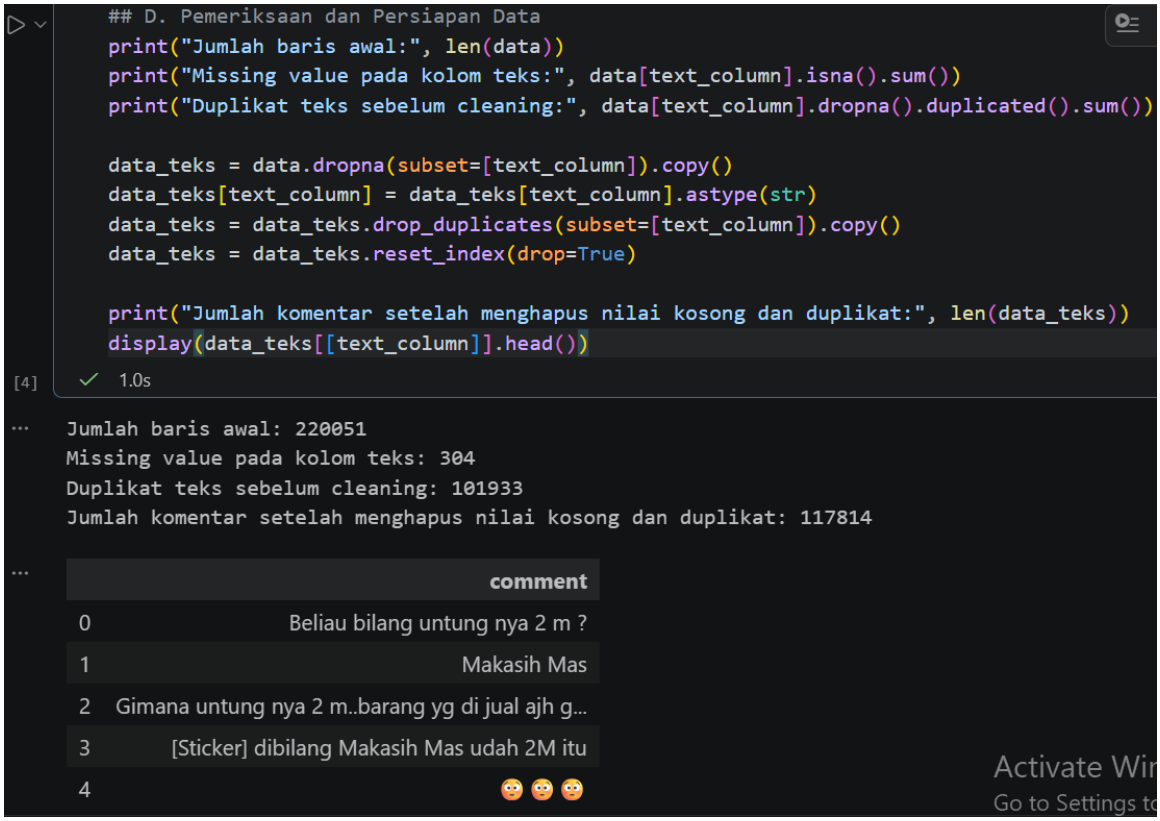
```
Lokasi dataset: dataset.csv
Ukuran dataset: (220051, 8)
Kolom: ['video_id', 'comment_id', 'parent_comment_id', 'level', 'username', 'nickname', 'comment', 'create_time
```

	video_id	comment_id	parent_comment_id	level	username	nickname	comment	create_time
0	7.526460e+18	7.643300e+18	NaN	0.0	jbm123jayabetonmandiri	Christian sudartio	Beliau bilang untung nya 2 m ?	5/24/2026 10:58
1	7.526460e+18	7.643360e+18	7.643300e+18	1.0	sahrulmustakimm	sahrul	Makasih Mas	5/24/2026 14:47
2	7.526460e+18	7.643370e+18	7.643300e+18	1.0	jbm123jayabetonmandiri	Christian sudartio	Gimana untung nya 2 m..barang yg di jual ajh g...	5/24/2026 15:13
3	7.526460e+18	7.643370e+18	7.643300e+18	1.0	sahrulmustakimm	sahrul	[Sticker] dibilang Makasih Mas udah 2M itu	5/24/2026 15:35
4	7.526460e+18	7.643370e+18	7.643300e+18	1.0	jbm123jayabetonmandiri	Christian sudartio		5/24/2026 15:12

Activate Windows

Gambar 2. Load Dataset dan Pemeriksaan Struktur Data.

Dataset dimuat dari berkas dataset.csv. Ukuran awal dataset adalah 220.051 baris dan 8 kolom, yaitu video_id, comment_id, parent_comment_id, level, username, nickname, comment, dan create_time. Tampilan beberapa baris pertama membantu mengenali struktur dan contoh isi dataset.



```
## D. Pemeriksaan dan Persiapan Data
print("Jumlah baris awal:", len(data))
print("Missing value pada kolom teks:", data[text_column].isna().sum())
print("Duplikat teks sebelum cleaning:", data[text_column].dropna().duplicated().sum())

data_teks = data.dropna(subset=[text_column]).copy()
data_teks[text_column] = data_teks[text_column].astype(str)
data_teks = data_teks.drop_duplicates(subset=[text_column]).copy()
data_teks = data_teks.reset_index(drop=True)

print("Jumlah komentar setelah menghapus nilai kosong dan duplikat:", len(data_teks))
display(data_teks[[text_column]].head())
```

[4] ✓ 1.0s

```
Jumlah baris awal: 220051
Missing value pada kolom teks: 304
Duplikat teks sebelum cleaning: 101933
Jumlah komentar setelah menghapus nilai kosong dan duplikat: 117814
```

	comment
0	Beliau bilang untung nya 2 m ?
1	Makasih Mas
2	Gimana untung nya 2 m..barang yg di jual ajh g...
3	[Sticker] dibilang Makasih Mas udah 2M itu
4	😳😳😳

Activate Wir
Go to Settings to

Gambar 3. Pemeriksaan dan Pembersihan Dataset.

Pemeriksaan menemukan 304 nilai kosong pada kolom teks dan 101.933 teks duplikat. Baris dengan teks kosong dihapus, lalu duplikat berdasarkan kolom komentar dihilangkan. Setelah pembersihan, tersisa 117.814 komentar untuk analisis berikutnya.

```
## E. Sentence Segmentation
text = (
    "Saya belajar NLP. "
    "NLP sangat menarik. "
    "Saya ingin menjadi NLP Engineer."
)

sentences = sent_tokenize(text)
for i, sentence in enumerate(sentences, start=1):
    print(f"Kalimat {i}: {sentence}")
print("Jumlah kalimat:", len(sentences))
```

[5] ✓ 0.1s

```
Kalimat 1: Saya belajar NLP.
Kalimat 2: NLP sangat menarik.
Kalimat 3: Saya ingin menjadi NLP Engineer.
Jumlah kalimat: 3
```

Gambar 4. Sentence Segmentation.

Fungsi sent_tokenize() memisahkan teks menjadi kalimat. Contoh menghasilkan tiga kalimat: “Saya belajar NLP.”, “NLP sangat menarik.”, dan “Saya ingin menjadi NLP Engineer.” Hasil ini menunjukkan bahwa teks dapat dipecah berdasarkan batas kalimat.

```
## F. Word Tokenization
text = "Saya belajar Natural Language Processing."
tokens = word_tokenize(text)

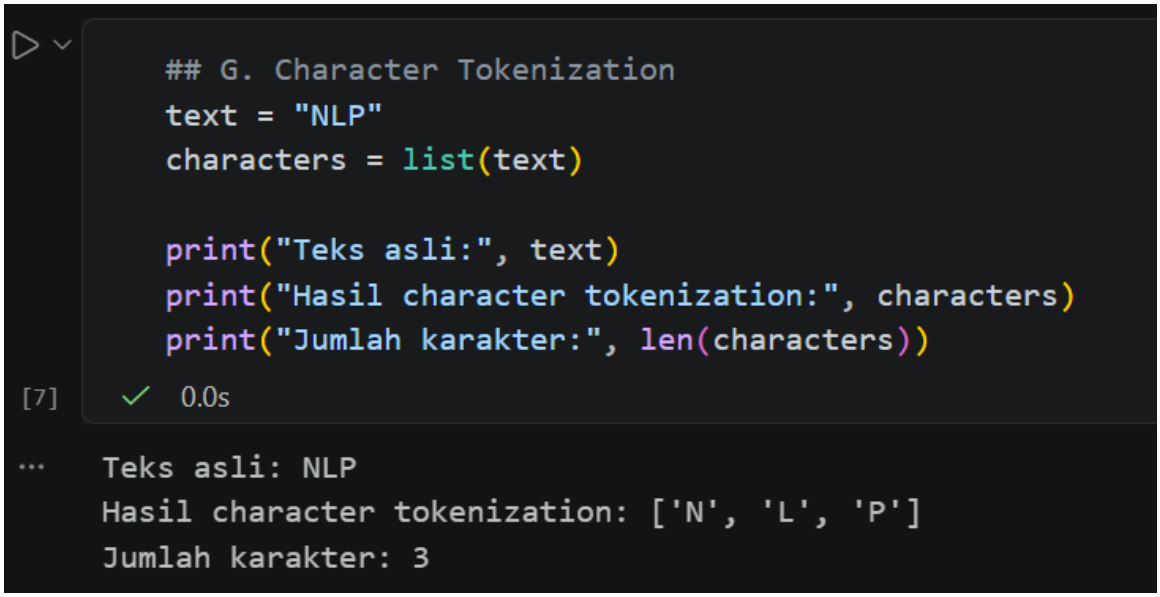
print("Teks asli:", text)
print("Hasil tokenisasi:", tokens)
print("Jumlah token:", len(tokens))
```

[6] ✓ 0.0s

```
Teks asli: Saya belajar Natural Language Processing.
Hasil tokenisasi: ['Saya', 'belajar', 'Natural', 'Language', 'Processing', '.']
Jumlah token: 6
```

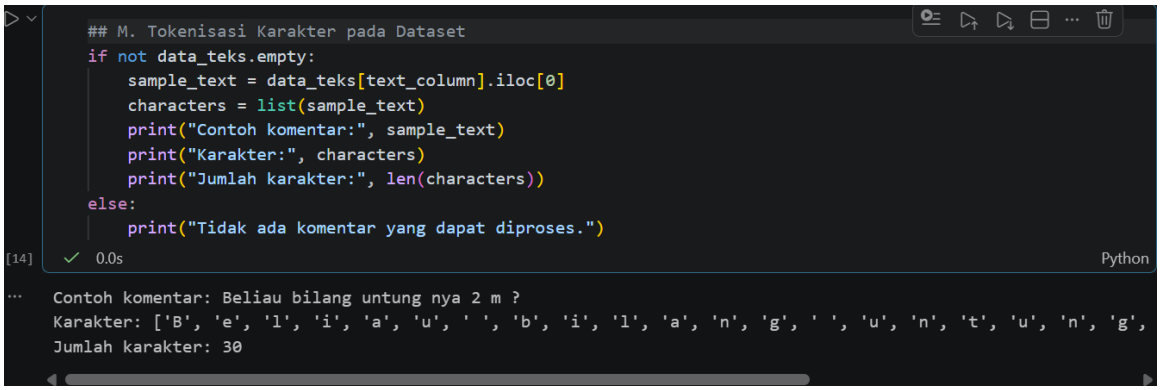
Gambar 5. Word Tokenization dengan NLTK.

Fungsi word_tokenize() memecah teks “Saya belajar Natural Language Processing.” menjadi token kata dan tanda baca. Output menunjukkan enam token, termasuk tanda titik sebagai token tersendiri.



Gambar 6. Character Tokenization pada Teks Contoh.

Teks "NLP" diubah menjadi karakter individual, yaitu N, L, dan P. Jumlah karakter yang dihasilkan adalah tiga. Contoh ini menunjukkan tokenisasi pada tingkat karakter.



Gambar 7. Penerapan Character Tokenization pada Komentar Dataset.

Program mengambil satu komentar dari dataset dan mengubahnya menjadi daftar karakter menggunakan list(). Output menampilkan karakter komentar satu per satu serta jumlah karakternya. Ini merupakan penerapan tambahan tokenisasi karakter pada data komentar nyata.

```
## H. Tokenisasi Dataset
data_teks["tokens"] = data_teks[text_column].apply(word_tokenize)
data_teks["jumlah_token"] = data_teks["tokens"].apply(len)

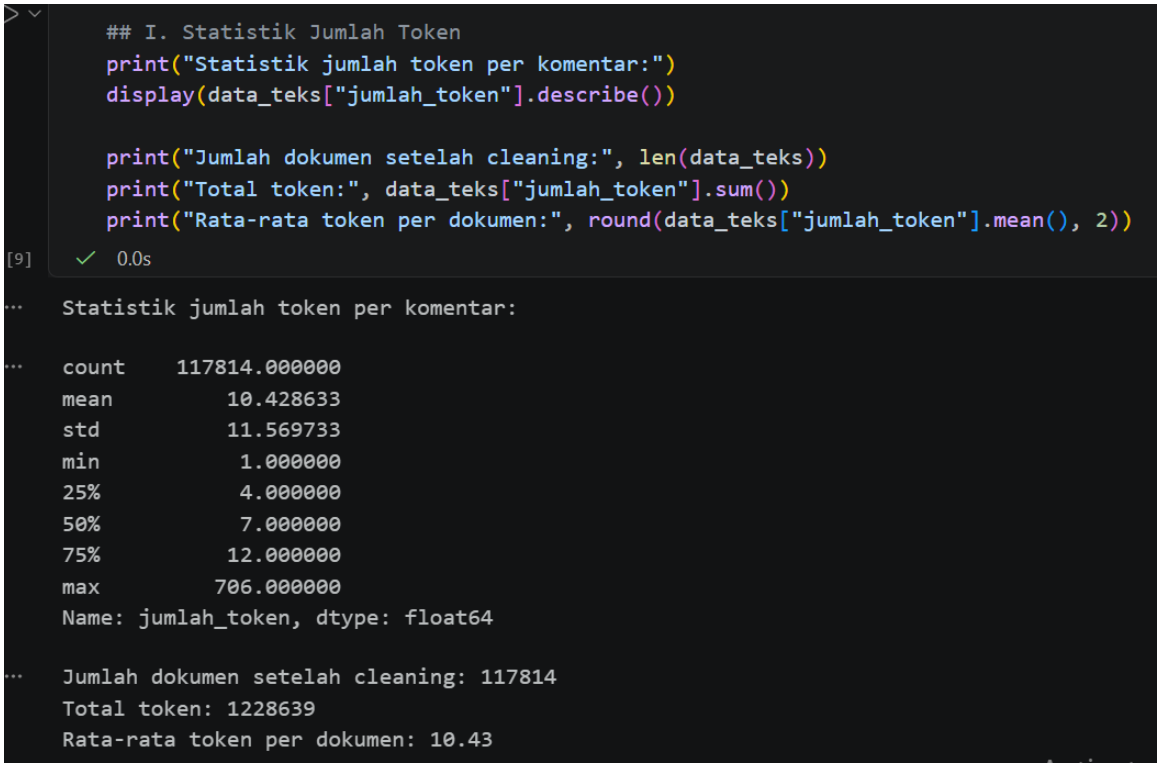
display(data_teks[[text_column, "tokens", "jumlah_token"]].head(10))
```

[8] ✓ 28.8s

	comment	tokens	jumlah_token
0	Beliau bilang untung nya 2 m ?	[Beliau, bilang, untung, nya, 2, m, ?]	7
1	Makasih Mas	[Makasih, Mas]	2
2	Gimana untung nya 2 m..barang yg di jual ajh g...	[Gimana, untung, nya, 2, m, .., barang, yg, di...	16
3	[Sticker] dibilang Makasih Mas udah 2M itu	[[, Sticker,], dibilang, Makasih, Mas, udah, ...	9
4	😳😳😳	[😳😳😳]	1
5	Kalo niat membantu UMKM, harusnya koprasi mera...	[Kalo, niat, membantu, UMKM, ,, harusnya, kopr...	122
6	sabar mbak paling 6 bulan dah tutup	[sabar, mbak, paling, 6, bulan, dah, tutup]	7
7	kalo ternyata harga murah, saya harap bisa ber...	[kalo, ternyata, harga, murah, ,, saya, harap,...	11
8	SAYA SETUJU BANG, soalnya saya punya warung 🤩	[SAYA, SETUJU, BANG, ,, soalnya, saya, punya, ...	9
9	Ancen Dancuk 🤡🤡🤡	[Ancen, Dancuk, 🤡🤡🤡]	3

Gambar 8. Tokenisasi Dataset.

Fungsi word_tokenize() diterapkan pada kolom komentar yang telah dibersihkan. Hasil tokenisasi disimpan pada kolom tokens, sedangkan jumlah token setiap komentar disimpan pada kolom jumlah_token. Tabel memperlihatkan sepuluh contoh komentar beserta token dan jumlah tokennya.



Gambar 9. Statistik Jumlah Token.

Statistik deskriptif menunjukkan 117.814 komentar dengan rata-rata 10,43 token per komentar. Nilai minimum adalah 1 token, median 7 token, dan maksimum 706 token. Total token yang tercatat pada dataset adalah 1.228.639.

```
## J. Perbandingan Jumlah Kata dan Jumlah Token
data_teks["jumlah_kata"] = data_teks[text_column].apply(lambda x: len(x.split()))

display(data_teks[[text_column, "jumlah_kata", "jumlah_token"]].head(10))
display(data_teks[["jumlah_kata", "jumlah_token"]].describe())
```

[11] ✓ 0.3s

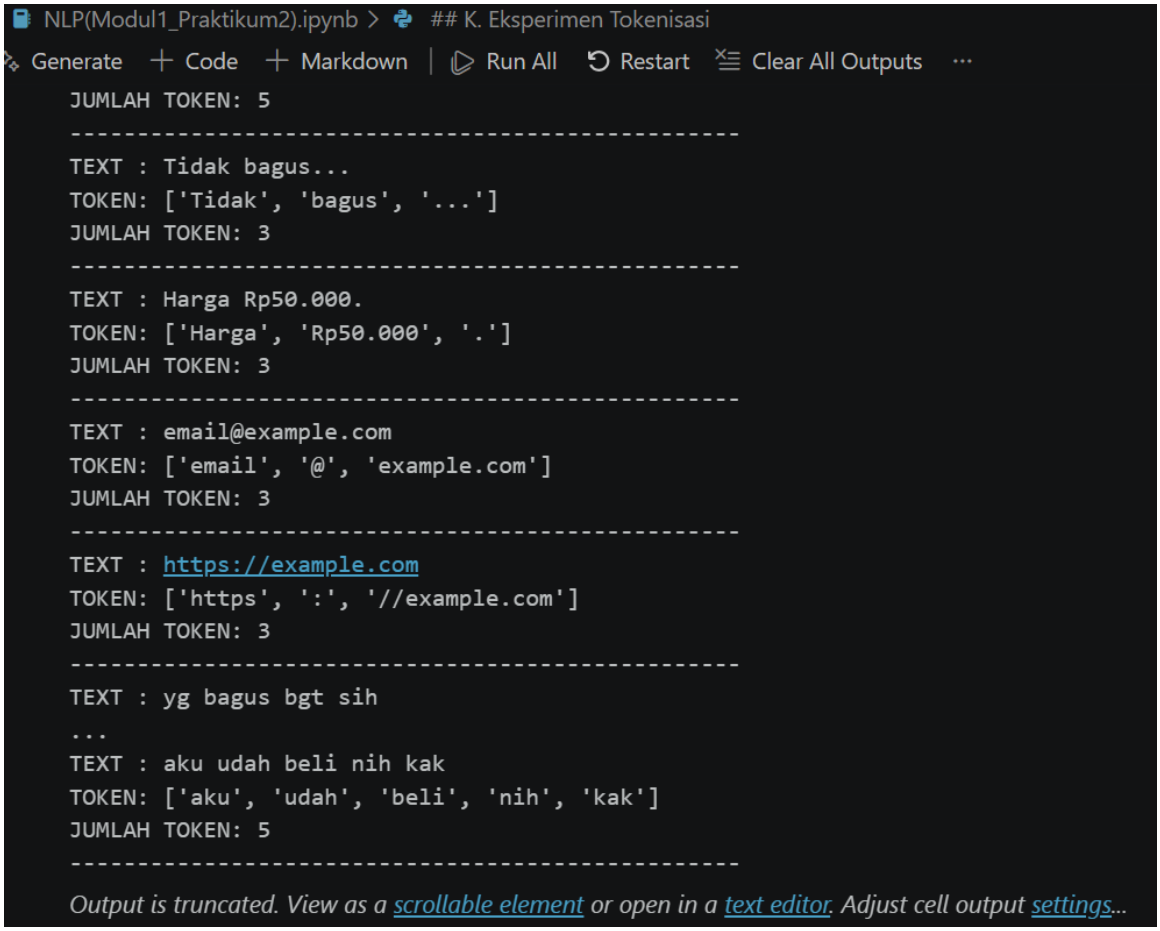
	comment	jumlah_kata	jumlah_token
0	Beliau bilang untung nya 2 m ?	7	7
1	Makasih Mas	2	2
2	Gimana untung nya 2 m..barang yg di jual ajh g...	14	16
3	[Sticker] dibilang Makasih Mas udah 2M itu	7	9
4	😳😳😳	1	1
5	Kalo niat membantu UMKM, harusnya koprasi mera...	109	122
6	sabar mbak paling 6 bulan dah tutup	7	7
7	kalo ternyata harga murah, saya harap bisa ber...	9	11
8	SAYA SETUJU BANG, soalnya saya punya warung 😩	8	9
9	Ancen Dancuk 🤡🤡🤡	3	3

	jumlah_kata	jumlah_token
count	117814.000000	117814.000000
mean	9.428480	10.428633
std	10.339934	11.569733
min	1.000000	1.000000
25%	4.000000	4.000000
50%	7.000000	7.000000
75%	11.000000	12.000000
max	706.000000	706.000000

Gambar 10. Membandingkan Jumlah Kata dan Jumlah Token .

Jumlah kata dihitung dengan memisahkan teks berdasarkan spasi menggunakan split(), lalu dibandingkan dengan jumlah token yang dihasilkan NLTK. Dua screenshot di atas merupakan tampilan berurutan dari bagian perbandingan dan ringkasan statistiknya. Hasil menunjukkan rata-rata 9,43 kata dengan split() dan 10,43 token dengan NLTK pada 117.814 komentar. Perbedaan muncul karena NLTK dapat memisahkan tanda baca menjadi token tersendiri.

```
TEXT : Saya suka produk ini.
TOKEN: ['Saya', 'suka', 'produk', 'ini', '.']
JUMLAH TOKEN: 5
--------------------------------------------------
TEXT : Bagus banget!!!
TOKEN: ['Bagus', 'banget', '!', '!', '!']
JUMLAH TOKEN: 5
--------------------------------------------------
TEXT : Tidak bagus...
TOKEN: ['Tidak', 'bagus', '...']
JUMLAH TOKEN: 3
--------------------------------------------------
TEXT : Harga Rp50.000.
TOKEN: ['Harga', 'Rp50.000', '.']
JUMLAH TOKEN: 3
--------------------------------------------------
TEXT : email@example.com
TOKEN: ['email', '@', 'example.com']
JUMLAH TOKEN: 3
--------------------------------------------------
TEXT : https://example.com
TOKEN: ['https', ':', '//example.com']
JUMLAH TOKEN: 3
--------------------------------------------------
TEXT : yg bagus bgt sih
...
TEXT : aku udah beli nih kak
```



Gambar 11. Eksperimen Tokenisasi pada Berbagai Teks .

Contoh pengujian mencakup tanda baca berulang, nominal harga, alamat email, URL, dan bahasa informal. Hasilnya menunjukkan bahwa karakteristik teks serta aturan tokenizer memengaruhi jumlah dan bentuk token.

```python
comparison_rows = []
for sample in comparison_texts:
    nltk_tokens = word_tokenize(sample)
    split_tokens = sample.split()
    comparison_rows.append({
        "Teks": sample,
        "NLTK word_tokenize": nltk_tokens,
        "Jumlah token NLTK": len(nltk_tokens),
        "split()": split_tokens,
        "Jumlah hasil split": len(split_tokens)
    })

comparison_df = pd.DataFrame(comparison_rows)
display(comparison_df)
```

[13] ✓ 0.2s

	Teks	NLTK word_tokenize	Jumlah token NLTK	split()	Jumlah hasil split
0	Saya belajar NLP.	[Saya, belajar, NLP, .]	4	[Saya, belajar, NLP.]	3
1	Bagus banget!!!	[Bagus, banget, !, !, !]	5	[Bagus, banget!!!]	2
2	Harga Rp50.000	[Harga, Rp50.000]	2	[Harga, Rp50.000]	2
3	yg bagus bgt sih	[yg, bagus, bgt, sih]	4	[yg, bagus, bgt, sih]	4
4	https://example.com	[https, :, //example.com]	3	[https://example.com]	1

Gambar 12. Perbandingan Tokenizer NLTK word_tokenize() dan split().

Kode membandingkan hasil word_tokenize() dari NLTK dengan split() pada beberapa contoh teks. NLTK dapat memisahkan tanda baca, misalnya titik dan tanda seru, sebagai token tersendiri. Sementara itu, split() umumnya memisahkan teks berdasarkan spasi sehingga tanda baca sering tetap melekat pada teks. Tabel membantu melihat perbedaan jumlah dan susunan token dari kedua metode.

E. Tugas Analisis

1. Apakah setiap kata selalu menghasilkan satu token?

   Tidak selalu. Pada word tokenization, kata umumnya menjadi satu token, tetapi tanda baca dapat menjadi token tersendiri. Pada subword tokenization, satu kata dapat dipecah menjadi beberapa bagian.

2. Bagaimana tokenizer menangani tanda baca?

   NLTK dapat memisahkan tanda baca tertentu dari kata. Misalnya, titik pada akhir kalimat dapat menjadi token tersendiri. Tanda baca berulang juga diproses sesuai aturan tokenizer.

3. Bagaimana tokenizer menangani angka seperti Rp50.000?

   Hasilnya bergantung pada tokenizer. `split()` biasanya mempertahankan `Rp50.000` sebagai satu bagian jika tidak ada spasi, sedangkan NLTK dapat memisahkan tanda baca tertentu. Hasil aktual dapat dilihat pada sel eksperimen.

4. Bagaimana tokenizer menangani URL?

   URL dapat dipertahankan sebagai satu token atau dipecah menjadi beberapa token, bergantung pada aturan tokenizer. Untuk kebutuhan tertentu, URL dapat ditangani dengan aturan khusus.

5. Bagaimana tokenizer menangani singkatan informal seperti yg, dgn, dan kak?

   Tokenizer umumnya memisahkan singkatan berdasarkan batas teks, tetapi tidak otomatis mengubah `yg` menjadi `yang` atau `dgn` menjadi `dengan`. Pengubahan bentuk tersebut merupakan bagian dari normalisasi teks.

6. Apa masalah tokenisasi yang spesifik untuk Bahasa Indonesia?

   Tantangannya meliputi variasi ejaan informal, imbuhan, singkatan, campuran bahasa, angka, URL, serta tanda baca berulang. Bentuk seperti `tidak`, `nggak`, dan `gak` dapat memiliki makna serupa tetapi tetap berupa bentuk teks berbeda.

7. Mengapa subword tokenizer diperlukan pada model Transformer?

   Subword tokenization membantu menangani kata yang jarang muncul atau kata baru dengan memecahnya menjadi unit yang lebih kecil. Cara ini mengurangi ketergantungan pada vocabulary yang hanya berisi kata utuh.

F. Pertanyaan Diskusi

1. Apakah tokenizer yang baik untuk Bahasa Inggris otomatis baik untuk Bahasa Indonesia?

   Tidak selalu. Bahasa Indonesia memiliki karakteristik seperti imbuhan dan variasi ejaan informal yang mungkin berbeda dari Bahasa Inggris. Tokenizer perlu dievaluasi pada bahasa dan jenis data yang digunakan.

2. Apa yang terjadi jika teks tidak di-tokenize terlebih dahulu?

   Teks akan tetap berupa rangkaian karakter yang belum dipecah menjadi unit sesuai kebutuhan analisis. Namun, beberapa model modern memiliki tokenizer internal yang melakukan proses tersebut secara otomatis.

3. Mengapa tanda baca bisa menjadi token tersendiri?

   Tanda baca dapat memberikan informasi tentang batas kalimat, pertanyaan, penekanan, dan ekspresi. Pemisahannya memungkinkan informasi itu diproses secara terpisah.

4. Apa hubungan tokenization dengan vocabulary dalam model NLP?

   Token hasil tokenisasi dipetakan ke ID numerik yang mengacu pada vocabulary model. ID tersebut menjadi representasi yang dapat diproses oleh model.

5. Mengapa model Transformer menggunakan subword tokenization?

   Subword tokenization membantu menangani kata langka dan variasi bentuk kata serta mengurangi masalah kata di luar vocabulary.

G. Daftar Pustaka

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media. https://www.nltk.org/book/

Hugging Face. (n.d.). *Tokenizers Documentation*. https://huggingface.co/docs/tokenizers/

NLTK Project. (n.d.). *NLTK Documentation*. https://www.nltk.org/

Modul	1 — Fundamental dan Text Processing
Topik	Text Normalization
Repository	https://github.com/lian180706/NATURAL-LANGUAGE-PROCESSING
Tanggal	-

A. Tujuan

1. Melakukan Case Folding
2. Membersihkan Teks (Url, Mention, Hashtag, Angka, Punctuation)
3. Menangani Slang Words
4. Melakukan Stopword Removal Bahasa Indonesia
5. Melakukan Stemming Bahasa Indonesia Dengan Sastrawi
6. Memahami Perbedaan Stemming Dan Lemmatization
7. Membangun Preprocessing Pipeline Lengkap

B. Requirement

1. Sistem Operasi yang digunakan : Microsoft Windows 11 Pro
2. Browser : Versi 153.0.8010.50 (Build Resmi) (64 bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Natural Language Processing (NLP)

   Natural Language Processing (NLP) merupakan bidang ilmu yang menggabungkan ilmu komputer, kecerdasan buatan, dan linguistik untuk memungkinkan komputer memproses serta menganalisis bahasa manusia. NLP digunakan dalam berbagai aplikasi, seperti analisis sentimen, klasifikasi teks, penerjemahan otomatis, pencarian informasi, dan chatbot.

   Data teks yang diperoleh dari dunia nyata sering kali memiliki format yang tidak konsisten, seperti penggunaan huruf kapital, singkatan, kesalahan penulisan, tanda baca berlebihan, dan kata tidak baku. Oleh karena itu, diperlukan tahap pemrosesan awal atau *text preprocessing* sebelum data digunakan dalam proses analisis. Bird, Klein, dan Loper (2009) menjelaskan berbagai teknik dasar pemrosesan bahasa, termasuk pengolahan teks mentah, tokenisasi, dan stemming.

2. Text Normalization

   Text normalization merupakan proses mengubah teks ke dalam bentuk yang lebih konsisten agar dapat diproses secara lebih efektif oleh sistem NLP. Normalisasi dapat mencakup perubahan huruf menjadi huruf kecil, penanganan tanda baca, penghapusan elemen yang tidak relevan, serta penyeragaman kata tidak baku.

   Sebagai contoh, kalimat “PELAYANANNYA BAGUUUS BANGET!!!” dapat dinormalisasi menjadi “pelayanannya baguuus banget”. Hasil tersebut sudah memiliki format huruf yang konsisten, tetapi variasi penulisan “baguuus” masih perlu ditangani apabila tujuan analisis mengharuskan normalisasi ejaan.

   Tahapan normalisasi harus disesuaikan dengan karakteristik dataset dan tujuan analisis. Penghapusan informasi secara berlebihan dapat menyebabkan hilangnya makna yang penting. Oleh sebab itu, text normalization tidak hanya bertujuan membersihkan teks, tetapi juga mempertahankan informasi yang relevan.

3. Case Folding

   Case folding adalah proses mengubah seluruh huruf dalam teks menjadi huruf kecil. Tahap ini bertujuan mengurangi perbedaan representasi kata yang hanya disebabkan oleh penggunaan huruf kapital.

Sebagai contoh, kata “Indonesia”, “INDONESIA”, dan “indonesia” akan diubah menjadi “indonesia”. Dengan demikian, ketiganya memiliki bentuk penulisan yang seragam.

Dalam Python, case folding sederhana dapat dilakukan menggunakan metode lower(). Tahap ini berguna untuk menyederhanakan variasi penulisan, meskipun pada kasus tertentu huruf kapital dapat mengandung informasi, misalnya pada singkatan atau nama khusus.

4. Text Cleaning

Text cleaning adalah proses membersihkan elemen teks yang tidak diperlukan sesuai dengan tujuan pemrosesan. Elemen yang dapat ditangani meliputi URL, mention, hashtag, angka, tanda baca, dan spasi berlebihan.

URL dapat dihapus apabila alamat tersebut tidak relevan dengan analisis isi teks. Mention seperti @pengguna dapat dihilangkan ketika identitas akun tidak diperlukan. Hashtag dapat dipertahankan sebagai kata karena berpotensi menunjukkan topik pembicaraan. Angka juga perlu dievaluasi sebelum dihapus karena dapat menunjukkan harga, tahun, jumlah, atau nilai penilaian.

Pembersihan tanda baca dan spasi dapat membantu menghasilkan teks yang lebih konsisten. Namun, keputusan pembersihan harus mempertimbangkan kebutuhan analisis. Dalam analisis sentimen, misalnya, tanda seru berulang dapat menunjukkan intensitas emosi sehingga penghapusannya perlu dilakukan secara hati-hati.

5. Tokenization

Tokenization atau tokenisasi merupakan proses memecah teks menjadi unit-unit yang lebih kecil, seperti kata atau kalimat. Dalam praktikum ini, tokenisasi dilakukan pada tingkat kata sehingga teks dapat diolah sebagai kumpulan token.

Contohnya, kalimat “saya belajar NLP” dapat diubah menjadi tiga token, yaitu ["saya", "belajar", "NLP"].

Tokenisasi merupakan tahap dasar dalam pemrosesan teks karena memungkinkan setiap kata diproses lebih lanjut, misalnya untuk menghapus stopword atau melakukan stemming. Library Natural Language Toolkit (NLTK) menyediakan fungsi word_tokenize() untuk melakukan tokenisasi kata (Bird et al., 2009).

6. Stopword Removal

Stopword removal adalah proses menghapus kata-kata tertentu yang dianggap kurang informatif untuk suatu tugas NLP. Contoh kata yang sering diperlakukan sebagai stopword dalam Bahasa Indonesia meliputi "yang", "dan", "di", "ke", serta "dari".

Penghapusan stopword dapat mengurangi jumlah token yang diproses dan membantu menonjolkan kata-kata yang dianggap lebih relevan. Meskipun demikian, tidak semua stopword harus dihapus karena kepentingannya bergantung pada konteks.

Contohnya, kalimat "produk ini tidak bagus" mengandung kata negasi "tidak". Apabila kata tersebut dihapus, kalimat menjadi "produk ini bagus" sehingga makna sentimennya berubah. Oleh karena itu, kata negasi sebaiknya dipertahankan ketika informasi polaritas sentimen diperlukan.

Dalam praktikum ini, daftar stopword diperoleh menggunakan corpus Bahasa Indonesia yang tersedia pada NLTK. Daftar tersebut perlu disesuaikan dengan kebutuhan dataset dan tujuan analisis.

7. Stemming Bahasa Indonesia

Stemming adalah proses mengubah kata berimbuhan menjadi bentuk dasar atau *stem* melalui aturan pemrosesan morfologi. Dalam Bahasa Indonesia, kata dapat memiliki awalan, akhiran, sisipan, atau kombinasi imbuhan.

Contohnya, kata "mempelajari" dapat diubah menjadi "ajar", sedangkan "berlari-lari" dapat diubah menjadi "lari". Proses ini membantu mengurangi variasi kata yang berasal dari bentuk dasar yang sama.

Sastrawi merupakan library yang dapat digunakan untuk melakukan stemming Bahasa Indonesia. Implementasinya memanfaatkan pendekatan stemming berbasis aturan dan kamus kata dasar. Dokumentasi Sastrawi menjelaskan bahwa library tersebut dikembangkan untuk mengubah kata berimbuhan Bahasa Indonesia menjadi bentuk stem.

Meskipun demikian, stemming tidak selalu menghasilkan bentuk yang tepat untuk setiap konteks. Oleh sebab itu, hasilnya perlu diperiksa, terutama pada teks informal, istilah khusus, dan nama produk.

8. Slang Word Normalization

   Slang word normalization adalah proses menyeragamkan kata tidak baku, singkatan, atau bahasa informal menjadi bentuk yang lebih umum. Contoh kata informal yang sering ditemukan pada teks media sosial adalah “gue”, “lu”, “bgt”, “gak”, dan “nggak”.

   Normalisasi dapat dilakukan menggunakan kamus pemetaan kata. Misalnya, “gue” dipetakan menjadi “saya”, “bgt” menjadi “banget”, dan “gak” menjadi “tidak”.

   Tahap ini dapat membantu mengurangi variasi kosakata dalam data teks. Namun, pemetaan harus memperhatikan konteks karena satu bentuk singkatan dapat memiliki arti yang berbeda. Kamus slang yang digunakan juga perlu disesuaikan dengan karakteristik data.

9. Preprocessing Pipeline

   Preprocessing pipeline merupakan rangkaian tahapan pemrosesan teks yang disusun secara sistematis agar setiap data melalui prosedur yang konsisten. Dalam praktikum ini, pipeline mencakup cleaning, tokenisasi, normalisasi slang, stopword removal, dan stemming.

   Urutan tahapan dapat memengaruhi hasil akhir. Sebagai contoh, cleaning dilakukan sebelum tokenisasi agar elemen yang tidak dibutuhkan dapat ditangani terlebih dahulu. Setelah itu, token diproses melalui normalisasi kata, penghapusan stopword, dan stemming.

   Hasil akhir dapat digabungkan kembali menjadi teks menggunakan pemisah spasi dan disimpan dalam format CSV. Penyimpanan tersebut memungkinkan hasil preprocessing digunakan pada tahap analisis selanjutnya tanpa harus mengulang seluruh proses dari awal.

10. Evaluasi Hasil Normalisasi

    Evaluasi hasil normalisasi dilakukan dengan membandingkan teks asli dan teks setelah setiap tahapan preprocessing. Perbandingan ini dapat menunjukkan perubahan format, pengurangan elemen yang tidak relevan, perubahan jumlah token, serta perubahan bentuk kata setelah stemming.

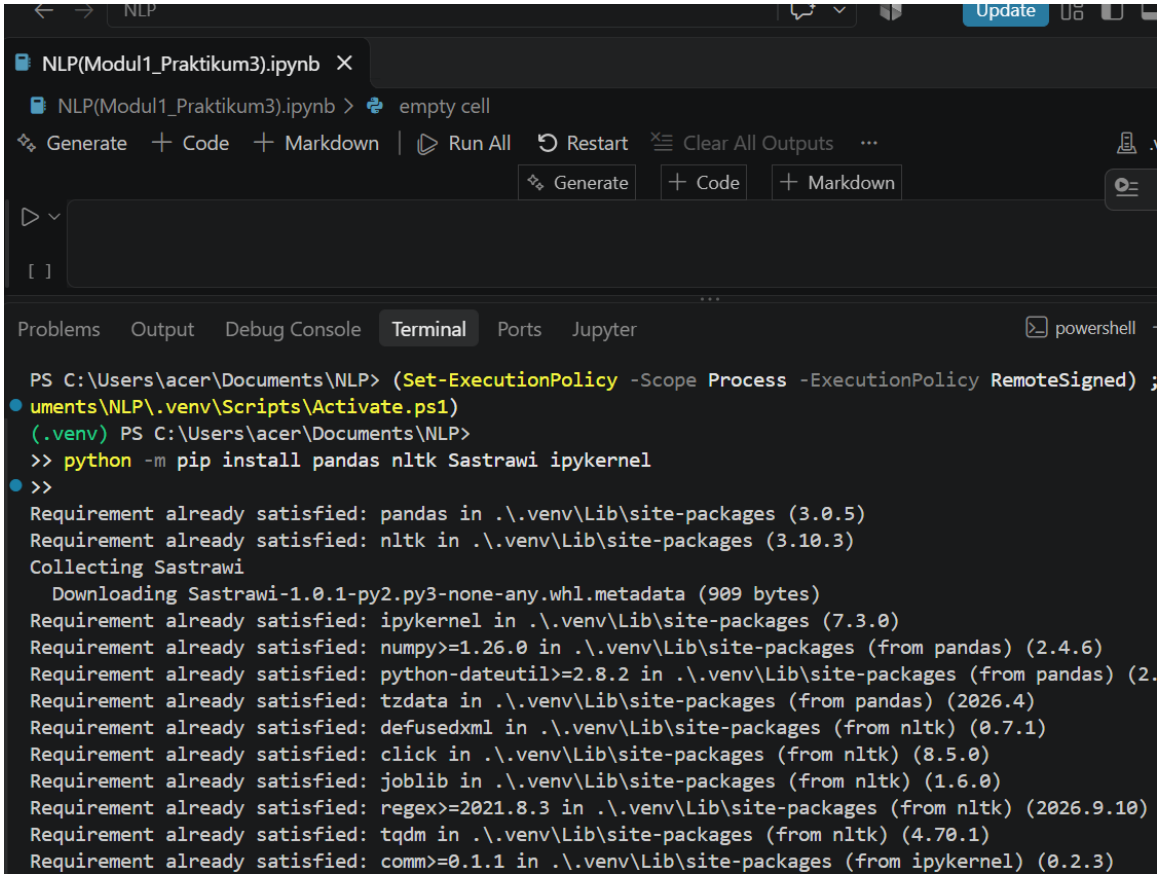
    Pada praktikum ini, minimal 20 data dibandingkan melalui tahapan raw text, case folding, cleaning, tokenisasi, stopword removal, dan stemming. Hasil

tersebut digunakan untuk menilai apakah proses preprocessing telah berjalan sesuai tujuan.

Evaluasi tidak hanya didasarkan pada banyaknya kata yang berhasil dihapus, tetapi juga pada kemampuan mempertahankan informasi yang relevan. Dengan demikian, kualitas preprocessing ditentukan oleh kesesuaiannya dengan karakteristik data dan tujuan tugas NLP.

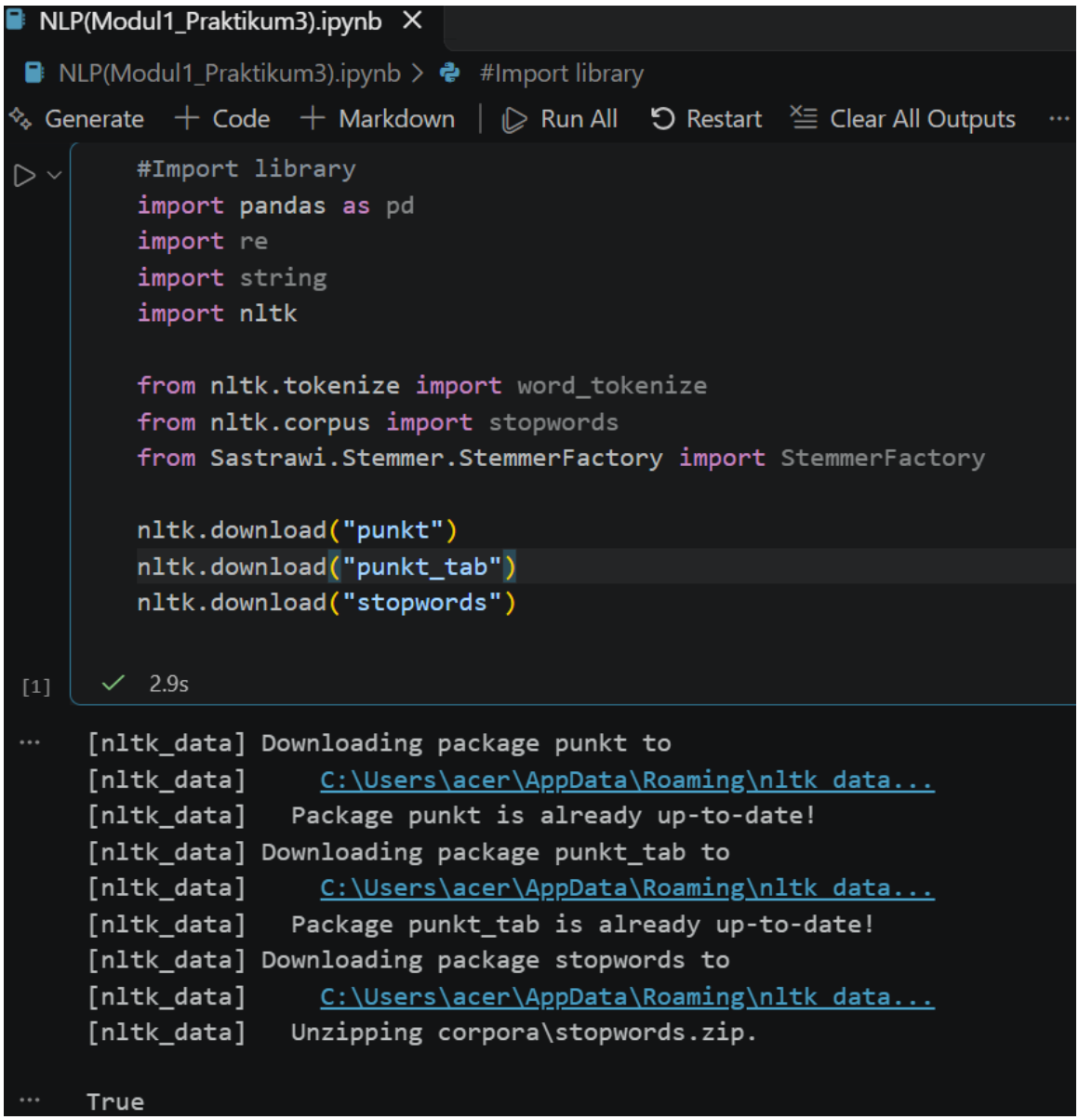
D. Implementasi

Gambar 1. Persiapan lingkungan dan instalasi pustaka



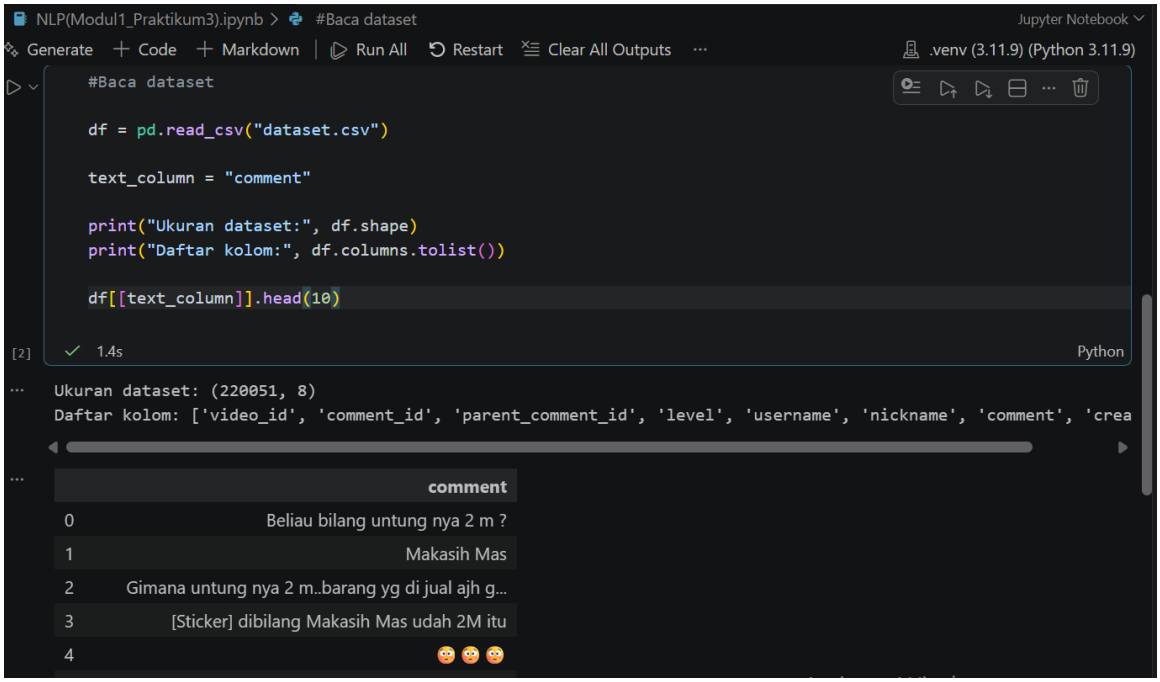
Penjelasan: Gambar ini menunjukkan persiapan lingkungan kerja di Visual Studio Code melalui terminal. Perintah instalasi dijalankan untuk memastikan pustaka yang dibutuhkan tersedia sebelum proses normalisasi teks dilakukan.

Gambar 2. Impor pustaka dan pemuatan dataset



Penjelasan: Pada tahap ini, pustaka Python yang diperlukan diimpor, kemudian dataset komentar dimuat ke dalam DataFrame. Tahap awal ini menyiapkan alat untuk membaca, membersihkan, dan mengolah komentar TikTok tentang Koperasi Desa Merah Putih.

Gambar 3. Pemeriksaan struktur dataset



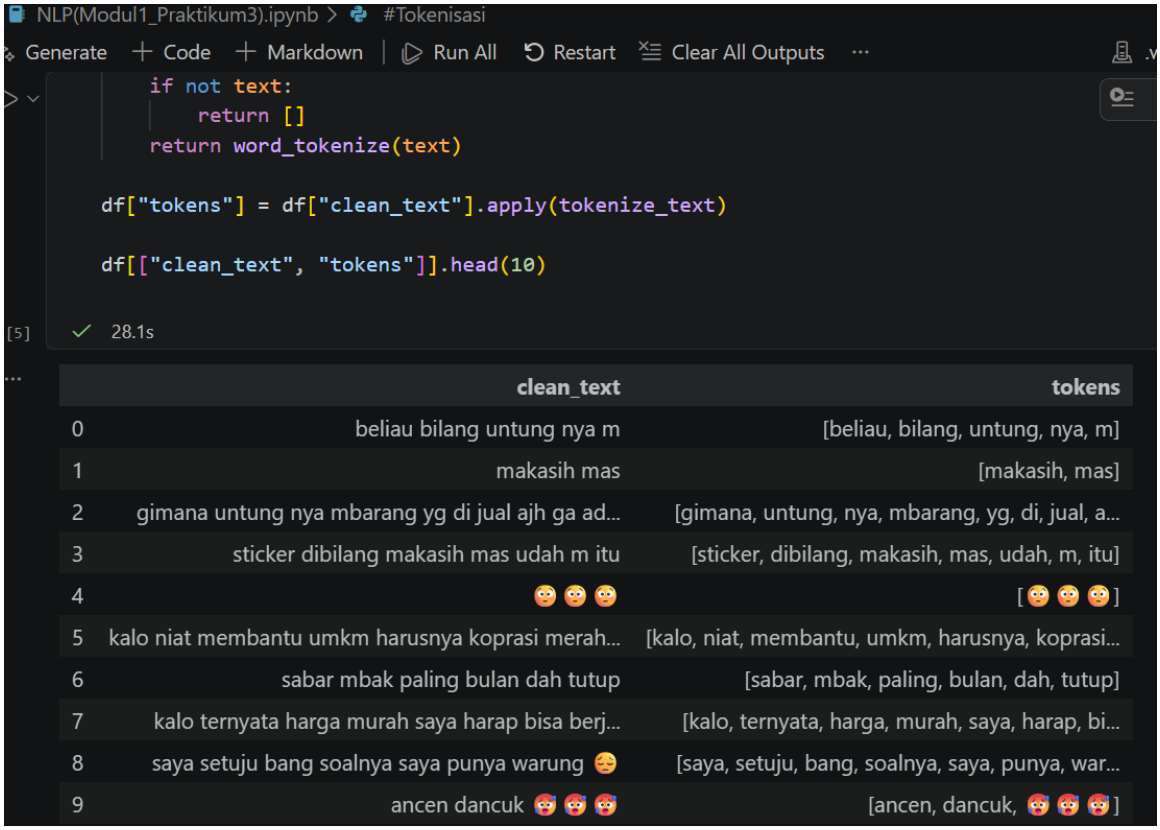
Penjelasan: Tampilan ini memperlihatkan pemeriksaan awal dataset, termasuk bentuk data dan contoh komentar. Pemeriksaan dilakukan untuk memastikan kolom komentar tersedia dan data dapat digunakan pada tahapan preprocessing.

Gambar 4. Implementasi case folding



Penjelasan: Kode membuat fungsi case folding dengan mengubah teks menjadi huruf kecil. Hasilnya ditampilkan berdampingan dengan komentar asli agar perubahan kapitalisasi dapat diperiksa.

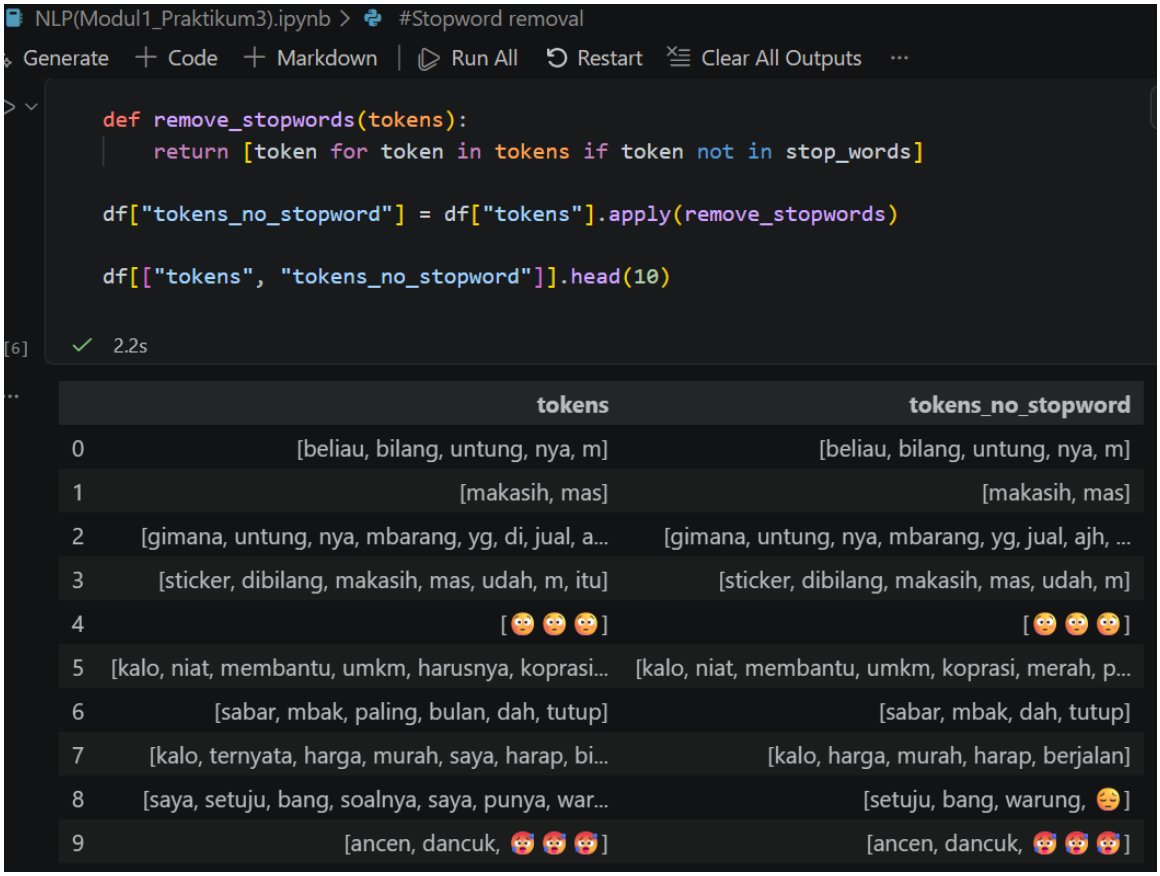
Gambar 5. Pembersihan teks dan tokenisasi



	clean_text	tokens
0	beliau bilang untung nya m	[beliau, bilang, untung, nya, m]
1	makasih mas	[makasih, mas]
2	gimana untung nya mbarang yg di jual ajh ga ad...	[gimana, untung, nya, mbarang, yg, di, jual, a...
3	sticker dibilang makasih mas udah m itu	[sticker, dibilang, makasih, mas, udah, m, itu]
4	😳😳😳	[😳😳😳]
5	kalo niat membantu umkm harusnya koprasi merah...	[kalo, niat, membantu, umkm, harusnya, koprasi...
6	sabar mbak paling bulan dah tutup	[sabar, mbak, paling, bulan, dah, tutup]
7	kalo ternyata harga murah saya harap bisa berj...	[kalo, ternyata, harga, murah, saya, harap, bi...
8	saya setuju bang soalnya saya punya warung 🤩	[saya, setuju, bang, soalnya, saya, punya, war...
9	ancen dancuk 🤡🤡🤡	[ancen, dancuk, 🤡🤡🤡]

Penjelasan: Bagian ini memperlihatkan proses pembersihan teks serta pemecahan teks menjadi token. Pembersihan menangani unsur yang ditetapkan dalam kode, kemudian tokenisasi mengubah komentar menjadi daftar kata.

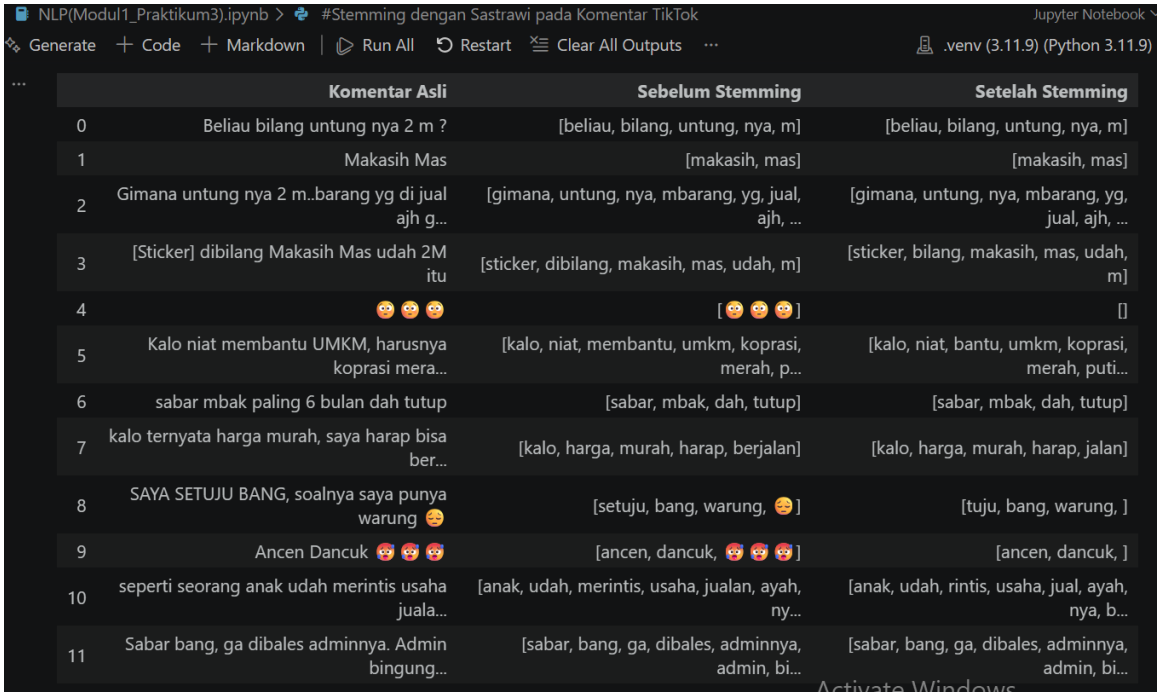
Gambar 6. Stopword removal



	tokens	tokens_no_stopword
0	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]
1	[makasih, mas]	[makasih, mas]
2	[gimana, untung, nya, mbarang, yg, di, jual, a...	[gimana, untung, nya, mbarang, yg, jual, ajh, ...
3	[sticker, dibilang, makasih, mas, udah, m, itu]	[sticker, dibilang, makasih, mas, udah, m]
4	[😍 😍 😍]	[😍 😍 😍]
5	[kalo, niat, membantu, umkm, harusnya, koprasi...	[kalo, niat, membantu, umkm, koprasi, merah, p...
6	[sabar, mbak, paling, bulan, dah, tutup]	[sabar, mbak, dah, tutup]
7	[kalo, ternyata, harga, murah, saya, harap, bi...	[kalo, harga, murah, harap, berjalan]
8	[saya, setuju, bang, soalnya, saya, punya, war...	[setuju, bang, warung, 😔]
9	[ancen, dancuk, 🤡 🤡 🤡]	[ancen, dancuk, 🤡 🤡 🤡]

Penjelasan: Fungsi stopword removal menghapus kata-kata umum berdasarkan daftar stopword yang digunakan. Kata negasi tertentu dipertahankan agar makna komentar, terutama opini atau sentimen, tidak berubah.

Gambar 7. Perbandingan hasil tahapan preprocessing



	Komentar Asli	Sebelum Stemming	Setelah Stemming
0	Beliau bilang untung nya 2 m ?	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]
1	Makasih Mas	[makasih, mas]	[makasih, mas]
2	Gimana untung nya 2 m..barang yg di jual ajh g...	[gimana, untung, nya, mbarang, yg, jual, ajh, ...	[gimana, untung, nya, mbarang, yg, jual, ajh, ...
3	[Sticker] dibilang Makasih Mas udah 2M itu	[sticker, dibilang, makasih, mas, udah, m]	[sticker, bilang, makasih, mas, udah, m]
4	😳😳😳	[😳😳😳]	[]
5	Kalo niat membantu UMKM, harusnya koprasi mera...	[kalo, niat, membantu, umkm, koprasi, merah, p...	[kalo, niat, bantu, umkm, koprasi, merah, puti...
6	sabar mbak paling 6 bulan dah tutup	[sabar, mbak, dah, tutup]	[sabar, mbak, dah, tutup]
7	kalo ternyata harga murah, saya harap bisa ber...	[kalo, harga, murah, harap, berjalan]	[kalo, harga, murah, harap, jalan]
8	SAYA SETUJU BANG, soalnya saya punya warung 😔	[setuju, bang, warung, 😔]	[tuju, bang, warung,]
9	Ancen Dancuk 🤡🤡🤡	[ancen, dancuk, 🤡🤡🤡]	[ancen, dancuk,]
10	seperti seorang anak udah merintis usaha juala...	[anak, udah, merintis, usaha, jualan, ayah, ny...	[anak, udah, rintis, usaha, jual, ayah, nya, b...
11	Sabar bang, ga dibales adminnya. Admin bingung...	[sabar, bang, ga, dibales, adminnya, admin, bi...	[sabar, bang, ga, dibales, adminnya, admin, bi...

Activate Windows

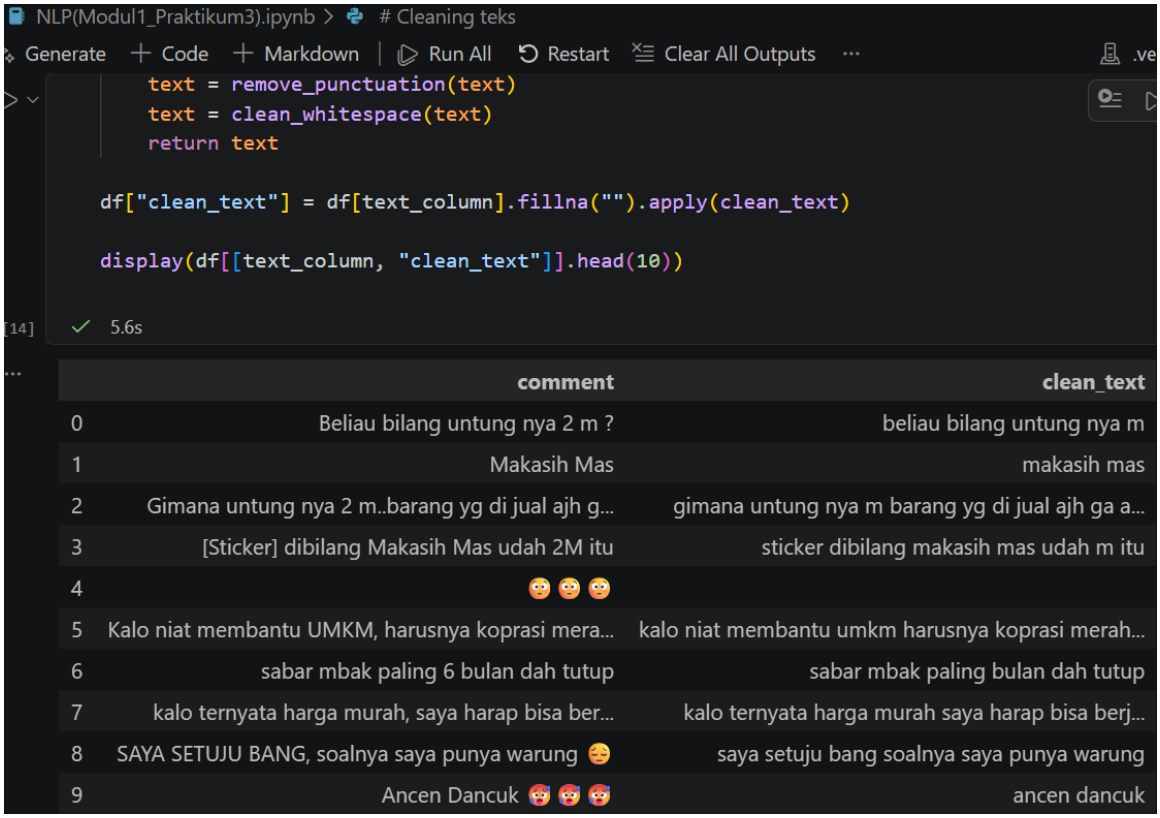
Penjelasan: Tabel ini menampilkan hasil beberapa tahap pemrosesan secara berdampingan. Perbandingan membantu melihat perubahan dari komentar asli menuju teks yang lebih konsisten dan token yang telah diproses.

Gambar 8. Pengujian hasil preprocessing per komentar



Penjelasan: Output ini memperlihatkan pengujian fungsi preprocessing terhadap beberapa contoh komentar. Setiap contoh menunjukkan hasil setelah tahapan normalisasi dijalankan sehingga keluaran dapat diperiksa secara manual.

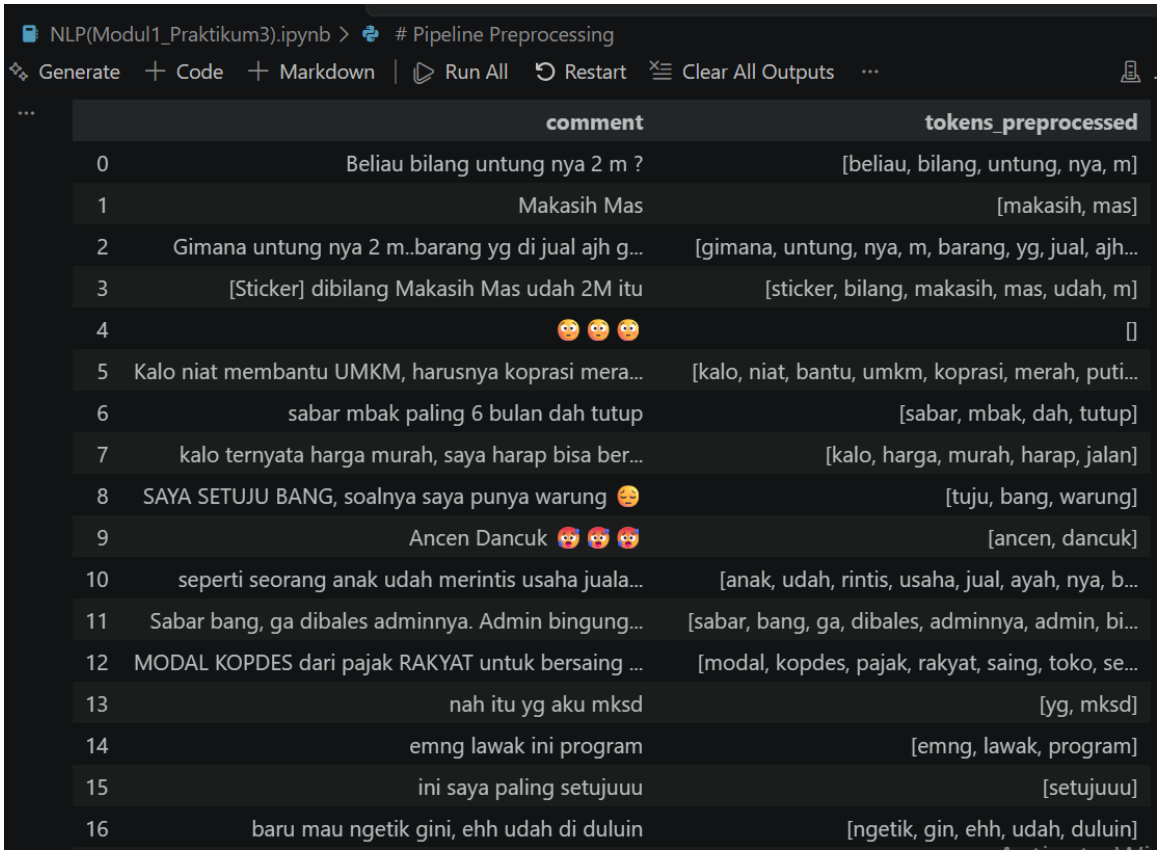
Gambar 9. Fungsi cleaning dan hasil teks bersih



	comment	clean_text
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya m
1	Makasih Mas	makasih mas
2	Gimana untung nya 2 m..barang yg di jual ajh g...	gimana untung nya m barang yg di jual ajh ga a...
3	[Sticker] dibilang Makasih Mas udah 2M itu	sticker dibilang makasih mas udah m itu
4	😳😳😳	
5	Kalo niat membantu UMKM, harusnya koprasi mera...	kalo niat membantu umkm harusnya koprasi merah...
6	sabar mbak paling 6 bulan dah tutup	sabar mbak paling bulan dah tutup
7	kalo ternyata harga murah, saya harap bisa ber...	kalo ternyata harga murah saya harap bisa berj...
8	SAYA SETUJU BANG, soalnya saya punya warung 😔	saya setuju bang soalnya saya punya warung
9	Ancen Dancuk 🤡🤡🤡	ancen dancuk

Penjelasan: Kode mendefinisikan fungsi untuk membersihkan teks dan menampilkan hasilnya pada contoh data. Hasil cleaning menjadi masukan bagi tahap tokenisasi dan pemrosesan lanjutan.

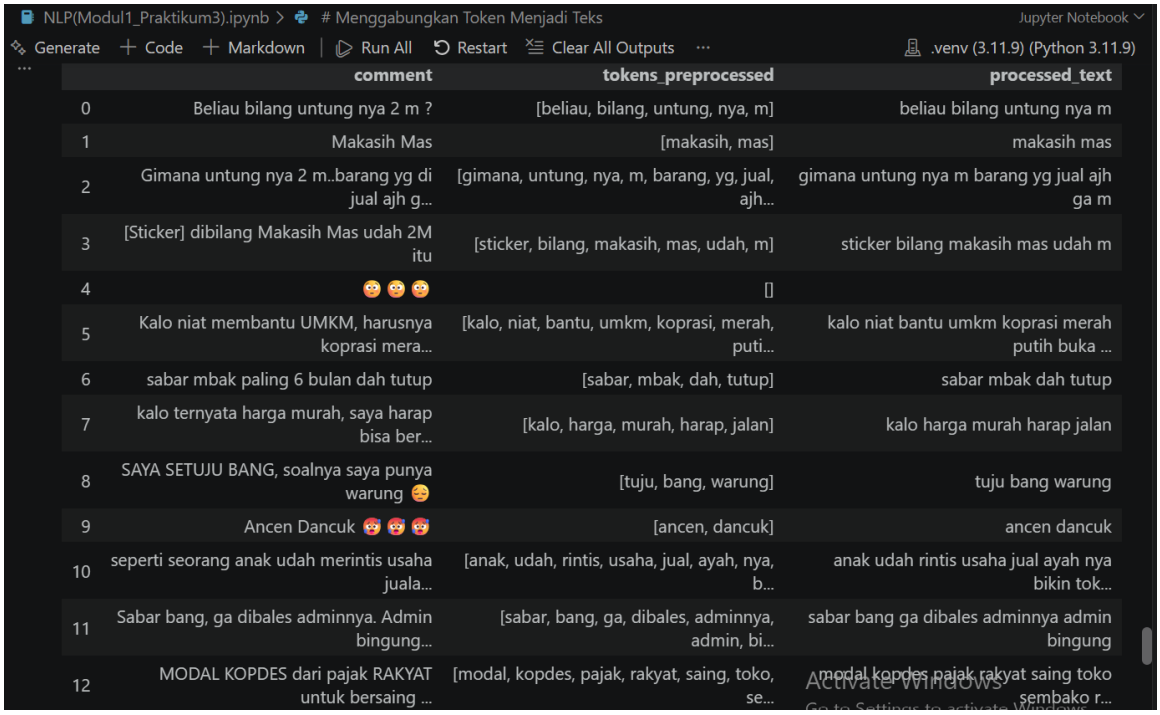
Gambar 10. Penerapan preprocessing pada data komentar



	comment	tokens_preprocessed
0	Beliau bilang untung nya 2 m ?	[beliau, bilang, untung, nya, m]
1	Makasih Mas	[makasih, mas]
2	Gimana untung nya 2 m..barang yg di jual ajh g...	[gimana, untung, nya, m, barang, yg, jual, ajh...
3	[Sticker] dibilang Makasih Mas udah 2M itu	[sticker, bilang, makasih, mas, udah, m]
4	😳😳😳	[]
5	Kalo niat membantu UMKM, harusnya koprasi mera...	[kalo, niat, bantu, umkm, koprasi, merah, puti...
6	sabar mbak paling 6 bulan dah tutup	[sabar, mbak, dah, tutup]
7	kalo ternyata harga murah, saya harap bisa ber...	[kalo, harga, murah, harap, jalan]
8	SAYA SETUJU BANG, soalnya saya punya warung 😔	[tuju, bang, warung]
9	Ancen Dancuk 🤡🤡🤡	[ancen, dancuk]
10	seperti seorang anak udah merintis usaha juala...	[anak, udah, rintis, usaha, jual, ayah, nya, b...
11	Sabar bang, ga dibales adminnya. Admin bingung...	[sabar, bang, ga, dibales, adminnya, admin, bi...
12	MODAL KOPDES dari pajak RAKYAT untuk bersaing ...	[modal, kopdes, pajak, rakyat, saing, toko, se...
13	nah itu yg aku mksd	[yg, mksd]
14	emng lawak ini program	[emng, lawak, program]
15	ini saya paling setujuuu	[setujuuu]
16	baru mau ngetik gini, ehh udah di duluin	[ngetik, gin, ehh, udah, duluin]

Penjelasan: Tahap ini menerapkan fungsi preprocessing pada data komentar dan menampilkan kolom hasilnya. Proses dilakukan agar setiap komentar memiliki representasi yang konsisten untuk analisis berikutnya.

Gambar 11. Hasil stemming pada komentar



NLP(Modul1_Praktikum3).ipynb > # Menggabungkan Token Menjadi Teks — Jupyter Notebook

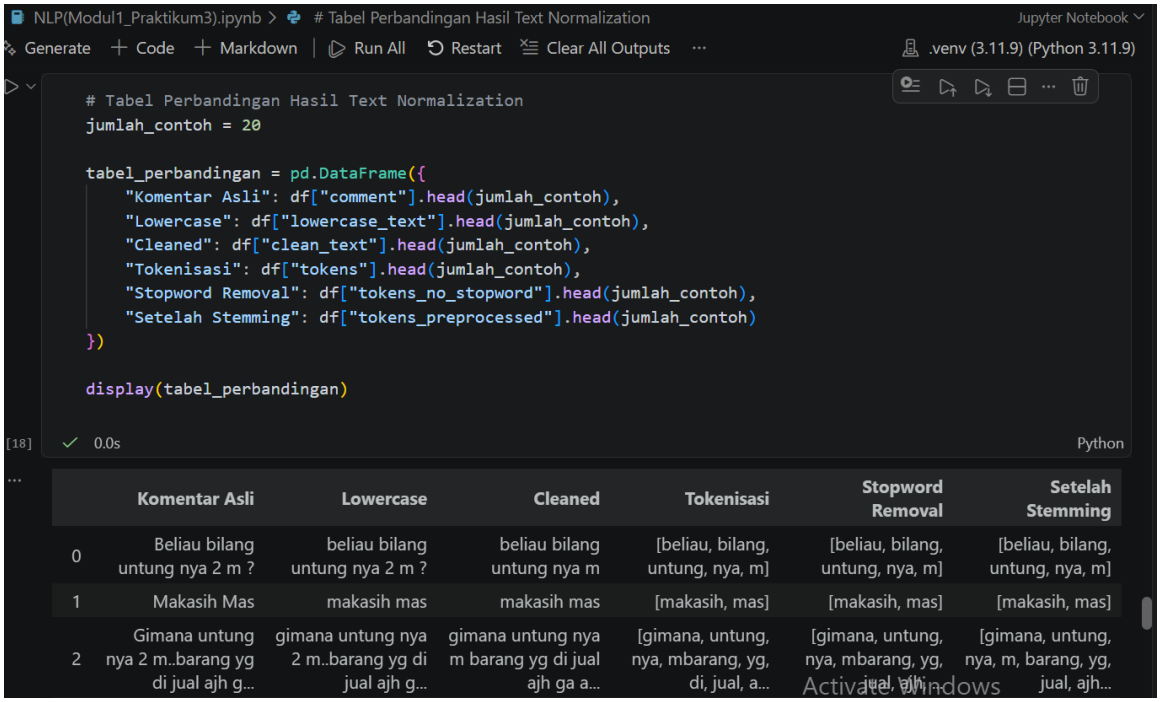
Generate | Code | Markdown | Run All | Restart | Clear All Outputs | ... | .venv (3.11.9) (Python 3.11.9)

	comment	tokens_preprocessed	processed_text
0	Beliau bilang untung nya 2 m ?	[beliau, bilang, untung, nya, m]	beliau bilang untung nya m
1	Makasih Mas	[makasih, mas]	makasih mas
2	Gimana untung nya 2 m..barang yg di jual ajh g...	[gimana, untung, nya, m, barang, yg, jual, ajh...	gimana untung nya m barang yg jual ajh ga m
3	[Sticker] dibilang Makasih Mas udah 2M itu	[sticker, bilang, makasih, mas, udah, m]	sticker bilang makasih mas udah m
4	😳😳😳	[]	
5	Kalo niat membantu UMKM, harusnya koprasi mera...	[kalo, niat, bantu, umkm, koprasi, merah, puti...	kalo niat bantu umkm koprasi merah putih buka ...
6	sabar mbak paling 6 bulan dah tutup	[sabar, mbak, dah, tutup]	sabar mbak dah tutup
7	kalo ternyata harga murah, saya harap bisa ber...	[kalo, harga, murah, harap, jalan]	kalo harga murah harap jalan
8	SAYA SETUJU BANG, soalnya saya punya warung 😔	[tuju, bang, warung]	tuju bang warung
9	Ancen Dancuk 🤯🤯🤯	[ancen, dancuk]	ancen dancuk
10	seperti seorang anak udah merintis usaha juala...	[anak, udah, rintis, usaha, jual, ayah, nya, b...	anak udah rintis usaha jual ayah nya bikin tok...
11	Sabar bang, ga dibales adminnya. Admin bingung...	[sabar, bang, ga, dibales, adminnya, admin, bi...	sabar bang ga dibales adminnya admin bingung
12	MODAL KOPDES dari pajak RAKYAT untuk bersaing ...	[modal, kopdes, pajak, rakyat, saing, toko, se...	modal kopdes pajak rakyat saing toko sembako r...

Activate Windows

Penjelasan: Tampilan menunjukkan kolom komentar dan hasil token setelah pemrosesan, termasuk stemming. Stemming menggunakan pustaka Sastrawi untuk mengubah kata berimbuhan ke bentuk dasar sejauh didukung oleh aturan dan kamusnya.

Gambar 12. Penyusunan tabel eksperimen 20 komentar



NLP(Modul1_Praktikum3).ipynb > # Tabel Perbandingan Hasil Text Normalization — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

```
# Tabel Perbandingan Hasil Text Normalization
jumlah_contoh = 20

tabel_perbandingan = pd.DataFrame({
    "Komentar Asli": df["comment"].head(jumlah_contoh),
    "Lowercase": df["lowercase_text"].head(jumlah_contoh),
    "Cleaned": df["clean_text"].head(jumlah_contoh),
    "Tokenisasi": df["tokens"].head(jumlah_contoh),
    "Stopword Removal": df["tokens_no_stopword"].head(jumlah_contoh),
    "Setelah Stemming": df["tokens_preprocessed"].head(jumlah_contoh)
})

display(tabel_perbandingan)
```

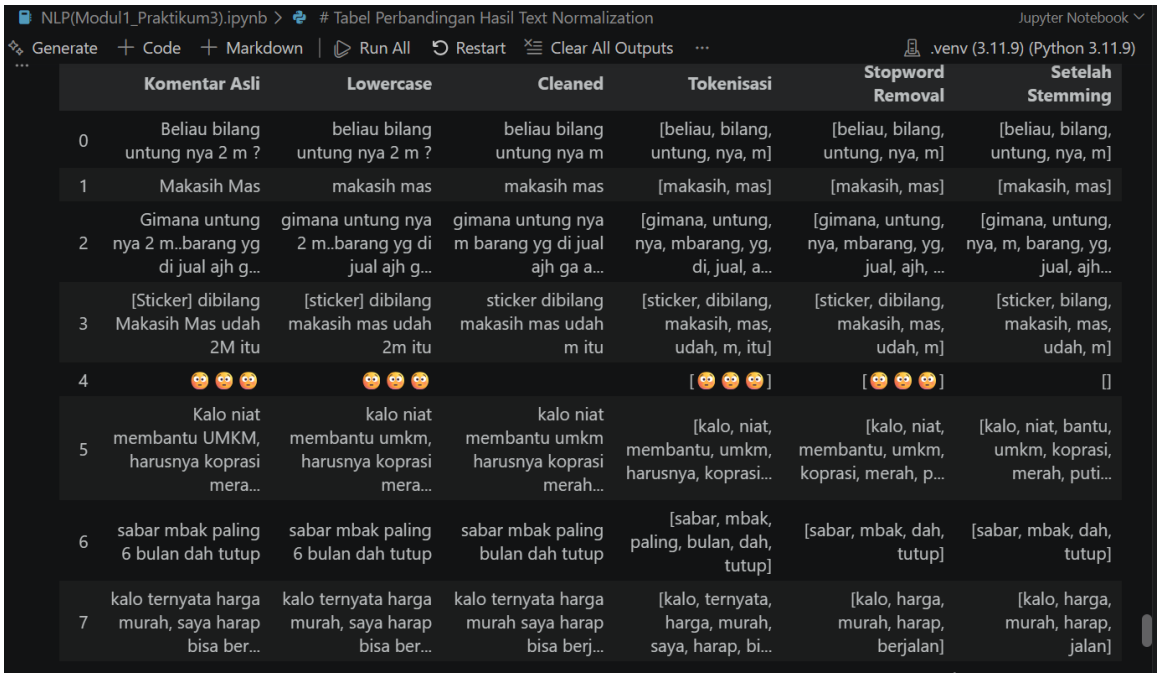
[18] ✓ 0.0s Python

	Komentar Asli	Lowercase	Cleaned	Tokenisasi	Stopword Removal	Setelah Stemming
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya 2 m ?	beliau bilang untung nya m	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]
1	Makasih Mas	makasih mas	makasih mas	[makasih, mas]	[makasih, mas]	[makasih, mas]
2	Gimana untung nya 2 m..barang yg di jual ajh g...	gimana untung nya 2 m..barang yg di jual ajh g...	gimana untung nya m barang yg di jual ajh ga a...	[gimana, untung, nya, mbarang, yg, di, jual, a...	[gimana, untung, nya, mbarang, yg, jual, ajh, ...	[gimana, untung, nya, m, barang, yg, jual, ajh...

Activate Windows

Penjelasan: Kode menyusun tabel eksperimen berisi 20 komentar dengan kolom Raw, Lowercase, Cleaned, Tokenized, Stopword Removal, dan Stemming. Tabel ini digunakan untuk mengevaluasi perubahan teks pada setiap tahapan.

Gambar 13. Hasil tabel eksperimen 20 komentar



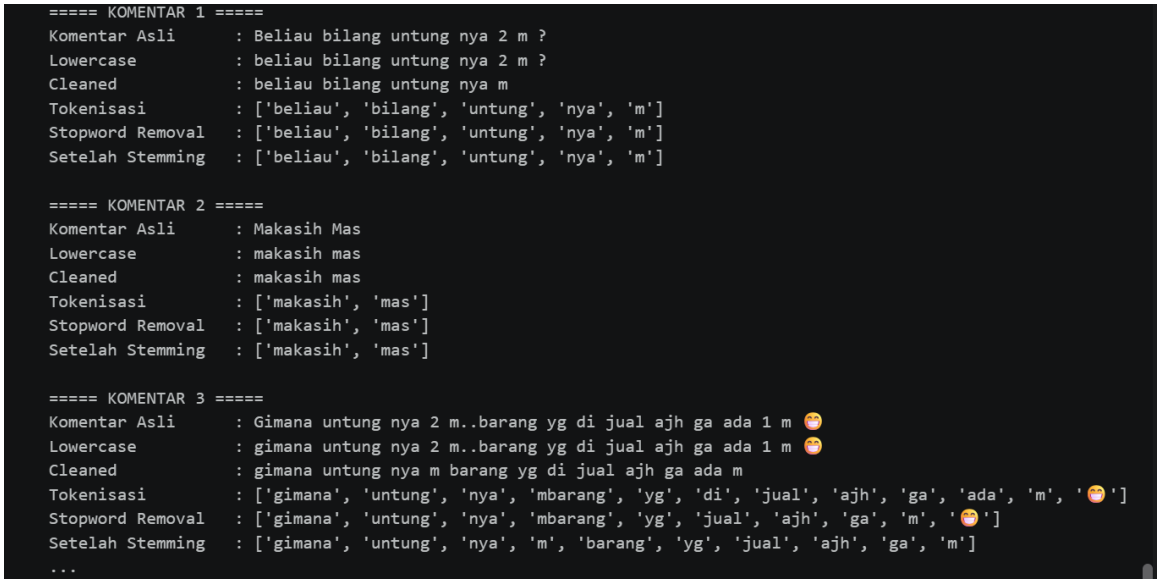
NLP(Modul1_Praktikum3).ipynb > # Tabel Perbandingan Hasil Text Normalization — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

	Komentar Asli	Lowercase	Cleaned	Tokenisasi	Stopword Removal	Setelah Stemming
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya 2 m ?	beliau bilang untung nya m	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]	[beliau, bilang, untung, nya, m]
1	Makasih Mas	makasih mas	makasih mas	[makasih, mas]	[makasih, mas]	[makasih, mas]
2	Gimana untung nya 2 m..barang yg di jual ajh g...	gimana untung nya 2 m..barang yg di jual ajh g...	gimana untung nya m barang yg di jual ajh ga a...	[gimana, untung, nya, mbarang, yg, di, jual, a...	[gimana, untung, nya, mbarang, yg, jual, ajh, ...	[gimana, untung, nya, m, barang, yg, jual, ajh...
3	[Sticker] dibilang Makasih Mas udah 2M itu	[sticker] dibilang makasih mas udah 2m itu	sticker dibilang makasih mas udah m itu	[sticker, dibilang, makasih, mas, udah, m, itu]	[sticker, dibilang, makasih, mas, udah, m]	[sticker, bilang, makasih, mas, udah, m]
4	😊😊😊	😊😊😊		[😊😊😊]	[😊😊😊]	[]
5	Kalo niat membantu UMKM, harusnya koprasi mera...	kalo niat membantu umkm, harusnya koprasi mera...	kalo niat membantu umkm harusnya koprasi merah...	[kalo, niat, membantu, umkm, harusnya, koprasi...	[kalo, niat, membantu, umkm, koprasi, merah, p...	[kalo, niat, bantu, umkm, koprasi, merah, puti...
6	sabar mbak paling 6 bulan dah tutup	sabar mbak paling 6 bulan dah tutup	sabar mbak paling bulan dah tutup	[sabar, mbak, paling, bulan, dah, tutup]	[sabar, mbak, dah, tutup]	[sabar, mbak, dah, tutup]
7	kalo ternyata harga murah, saya harap bisa ber...	kalo ternyata harga murah, saya harap bisa ber...	kalo ternyata harga murah saya harap bisa berj...	[kalo, ternyata, harga, murah, saya, harap, bi...	[kalo, harga, murah, harap, berjalan]	[kalo, harga, murah, harap, jalan]

Penjelasan: Output tabel memperlihatkan perbandingan hasil preprocessing untuk contoh komentar. Perbandingan ini membantu menilai apakah tahapan yang diterapkan sudah sesuai dengan karakteristik dataset.

Gambar 14. Pemeriksaan hasil tiap tahap pada contoh



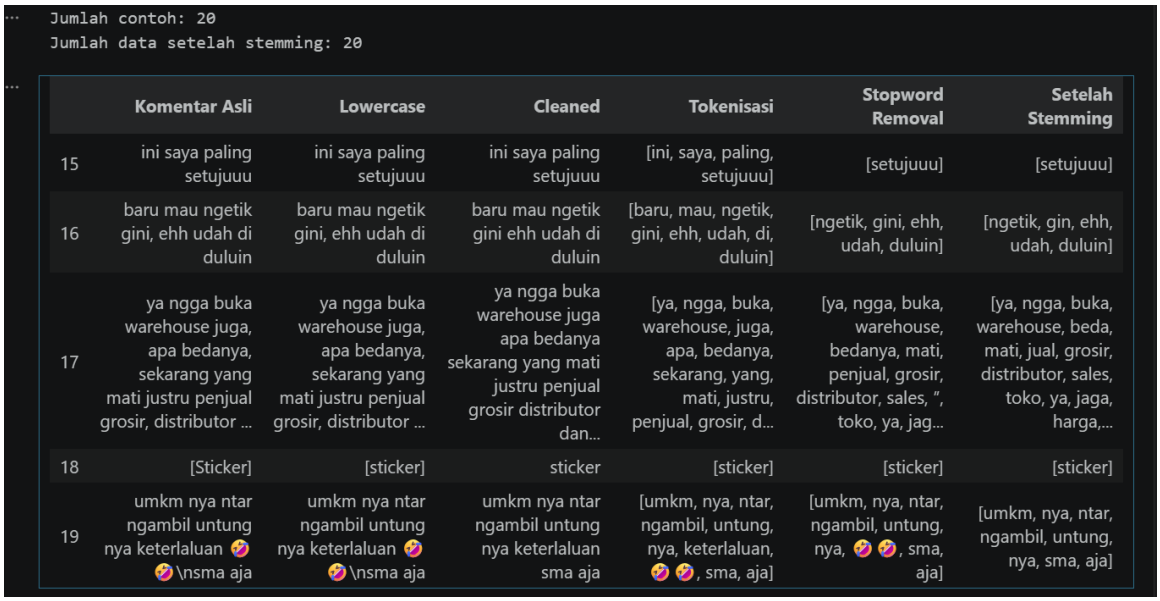
```
===== KOMENTAR 1 =====
Komentar Asli      : Beliau bilang untung nya 2 m ?
Lowercase          : beliau bilang untung nya 2 m ?
Cleaned            : beliau bilang untung nya m
Tokenisasi         : ['beliau', 'bilang', 'untung', 'nya', 'm']
Stopword Removal   : ['beliau', 'bilang', 'untung', 'nya', 'm']
Setelah Stemming   : ['beliau', 'bilang', 'untung', 'nya', 'm']

===== KOMENTAR 2 =====
Komentar Asli      : Makasih Mas
Lowercase          : makasih mas
Cleaned            : makasih mas
Tokenisasi         : ['makasih', 'mas']
Stopword Removal   : ['makasih', 'mas']
Setelah Stemming   : ['makasih', 'mas']

===== KOMENTAR 3 =====
Komentar Asli      : Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁
Lowercase          : gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁
Cleaned            : gimana untung nya m barang yg di jual ajh ga ada m
Tokenisasi         : ['gimana', 'untung', 'nya', 'mbarang', 'yg', 'di', 'jual', 'ajh', 'ga', 'ada', 'm', '😁']
Stopword Removal   : ['gimana', 'untung', 'nya', 'mbarang', 'yg', 'jual', 'ajh', 'ga', 'm', '😁']
Setelah Stemming   : ['gimana', 'untung', 'nya', 'm', 'barang', 'yg', 'jual', 'ajh', 'ga', 'm']
...
```

Penjelasan: Beberapa komentar ditampilkan bersama hasil case folding, cleaning, tokenisasi, stopword removal, dan stemming. Pemeriksaan ini memberikan contoh konkret perubahan yang terjadi pada teks.

Gambar 15. Lanjutan tabel perbandingan komentar



```
Jumlah contoh: 20
Jumlah data setelah stemming: 20
```

	Komentar Asli	Lowercase	Cleaned	Tokenisasi	Stopword Removal	Setelah Stemming
15	ini saya paling setujuuu	ini saya paling setujuuu	ini saya paling setujuuu	[ini, saya, paling, setujuuu]	[setujuuu]	[setujuuu]
16	baru mau ngetik gini, ehh udah di duluin	baru mau ngetik gini, ehh udah di duluin	baru mau ngetik gini ehh udah di duluin	[baru, mau, ngetik, gini, ehh, udah, di, duluin]	[ngetik, gini, ehh, udah, duluin]	[ngetik, gin, ehh, udah, duluin]
17	ya ngga buka warehouse juga, apa bedanya, sekarang yang mati justru penjual grosir, distributor ...	ya ngga buka warehouse juga, apa bedanya, sekarang yang mati justru penjual grosir, distributor ...	ya ngga buka warehouse juga apa bedanya sekarang yang mati justru penjual grosir distributor dan...	[ya, ngga, buka, warehouse, juga, apa, bedanya, sekarang, yang, mati, justru, penjual, grosir, d...	[ya, ngga, buka, warehouse, bedanya, mati, penjual, grosir, distributor, sales, '', toko, ya, jag...	[ya, ngga, buka, warehouse, beda, mati, jual, grosir, distributor, sales, toko, ya, jaga, harga,...
18	[Sticker]	[sticker]	sticker	[sticker]	[sticker]	[sticker]
19	umkm nya ntar ngambil untung nya keterlaluan 🤣🤣\nsma aja	umkm nya ntar ngambil untung nya keterlaluan 🤣🤣\nsma aja	umkm nya ntar ngambil untung nya keterlaluan sma aja	[umkm, nya, ntar, ngambil, untung, nya, keterlaluan, 🤣🤣, sma, aja]	[umkm, nya, ntar, ngambil, untung, nya, 🤣🤣, sma, aja]	[umkm, nya, ntar, ngambil, untung, nya, sma, aja]

Penjelasan: Bagian ini menunjukkan baris lanjutan dari tabel eksperimen. Hasilnya memperlihatkan bahwa teks media sosial dapat mengandung bahasa informal, singkatan, serta emoji yang memengaruhi hasil pembersihan.

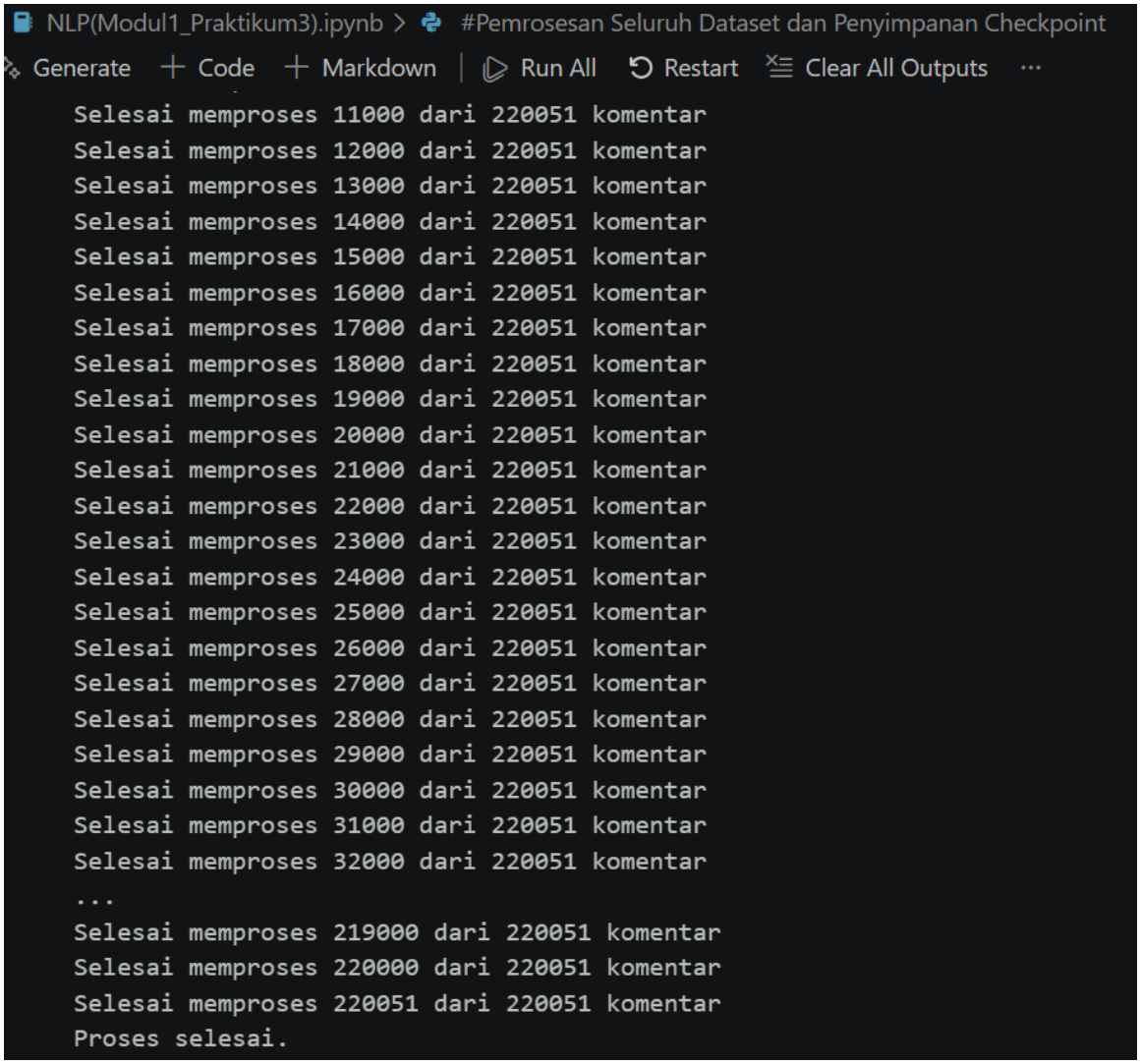
Gambar 16. Perbandingan komentar asli dan processed_text



	comment	processed_text
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya m
1	Makasih Mas	makasih mas
2	Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁	gimana untung nya m barang yg jual ajh ga m
3	[Sticker] dibilang Makasih Mas udah 2M itu	sticker bilang makasih mas udah m
4	😁😁😁	
5	Kalo niat membantu UMKM, harusnya koprasi merah putih dibuka dalam bentuk warehouse (gudang), bu...	kalo niat bantu umkm koprasi merah putih buka bentuk warehouse gudang bukan bentuk toko belanja ...
6	sabar mbak paling 6 bulan dah tutup	sabar mbak dah tutup
7	kalo ternyata harga murah, saya harap bisa berjalan terus.	kalo harga murah harap jalan
8	SAYA SETUJU BANG, soalnya saya punya warung 😍	tuju bang warung
9	Ancen Dancuk 🤣🤣🤣	ancen dancuk

Penjelasan: Tabel ini menampilkan komentar asli dan teks akhir pada kolom processed_text. Perbandingan digunakan untuk memastikan hasil akhir tersimpan dan perubahan teks dapat ditinjau.

Gambar 17. Pemrosesan data secara bertahap



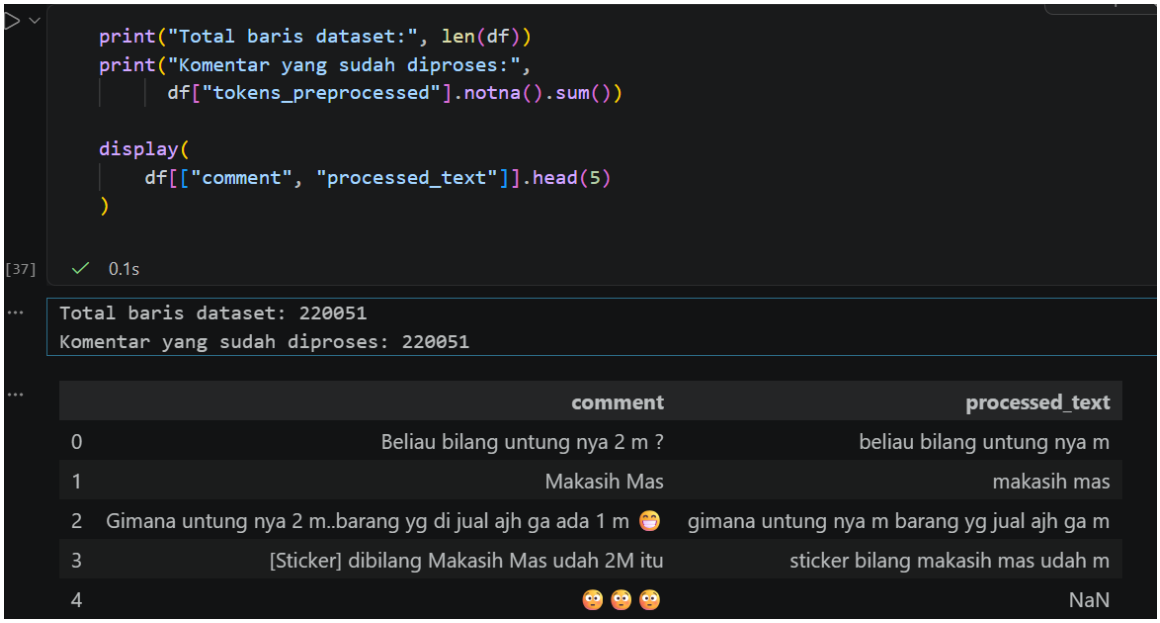
Penjelasan: Output terminal menunjukkan kemajuan pemrosesan dalam batch hingga seluruh 220.051 komentar selesai diproses. Pemrosesan bertahap membantu memantau kemajuan dan menyimpan checkpoint selama pengerjaan.

Gambar 18. Penyimpanan dataset hasil preprocessing



Penjelasan: Kode menyimpan DataFrame ke file dataset_preprocessed.csv dengan encoding UTF-8-SIG. Output menyatakan bahwa dataset final berhasil disimpan dan menampilkan jumlah total komentar serta komentar yang telah diproses.

Gambar 19. Validasi jumlah data dan contoh hasil akhir



```
print("Total baris dataset:", len(df))
print("Komentar yang sudah diproses:",
      df["tokens_preprocessed"].notna().sum())

display(
    df[["comment", "processed_text"]].head(5)
)
```

[37] ✓ 0.1s

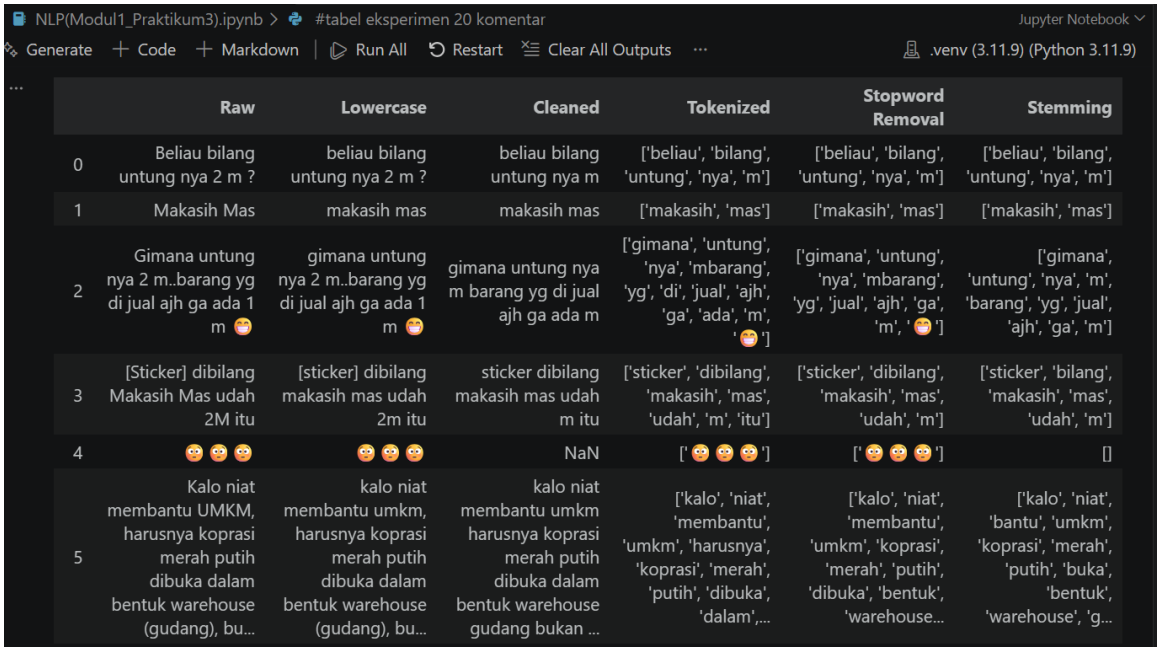
```
Total baris dataset: 220051
Komentar yang sudah diproses: 220051
```

	comment	processed_text
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya m
1	Makasih Mas	makasih mas
2	Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁	gimana untung nya m barang yg jual ajh ga m
3	[Sticker] dibilang Makasih Mas udah 2M itu	sticker bilang makasih mas udah m
4	😁😁😁	NaN

Penjelasan: Tahap validasi menampilkan jumlah baris dataset, jumlah komentar yang sudah diproses, serta contoh pasangan komentar asli dan hasil processed_text. Hasil yang ditunjukkan berjumlah 220.051 komentar.

E. Tugas Eksperimen

Gambar 20. Tabel eksperimen 20 komentar



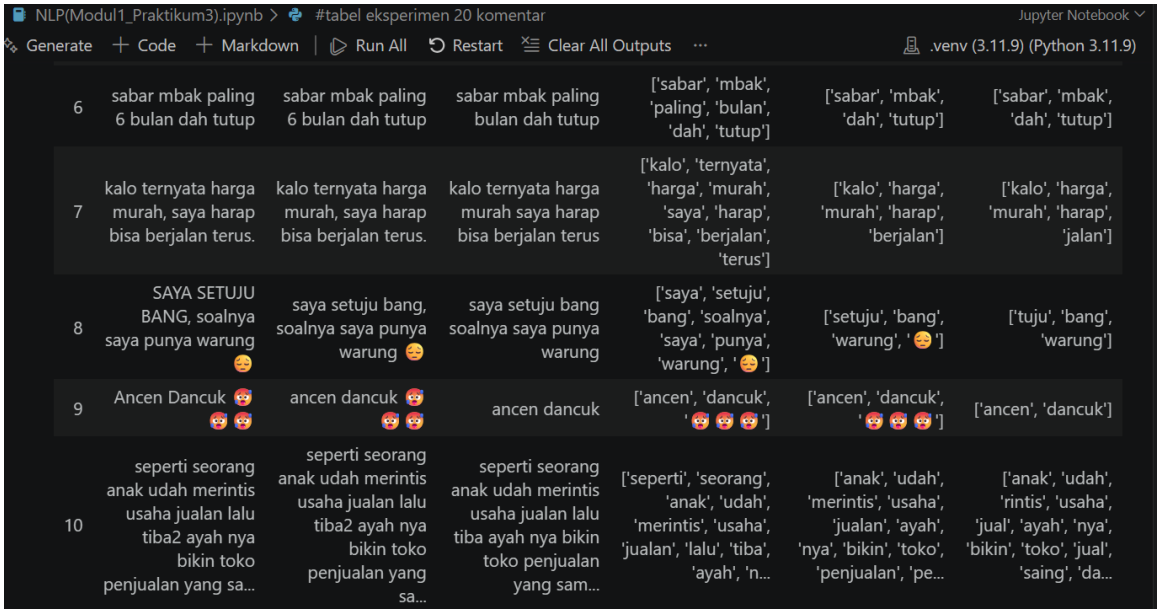
NLP(Modul1_Praktikum3).ipynb > #tabel eksperimen 20 komentar — Jupyter Notebook

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

	Raw	Lowercase	Cleaned	Tokenized	Stopword Removal	Stemming
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya 2 m ?	beliau bilang untung nya m	['beliau', 'bilang', 'untung', 'nya', 'm']	['beliau', 'bilang', 'untung', 'nya', 'm']	['beliau', 'bilang', 'untung', 'nya', 'm']
1	Makasih Mas	makasih mas	makasih mas	['makasih', 'mas']	['makasih', 'mas']	['makasih', 'mas']
2	Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁	gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁	gimana untung nya m barang yg di jual ajh ga ada m	['gimana', 'untung', 'nya', 'mbarang', 'yg', 'di', 'jual', 'ajh', 'ga', 'ada', 'm', '😁']	['gimana', 'untung', 'nya', 'mbarang', 'yg', 'jual', 'ajh', 'ga', 'm', '😁']	['gimana', 'untung', 'nya', 'm', 'barang', 'yg', 'jual', 'ajh', 'ga', 'm']
3	[Sticker] dibilang Makasih Mas udah 2M itu	[sticker] dibilang makasih mas udah 2m itu	sticker dibilang makasih mas udah m itu	['sticker', 'dibilang', 'makasih', 'mas', 'udah', 'm', 'itu']	['sticker', 'dibilang', 'makasih', 'mas', 'udah', 'm']	['sticker', 'bilang', 'makasih', 'mas', 'udah', 'm']
4	😁😁😁	😁😁😁	NaN	['😁😁😁']	['😁😁😁']	[]
5	Kalo niat membantu UMKM, harusnya koprasi merah putih dibuka dalam bentuk warehouse (gudang), bu...	kalo niat membantu umkm, harusnya koprasi merah putih dibuka dalam bentuk warehouse (gudang), bu...	kalo niat membantu umkm harusnya koprasi merah putih dibuka dalam bentuk warehouse gudang bukan ...	['kalo', 'niat', 'membantu', 'umkm', 'harusnya', 'koprasi', 'merah', 'putih', 'dibuka', 'dalam',...	['kalo', 'niat', 'membantu', 'umkm', 'koprasi', 'merah', 'putih', 'dibuka', 'bentuk', 'warehouse...	['kalo', 'niat', 'bantu', 'umkm', 'koprasi', 'merah', 'putih', 'buka', 'bentuk', 'warehouse', 'g...

Penjelasan: Tabel ini menampilkan 20 komentar dan hasil dari setiap tahap normalisasi. Kolom-kolom tersebut memungkinkan pemeriksaan terstruktur dari teks mentah sampai hasil stemming.

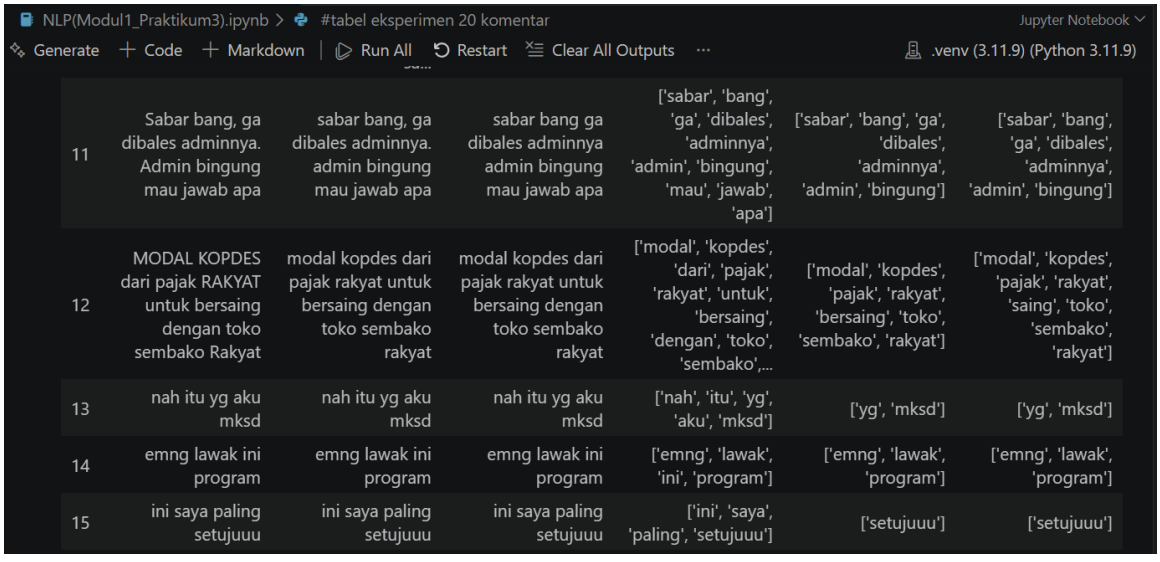
Gambar 21. Lanjutan hasil eksperimen



6	sabar mbak paling 6 bulan dah tutup	sabar mbak paling 6 bulan dah tutup	sabar mbak paling bulan dah tutup	['sabar', 'mbak', 'paling', 'bulan', 'dah', 'tutup']	['sabar', 'mbak', 'dah', 'tutup']	['sabar', 'mbak', 'dah', 'tutup']
7	kalo ternyata harga murah, saya harap bisa berjalan terus.	kalo ternyata harga murah, saya harap bisa berjalan terus.	kalo ternyata harga murah saya harap bisa berjalan terus	['kalo', 'ternyata', 'harga', 'murah', 'saya', 'harap', 'bisa', 'berjalan', 'terus']	['kalo', 'harga', 'murah', 'harap', 'berjalan']	['kalo', 'harga', 'murah', 'harap', 'jalan']
8	SAYA SETUJU BANG, soalnya saya punya warung 😩	saya setuju bang, soalnya saya punya warung 😩	saya setuju bang soalnya saya punya warung	['saya', 'setuju', 'bang', 'soalnya', 'saya', 'punya', 'warung', '😩']	['setuju', 'bang', 'warung', '😩']	['tuju', 'bang', 'warung']
9	Ancen Dancuk 🤡🤡🤡	ancen dancuk 🤡🤡	ancen dancuk	['ancen', 'dancuk', '🤡🤡🤡']	['ancen', 'dancuk', '🤡🤡🤡']	['ancen', 'dancuk']
10	seperti seorang anak udah merintis usaha jualan lalu tiba2 ayah nya bikin toko penjualan yang sa...	seperti seorang anak udah merintis usaha jualan lalu tiba2 ayah nya bikin toko penjualan yang sa...	seperti seorang anak udah merintis usaha jualan lalu tiba ayah nya bikin toko penjualan yang sam...	['seperti', 'seorang', 'anak', 'udah', 'merintis', 'usaha', 'jualan', 'lalu', 'tiba', 'ayah', 'n...	['anak', 'udah', 'merintis', 'usaha', 'jualan', 'ayah', 'nya', 'bikin', 'toko', 'penjualan', 'pe...	['anak', 'udah', 'rintis', 'usaha', 'jual', 'ayah', 'nya', 'bikin', 'toko', 'jual', 'saing', 'da...

Penjelasan: Tampilan ini merupakan bagian lanjutan dari tabel eksperimen. Perbedaan antar kolom memperlihatkan dampak cleaning, tokenisasi, penghapusan stopword, dan stemming pada komentar.

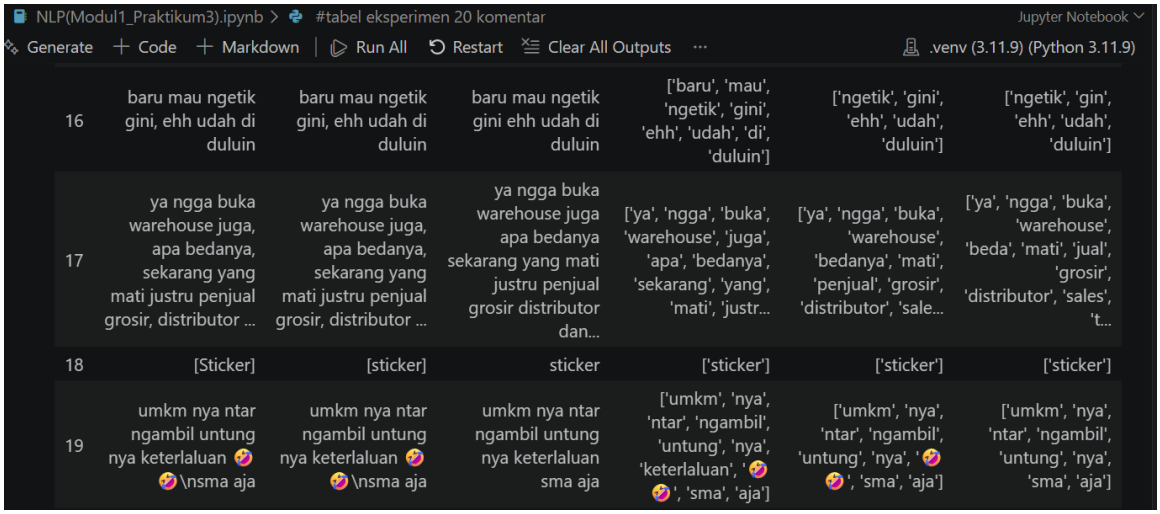
Gambar 22. Lanjutan hasil eksperimen



11	Sabar bang, ga dibales adminnya. Admin bingung mau jawab apa	sabar bang, ga dibales adminnya. admin bingung mau jawab apa	sabar bang ga dibales adminnya admin bingung mau jawab apa	['sabar', 'bang', 'ga', 'dibales', 'adminnya', 'admin', 'bingung', 'mau', 'jawab', 'apa']	['sabar', 'bang', 'ga', 'dibales', 'adminnya', 'admin', 'bingung']	['sabar', 'bang', 'ga', 'dibales', 'adminnya', 'admin', 'bingung']
12	MODAL KOPDES dari pajak RAKYAT untuk bersaing dengan toko sembako Rakyat	modal kopdes dari pajak rakyat untuk bersaing dengan toko sembako rakyat	modal kopdes dari pajak rakyat untuk bersaing dengan toko sembako rakyat	['modal', 'kopdes', 'dari', 'pajak', 'rakyat', 'untuk', 'bersaing', 'dengan', 'toko', 'sembako',...	['modal', 'kopdes', 'pajak', 'rakyat', 'bersaing', 'toko', 'sembako', 'rakyat']	['modal', 'kopdes', 'pajak', 'rakyat', 'saing', 'toko', 'sembako', 'rakyat']
13	nah itu yg aku mksd	nah itu yg aku mksd	nah itu yg aku mksd	['nah', 'itu', 'yg', 'aku', 'mksd']	['yg', 'mksd']	['yg', 'mksd']
14	emng lawak ini program	emng lawak ini program	emng lawak ini program	['emng', 'lawak', 'ini', 'program']	['emng', 'lawak', 'program']	['emng', 'lawak', 'program']
15	ini saya paling setujuuu	ini saya paling setujuuu	ini saya paling setujuuu	['ini', 'saya', 'paling', 'setujuuu']	['setujuuu']	['setujuuu']

Penjelasan: Baris lanjutan ini digunakan untuk memeriksa konsistensi hasil pada contoh komentar yang berbeda. Karena data berasal dari media sosial, sebagian kata informal atau istilah khusus mungkin tetap perlu evaluasi.

Gambar 23. Lanjutan hasil eksperimen



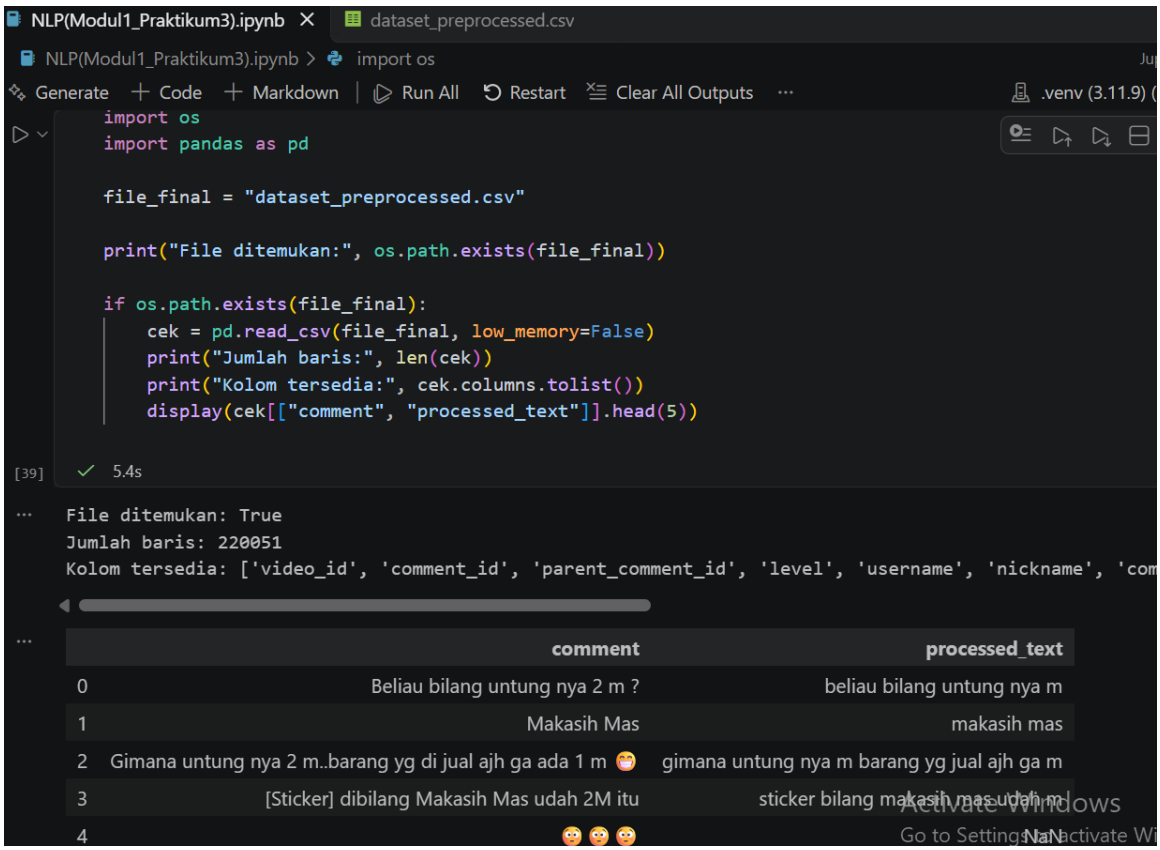
NLP(Modul1_Praktikum3).ipynb > #tabel eksperimen 20 komentar — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

16	baru mau ngetik gini, ehh udah di duluin	baru mau ngetik gini, ehh udah di duluin	baru mau ngetik gini ehh udah di duluin	['baru', 'mau', 'ngetik', 'gini', 'ehh', 'udah', 'di', 'duluin']	['ngetik', 'gini', 'ehh', 'udah', 'duluin']	['ngetik', 'gin', 'ehh', 'udah', 'duluin']
17	ya ngga buka warehouse juga, apa bedanya, sekarang yang mati justru penjual grosir, distributor ...	ya ngga buka warehouse juga, apa bedanya, sekarang yang mati justru penjual grosir, distributor ...	ya ngga buka warehouse juga apa bedanya sekarang yang mati justru penjual grosir distributor dan...	['ya', 'ngga', 'buka', 'warehouse', 'juga', 'apa', 'bedanya', 'sekarang', 'yang', 'mati', 'justr...	['ya', 'ngga', 'buka', 'warehouse', 'bedanya', 'mati', 'penjual', 'grosir', 'distributor', 'sale...	['ya', 'ngga', 'buka', 'warehouse', 'beda', 'mati', 'jual', 'grosir', 'distributor', 'sales', 't...
18	[Sticker]	[sticker]	sticker	['sticker']	['sticker']	['sticker']
19	umkm nya ntar ngambil untung nya keterlaluan 🤣🤣\nsma aja	umkm nya ntar ngambil untung nya keterlaluan 🤣🤣\nsma aja	umkm nya ntar ngambil untung nya keterlaluan sma aja	['umkm', 'nya', 'ntar', 'ngambil', 'untung', 'nya', 'keterlaluan', '🤣🤣', 'sma', 'aja']	['umkm', 'nya', 'ntar', 'ngambil', 'untung', 'nya', '🤣🤣', 'sma', 'aja']	['umkm', 'nya', 'ntar', 'ngambil', 'untung', 'nya', 'sma', 'aja']

Penjelasan: Bagian ini melengkapi perbandingan hasil pada 20 komentar. Evaluasi visual membantu menemukan kemungkinan informasi yang terhapus atau perubahan kata yang perlu diperhatikan.

Gambar 24. Pemeriksaan file dataset_preprocessed.csv



NLP(Modul1_Praktikum3).ipynb | dataset_preprocessed.csv

NLP(Modul1_Praktikum3).ipynb > import os

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9)

```
import os
import pandas as pd

file_final = "dataset_preprocessed.csv"

print("File ditemukan:", os.path.exists(file_final))

if os.path.exists(file_final):
    cek = pd.read_csv(file_final, low_memory=False)
    print("Jumlah baris:", len(cek))
    print("Kolom tersedia:", cek.columns.tolist())
    display(cek[["comment", "processed_text"]].head(5))
```

[39] ✓ 5.4s

```
File ditemukan: True
Jumlah baris: 220051
Kolom tersedia: ['video_id', 'comment_id', 'parent_comment_id', 'level', 'username', 'nickname', 'com
```

	comment	processed_text
0	Beliau bilang untung nya 2 m ?	beliau bilang untung nya m
1	Makasih Mas	makasih mas
2	Gimana untung nya 2 m..barang yg di jual ajh ga ada 1 m 😁	gimana untung nya m barang yg jual ajh ga m
3	[Sticker] dibilang Makasih Mas udah 2M itu	sticker bilang makasih mas udah m
4	😳😳😳	NaN

Activate Windows
Go to Settings to activate Wi

Penjelasan: Tampilan ini memeriksa apakah dataset_preprocessed.csv ditemukan, dapat dibaca, dan memiliki 220.051 baris. Kolom yang tersedia mencakup data asli, kolom tiap tahapan preprocessing, tokens_preprocessed, dan processed_text.

F. Tugas Analisis

1. Apakah semua stopword harus dihapus? Berikan contoh ketika stopword penting.

   Tidak semua stopword harus dihapus dalam proses preprocessing teks. Stopword merupakan kata-kata umum yang sering muncul, seperti “yang”, “dan”, “di”, “ke”, dan “dari”. Penghapusan stopword dapat membantu mengurangi kata yang kurang relevan, tetapi beberapa kata memiliki peran penting dalam menentukan makna kalimat.

   Contohnya adalah kata negasi “tidak” pada kalimat “program ini tidak bagus”. Jika kata “tidak” dihapus, kalimat tersebut menjadi “program ini bagus”, sehingga maknanya berubah. Oleh karena itu, dalam analisis komentar TikTok tentang Koperasi Desa Merah Putih, kata negasi seperti “tidak”, “bukan”, dan “belum” sebaiknya dipertahankan agar sentimen komentar tidak berubah.

2. Apakah stemming selalu meningkatkan kualitas teks? Kapan stemming dapat merugikan?

   Stemming tidak selalu meningkatkan kualitas teks karena manfaatnya bergantung pada tujuan analisis. Stemming bertujuan mengubah kata berimbuhan menjadi kata dasar agar kata-kata yang memiliki bentuk berbeda dapat dikelompokkan.

   Contohnya, kata “membangun”, “dibangun”, dan “pembangunan” dapat diproses agar lebih mudah dianalisis berdasarkan bentuk dasarnya. Namun, stemming dapat merugikan apabila kata yang dihasilkan tidak sesuai dengan konteks atau kehilangan perbedaan makna. Hal ini dapat terjadi pada kata tidak baku, bahasa gaul, singkatan, atau istilah khusus. Oleh sebab itu, hasil stemming menggunakan Sastrawi perlu diperiksa untuk memastikan kata yang dihasilkan tetap relevan dengan isi komentar.

3. Apakah angka harus selalu dihapus? Berikan contoh angka yang penting.

   Angka tidak harus selalu dihapus karena beberapa angka mengandung informasi penting. Penghapusan angka cocok dilakukan apabila angka tidak berhubungan dengan tujuan analisis. Namun, angka dapat menjadi informasi utama dalam konteks tertentu.

   Contohnya, komentar “bantuan sebesar 5 juta rupiah” mengandung informasi nominal yang penting. Jika angka dihapus, komentar tersebut kehilangan informasi jumlah bantuan. Dalam dataset komentar tentang Koperasi

Desa Merah Putih, angka dapat menunjukkan nominal bantuan, tahun program, jumlah dana, atau nomor tertentu. Oleh karena itu, penghapusan angka sebaiknya disesuaikan dengan kebutuhan analisis.

4. Apakah URL selalu tidak berguna? Kapan URL dapat memberikan informasi?

   URL tidak selalu tidak berguna. URL sering dihapus karena dapat menambah variasi teks yang tidak diperlukan dalam analisis bahasa. Namun, URL dapat memberikan informasi apabila digunakan untuk mengetahui sumber informasi, mengidentifikasi berita, atau menganalisis tautan yang dibagikan pengguna.

   Contohnya, sebuah komentar dapat menyertakan tautan menuju berita mengenai pelaksanaan program Koperasi Desa Merah Putih. Tautan tersebut dapat membantu peneliti menelusuri sumber informasi. Dengan demikian, URL dapat dihapus untuk analisis sentimen berbasis teks, tetapi sebaiknya dipertahankan atau disimpan secara terpisah apabila sumber tautan juga menjadi bagian dari penelitian.

5. Apakah hashtag harus selalu dihapus?

   Hashtag tidak harus selalu dihapus karena dapat mengandung informasi mengenai topik pembicaraan. Dalam media sosial, hashtag sering digunakan untuk mengelompokkan komentar berdasarkan isu tertentu.

   Contohnya, #KoperasiDesaMerahPutih dapat menunjukkan bahwa sebuah komentar berkaitan langsung dengan program tersebut. Jika hashtag dihapus seluruhnya, informasi topik yang terkandung di dalamnya juga dapat hilang. Untuk analisis teks biasa, simbol # dapat dihapus sementara kata setelahnya dipertahankan. Pada kode preprocessing yang digunakan saat ini, hashtag beserta katanya dihapus, sehingga pendekatan tersebut memiliki keterbatasan apabila hashtag mengandung informasi penting.

6. Apakah preprocessing yang sama cocok untuk semua tugas NLP?

   Preprocessing yang sama tidak selalu cocok untuk semua tugas Natural Language Processing (NLP). Setiap tugas memiliki kebutuhan yang berbeda sehingga tahapan preprocessing harus disesuaikan.

   Contohnya, analisis sentimen perlu mempertahankan kata negasi seperti “tidak” dan “bukan” karena kata-kata tersebut memengaruhi polaritas komentar. Sementara itu, klasifikasi topik mungkin lebih berfokus pada kata-kata yang

menunjukkan tema pembicaraan. Untuk tugas penerjemahan atau analisis bahasa yang memperhatikan susunan kalimat, penghapusan stopword dan stemming secara berlebihan juga dapat merusak informasi.

Oleh karena itu, preprocessing pada dataset komentar TikTok tentang Koperasi Desa Merah Putih perlu disesuaikan dengan tujuan analisis dan karakteristik bahasa yang digunakan oleh pengguna.

G. Pertanyaan Diskusi

1. Mengapa preprocessing teks berbeda untuk analisis sentimen dan klasifikasi teks?

Preprocessing teks berbeda karena analisis sentimen dan klasifikasi teks memiliki tujuan yang berbeda. Analisis sentimen bertujuan mengetahui kecenderungan opini pengguna, seperti positif, negatif, atau netral. Oleh karena itu, kata yang menunjukkan emosi dan negasi harus dipertahankan.

Sementara itu, klasifikasi teks bertujuan mengelompokkan teks berdasarkan kategori tertentu, seperti topik program, jenis keluhan, atau informasi bantuan. Pada tugas tersebut, kata-kata yang menunjukkan topik dapat lebih diutamakan. Dengan demikian, teknik preprocessing harus disesuaikan dengan tujuan analisis agar informasi penting tidak hilang.

2. Apa dampak menghapus kata “tidak” dari kalimat “tidak bagus”?

Penghapusan kata “tidak” dapat mengubah makna kalimat secara berlawanan. Kalimat “tidak bagus” menunjukkan penilaian negatif, sedangkan setelah kata “tidak” dihapus, kalimat menjadi “bagus” yang menunjukkan penilaian positif.

Hal ini dapat menyebabkan kesalahan dalam analisis sentimen karena sistem mungkin mengklasifikasikan komentar negatif sebagai positif. Oleh karena itu, pada preprocessing dataset komentar TikTok, kata “tidak” dipertahankan dengan mengeluarkannya dari daftar stopword yang dihapus.

3. Bagaimana cara menangani bahasa gaul Indonesia seperti “gue”, “lu”, dan “bgt”?

Bahasa gaul dapat ditangani melalui proses normalisasi kata tidak baku menjadi bentuk yang lebih umum, misalnya “gue” menjadi “saya”, “lu” menjadi “kamu”, dan “bgt” menjadi “banget”. Proses tersebut dapat dilakukan dengan menggunakan kamus bahasa gaul yang sesuai dengan konteks penelitian.

Namun, normalisasi tidak boleh dilakukan secara sembarangan karena sebuah kata dapat memiliki arti berbeda tergantung konteksnya. Pada dataset komentar TikTok, bahasa gaul perlu diperhatikan karena pengguna sering menulis secara informal, menggunakan singkatan, atau menggabungkan beberapa kata. Penambahan kamus normalisasi dapat membantu meningkatkan konsistensi teks sebelum dilakukan analisis lebih lanjut.

4. Apakah Sastrawi cocok untuk semua jenis domain teks?

   Sastrawi merupakan pustaka stemming yang dirancang untuk bahasa Indonesia dan dapat membantu mengubah kata berimbuhan menjadi bentuk dasar. Namun, Sastrawi tidak selalu memberikan hasil optimal untuk semua domain teks.

   Pada komentar media sosial, terdapat banyak kata tidak baku, bahasa gaul, singkatan, dan istilah khusus yang mungkin tidak diproses secara tepat. Hal serupa dapat terjadi pada teks yang menggunakan istilah teknis, nama program, atau nama organisasi. Oleh karena itu, hasil stemming Sastrawi perlu dievaluasi dan, jika diperlukan, dilengkapi dengan kamus khusus yang sesuai dengan domain penelitian.

5. Apa perbedaan preprocessing pada Twitter atau media sosial dengan berita formal?

   Preprocessing teks media sosial berbeda dari berita formal karena karakteristik bahasanya tidak sama. Komentar media sosial biasanya lebih informal, singkat, dan mengandung bahasa gaul, singkatan, emoji, hashtag, mention, serta kesalahan penulisan. Oleh karena itu, preprocessing media sosial sering memerlukan penanganan khusus terhadap unsur-unsur tersebut.

   Sebaliknya, berita formal umumnya menggunakan struktur kalimat yang lebih teratur dan kosakata yang lebih baku. Proses preprocessing berita dapat lebih berfokus pada pembersihan teks, tokenisasi, dan pengolahan kata sesuai tujuan penelitian.

   Pada dataset komentar TikTok tentang Koperasi Desa Merah Putih, karakteristik media sosial perlu diperhatikan agar kata yang menunjukkan opini, emosi, dan konteks pembicaraan tidak terhapus secara berlebihan.

H. Daftar Pustaka

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: Analyzing text with the Natural Language Toolkit*. O'Reilly Media. https://www.nltk.org/book/

NLTK Project. (n.d.). *NLTK documentation: Tokenizers*. https://www.nltk.org/api/nltk.tokenize.html

Sastrawi. (n.d.). *Sastrawi: High quality stemmer library for Indonesian language*. GitHub. https://github.com/sastrawi/sastrawi

Suhaeni, C., Kamila, S. A., Fahira, F., Yusran, M., & Dito, G. A. (2025). Exploring a large language model on the ChatGPT platform for Indonesian text preprocessing tasks. *Indonesian Journal of Statistics and Its Applications, 9*(1), 100–116. https://doi.org/10.29244/ijsa.v9i1p100-116

Modul	1 — Fundamental dan Text Processing
Topik	Linguistic Processing
Repository	https://github.com/lian180706/NATURAL-LANGUAGE-PROCESSING
Tanggal	-

A. Tujuan

1. Memahami Lexical Analysis
2. Memahami Morphology Bahasa Indonesia
3. Memahami Konsep Part-Of-Speech (Pos)
4. Melakukan Pos Tagging
5. Memahami Syntax Dan Dependency Parsing
6. Membaca Struktur Kalimat
7. Memvisualisasikan Dependency Tree
8. Menganalisis Kesalahan Linguistic Processing

B. Requirement

1. Sistem Operasi yang digunakan : Microsoft Windows 11 Pro
2. Browser : Versi 153.0.8010.50 (Build Resmi) (64 bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Natural Language Processing (NLP)

Natural Language Processing (NLP) adalah cabang kecerdasan buatan yang mempelajari cara komputer memahami, menganalisis, dan menghasilkan bahasa manusia. Bidang ini menggabungkan ilmu linguistik, ilmu komputer, dan statistika agar mesin dapat memproses teks atau ucapan yang bersifat tidak terstruktur. Pemrosesan bahasa umumnya dilakukan secara bertahap, mulai dari tingkat kata (leksikal dan morfologi), tingkat kelas kata (sintaksis dangkal), hingga struktur kalimat (parsing). Setiap tahap menghasilkan keluaran yang menjadi masukan bagi tahap berikutnya (Jurafsky & Martin, 2024).

2. Lexical Analysis dan Tokenisasi

Lexical analysis berkaitan dengan unit-unit kata dalam bahasa dan merupakan tahap awal dalam pemrosesan teks. Pada tahap ini, teks dipecah menjadi unit-unit bermakna yang disebut token. Token dapat berupa kata, angka, maupun tanda baca. Proses pemecahan ini disebut tokenisasi, dan setiap kata dalam kalimat dianalisis secara individual. Kualitas tokenisasi sangat menentukan hasil seluruh tahap NLP berikutnya karena kesalahan pada tahap ini akan terbawa ke tahap selanjutnya (Manning & Schütze, 1999; Bird et al., 2009).

3. Morfologi dan Stemming

Morfologi adalah kajian tentang struktur internal dan pembentukan kata dari satuan terkecil yang bermakna, yaitu morfem (Kridalaksana, 2008). Bahasa Indonesia bersifat aglutinatif dan kaya akan imbuhan (afiks). Kata turunan dibentuk melalui afiksasi (prefiks, sufiks, konfiks, dan infiks), reduplikasi, serta pemajemukan (Alwi et al., 2003). Perubahan bentuk ini dapat mengubah makna maupun kelas kata, sehingga menjadi tantangan tersendiri bagi sistem NLP.

Stemming adalah proses mengembalikan kata berimbuhan ke bentuk dasarnya (stem). Proses ini berguna untuk menyatukan variasi bentuk kata yang bermakna sama sehingga analisis teks menjadi lebih efisien. Algoritma stemming bahasa Indonesia yang banyak digunakan adalah Nazief & Adriani, yang menghapus awalan dan akhiran berdasarkan aturan morfologi dan kamus kata

dasar (Adriani et al., 2007). Algoritma ini diimplementasikan dalam pustaka Sastrawi (Library Sastrawi, 2015).

4. Part-of-Speech (POS)

Part-of-Speech (POS) adalah kelas kata yang dibedakan berdasarkan fungsi gramatikal dan perilaku sintaksisnya dalam kalimat. Dalam tata bahasa Indonesia, kelas kata utama meliputi nomina, verba, adjektiva, adverbia, pronomina, numeralia, dan kata tugas (Alwi et al., 2003). Untuk keperluan komputasi lintas bahasa, Petrov et al. (2012) mengusulkan Universal POS Tagset, yang kemudian dikembangkan dalam proyek Universal Dependencies (Nivre et al., 2016; de Marneffe et al., 2021). Tagset ini mencakup NOUN, VERB, ADJ, ADV, PRON, DET, ADP, NUM, CCONJ, SCONJ, dan PUNCT. Untuk bahasa Indonesia, Dinakaramani et al. (2014) juga telah menyusun tagset beserta korpus berlabel POS.

5. POS Tagging

POS tagging adalah proses memberikan label kelas kata pada setiap token dalam kalimat. Hasilnya berupa pasangan antara token dan kelas katanya, yang menjadi dasar bagi analisis sintaksis. Kesulitan utamanya adalah ambiguitas, yaitu satu kata bisa memiliki lebih dari satu kelas kata tergantung konteks (Jurafsky & Martin, 2024). Pendekatan yang digunakan meliputi berbasis aturan, statistik (Hidden Markov Model, Conditional Random Field), dan neural network.

6. Dependency Parsing

Dependency parsing adalah analisis sintaksis yang menentukan hubungan gramatikal antar token dalam kalimat. Hubungan tersebut dinyatakan dalam pasangan *head* (induk) dan *dependent* (bawahan). Hasilnya berupa pohon dependensi yang berakar pada satu kata utama (ROOT), biasanya verba utama (Kübler et al., 2009). Konsep ini berakar pada tata bahasa dependensi Tesnière (1959). Relasi yang umum digunakan antara lain *nsubj* (subjek nominal) dan *obj* (objek) (de Marneffe et al., 2021). Dengan struktur ini, komputer dapat mengetahui siapa pelaku dan apa yang dikenai tindakan dalam sebuah kalimat.

D. Implementasi

Import library

```
#1. Import Library
import sys
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

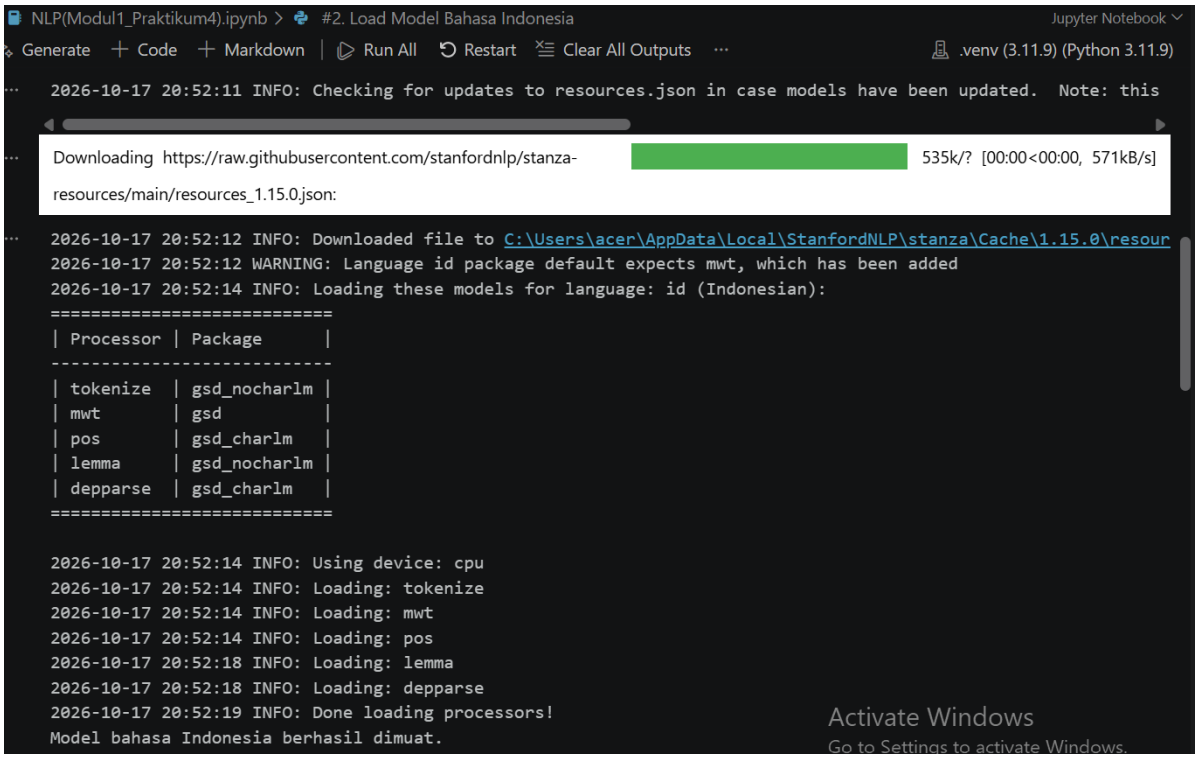
print("Python:", sys.version)
print("Pandas:", pd.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
```

[35] ✓ 0.0s

```
Python: 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
Pandas: 3.0.5
Matplotlib: 3.11.2
```

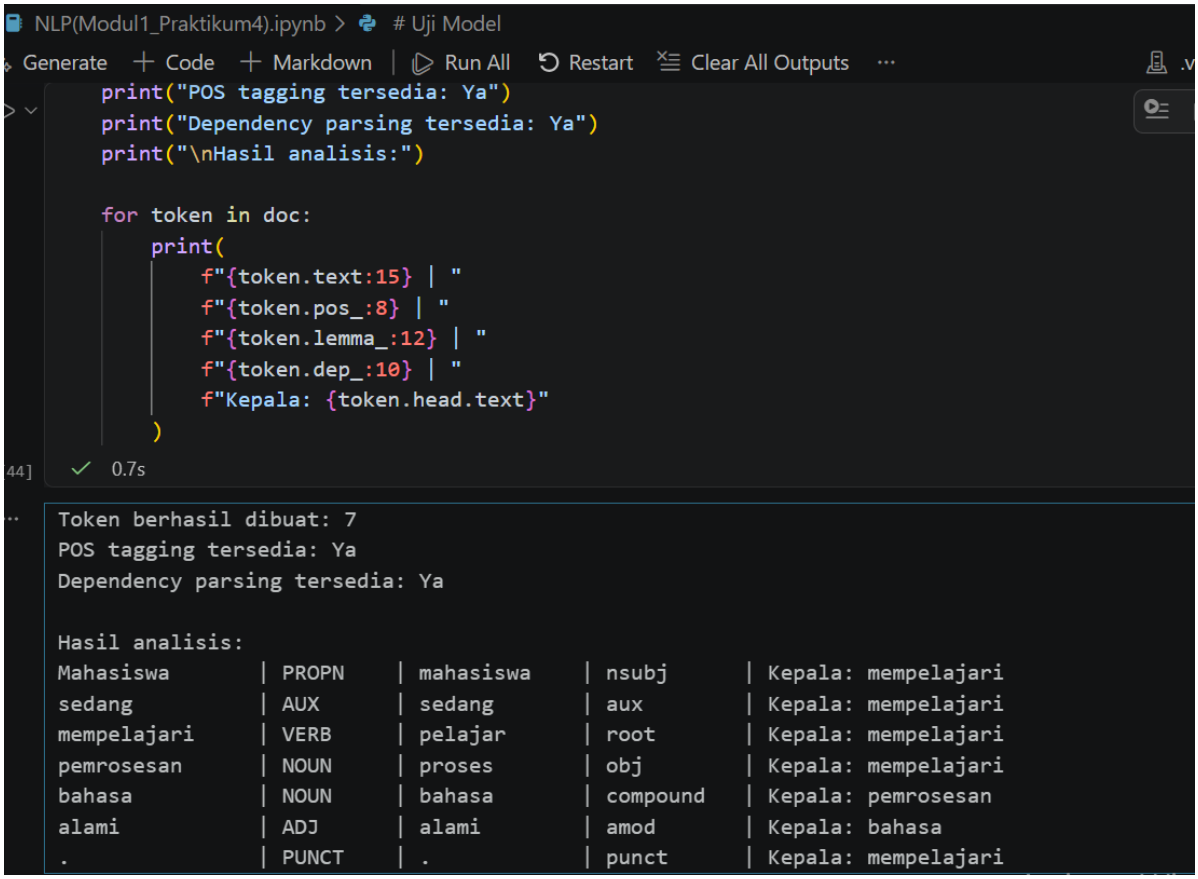
Sel ini mengimpor pustaka yang diperlukan untuk pemrosesan bahasa, pengolahan data, dan menampilkan hasil analisis. Library disiapkan terlebih dahulu agar dapat digunakan pada tahap berikutnya.

Persiapan lingkungan dan model



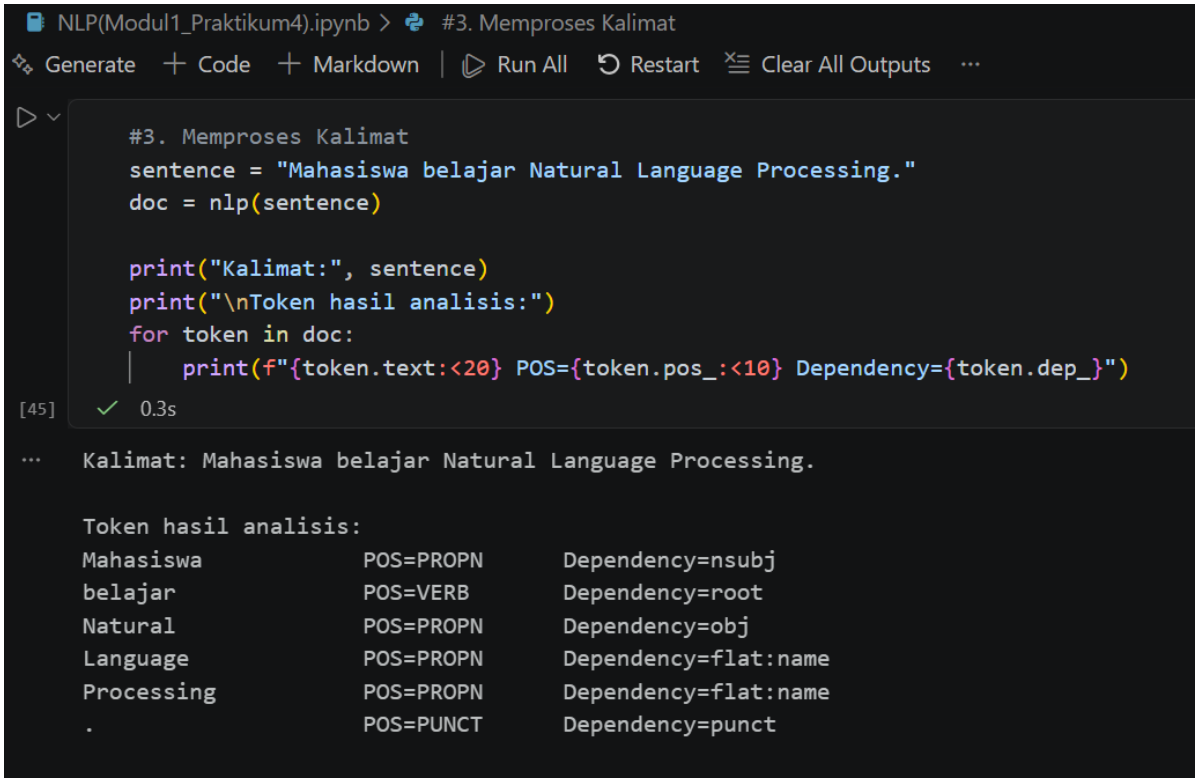
Tampilan ini menunjukkan persiapan lingkungan kerja serta pemeriksaan atau pemuatan komponen NLP. Tahap ini memastikan perangkat dan model siap dipakai sebelum kalimat dianalisis.

Menyiapkan kalimat dan proses analisis



Kode menyiapkan data kalimat untuk diproses dan menjalankan analisis linguistik. Hasil yang ditampilkan menjadi dasar untuk memeriksa token serta label yang diberikan model.

Menampilkan hasil POS tagging dan dependency



Bagian ini memperlihatkan pemanggilan proses analisis dan keluaran token beserta informasi linguistiknya. Keluaran digunakan untuk melihat bagaimana setiap kata diproses oleh model.

Analisis token pada kalimat

```
#4. Analisis POS Token
doc = nlp("Mahasiswa belajar Natural Language Processing.")

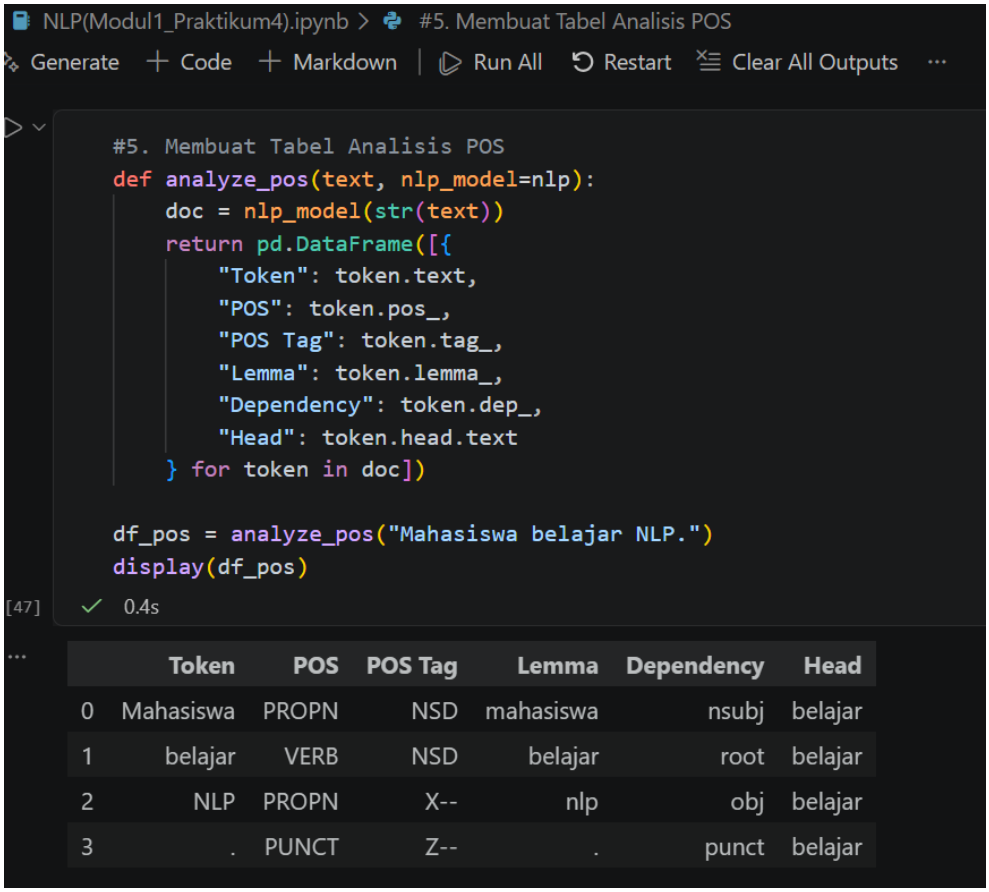
print(f"{'Token':<18} {'POS':<12} {'Lemma'}")
print("-" * 55)
for token in doc:
    print(f"{token.text:<18} {token.pos_:<12} {token.lemma_}")
```

[46] ✓ 0.3s

```
Token              POS          Lemma
-------------------------------------------------------
Mahasiswa          PROPN        mahasiswa
belajar            VERB         belajar
Natural            PROPN        natural
Language           PROPN        language
Processing         PROPN        processing
.                  PUNCT        .
```

Kalimat dipecah menjadi token, kemudian informasi setiap token ditampilkan. Tokenisasi merupakan tahap awal yang membantu proses POS tagging dan analisis sintaksis.

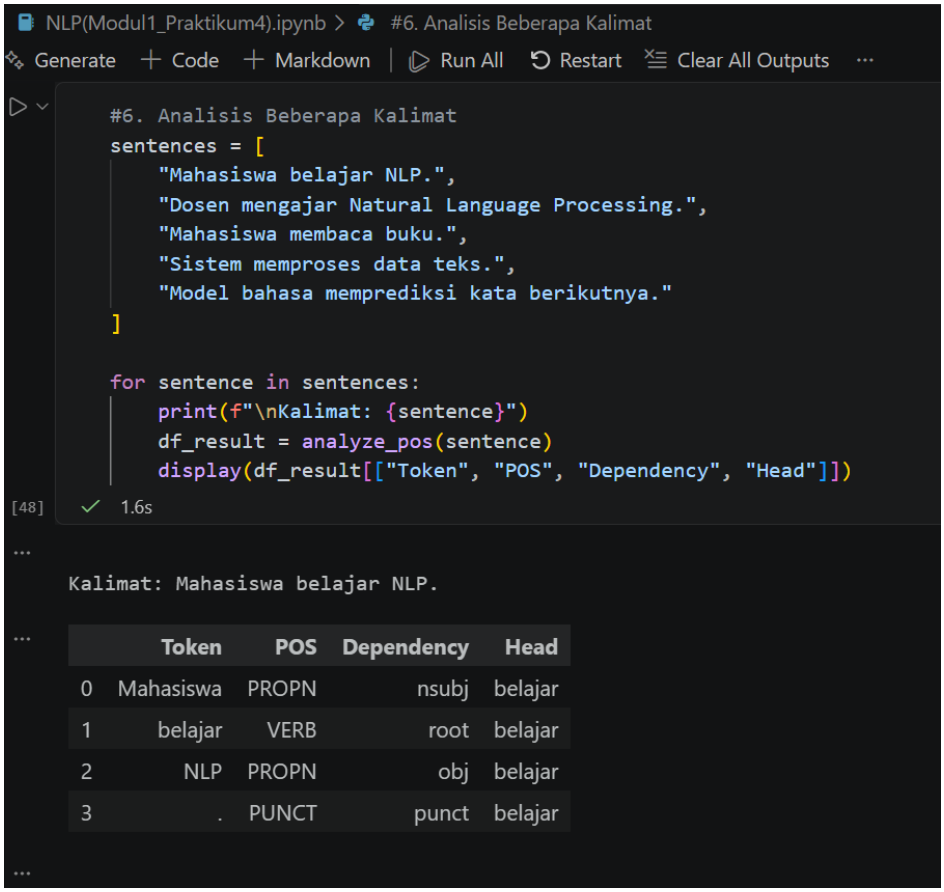
POS tagging pada kalimat uji



	Token	POS	POS Tag	Lemma	Dependency	Head
0	Mahasiswa	PROPN	NSD	mahasiswa	nsubj	belajar
1	belajar	VERB	NSD	belajar	root	belajar
2	NLP	PROPN	X--	nlp	obj	belajar
3	.	PUNCT	Z--	.	punct	belajar

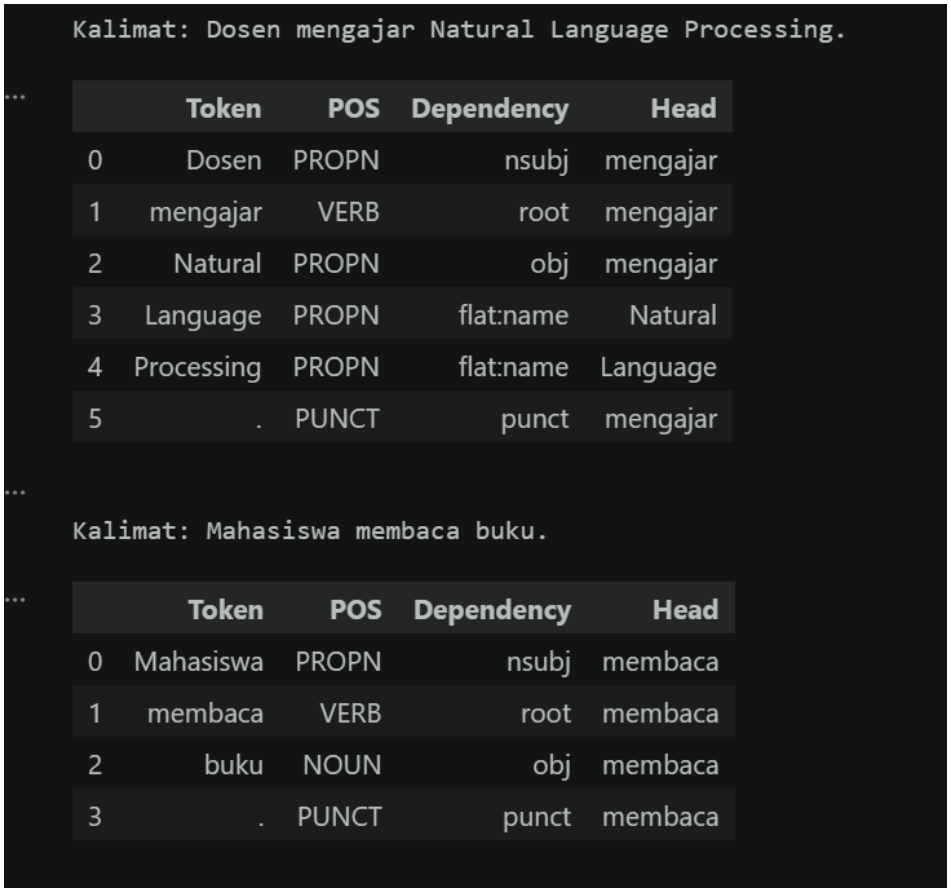
Kode dan tabel pada gambar memperlihatkan hasil pelabelan kelas kata. Kolom token dan POS membantu mengamati label yang diberikan pada kata dalam konteks kalimat.

Pemeriksaan hasil POS tagging



Hasil analisis ditampilkan dalam bentuk tabel agar token dan kelas katanya mudah diperiksa. Perbedaan konteks dapat menyebabkan kata memperoleh label yang berbeda.

Contoh hasil analisis kalimat



Kalimat: Dosen mengajar Natural Language Processing.

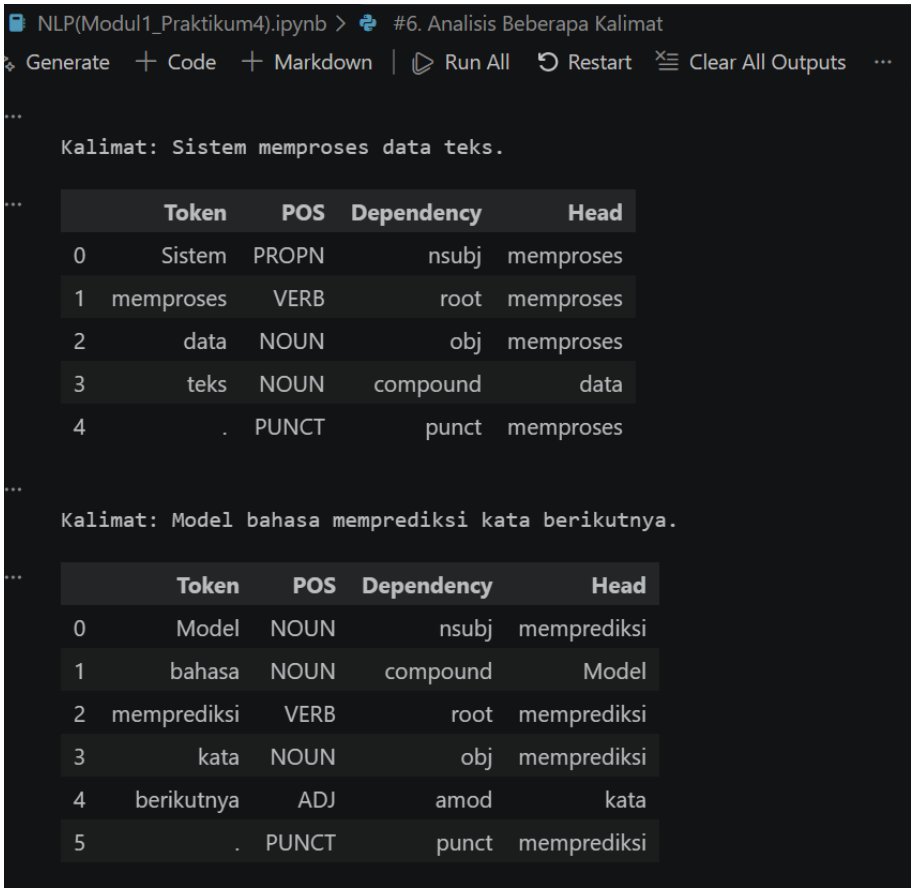
	Token	POS	Dependency	Head
0	Dosen	PROPN	nsubj	mengajar
1	mengajar	VERB	root	mengajar
2	Natural	PROPN	obj	mengajar
3	Language	PROPN	flat:name	Natural
4	Processing	PROPN	flat:name	Language
5	.	PUNCT	punct	mengajar

Kalimat: Mahasiswa membaca buku.

	Token	POS	Dependency	Head
0	Mahasiswa	PROPN	nsubj	membaca
1	membaca	VERB	root	membaca
2	buku	NOUN	obj	membaca
3	.	PUNCT	punct	membaca

Tabel menunjukkan keluaran analisis untuk kalimat lain. Setiap token dipasangkan dengan label POS dan informasi dependensi yang dihasilkan model.

Perbandingan hasil analisis token



Tampilan ini menyajikan hasil tokenisasi dan label linguistik untuk kalimat uji. Hasil tersebut dapat digunakan untuk memeriksa apakah pembagian kata dan labelnya sesuai konteks.

Dependency parsing

NLP(Modul1_Praktikum4).ipynb > #7. Dependency Parsing

Generate + Code + Markdown | Run All Restart Clear All Outputs ···

```
#7. Dependency Parsing
doc = nlp("Mahasiswa belajar NLP.")

print(f"{'Token':<18} {'Dependency':<15} {'Head'}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<18} {token.dep_:<15} {token.head.text}")
```

[49] ✓ 0.2s

```
Token              Dependency      Head
--------------------------------------------------
Mahasiswa          nsubj           belajar
belajar            root            belajar
NLP                obj             belajar
.                  punct           belajar
```

Kode menjalankan dependency parsing dan menampilkan relasi dependensi beserta head atau kata induknya. Relasi ini membantu menjelaskan keterkaitan gramatikal antarkata.

Visualisasi dependency tree

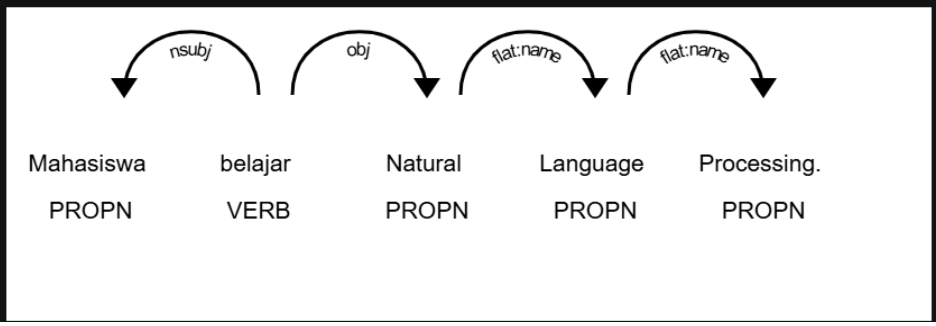
NLP(Modul1_Praktikum4).ipynb > #8. Visualisasi Dependency

Generate + Code + Markdown | Run All Restart Clear All Outputs ···

```
#8. Visualisasi Dependency
from spacy import displacy

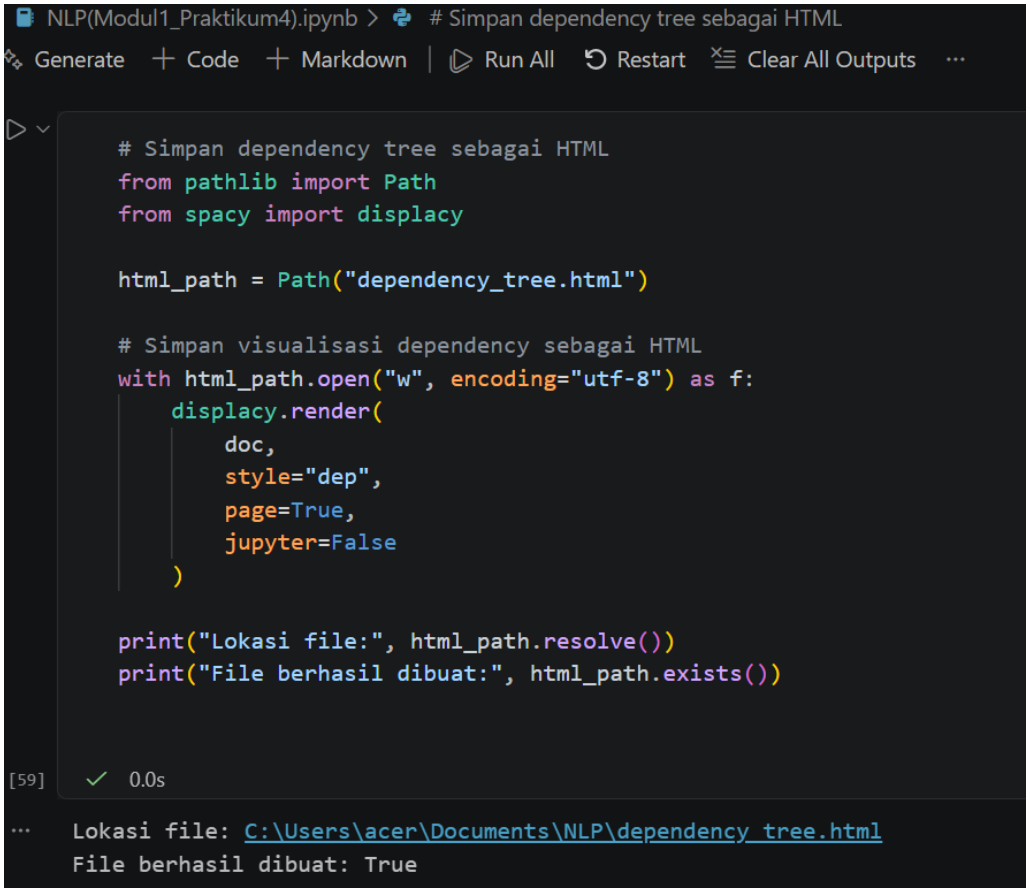
doc = nlp("Mahasiswa belajar Natural Language Processing.")
displacy.render(doc, style="dep", jupyter=True, options={"distance": 100})
```

[50] ✓ 0.3s



Pohon dependensi memperlihatkan hubungan antar-token dalam kalimat secara visual. Busur menunjukkan relasi antara kata yang menjadi head dan kata yang bergantung padanya.

Kode untuk visualisasi dependency tree



Bagian ini menyiapkan kode untuk menghasilkan visualisasi struktur dependensi dari kalimat. Hasil visual membantu pembacaan relasi sintaksis yang sebelumnya ditampilkan dalam tabel.

Pengaturan visualisasi dependency

NLP(Modul1_Praktikum4).ipynb > # Simpan dependency tree sebagai HTML

Generate + Code + Markdown | Run All Restart Clear All Outputs

```
from pathlib import Path
from spacy import displacy

# Buat HTML diagram dependency tree
html_content = displacy.render(
    doc,
    style="dep",
    page=True,
    jupyter=False
)

# Simpan HTML ke file
html_path = Path("dependency_tree.html")
html_path.write_text(html_content, encoding="utf-8")

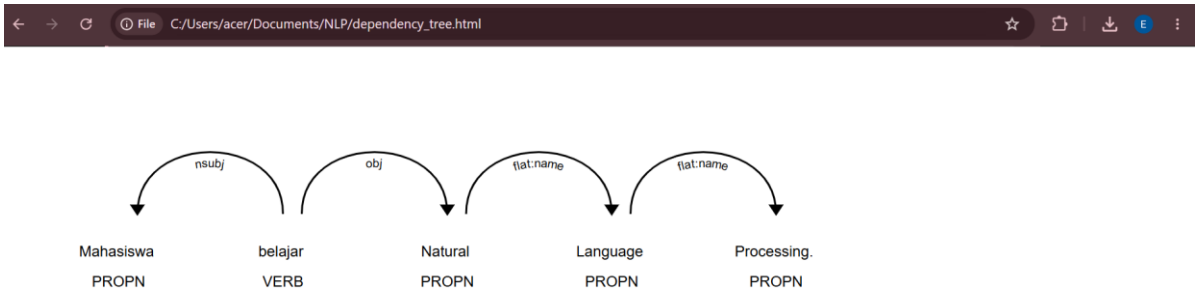
print("Diagram dependency tree berhasil disimpan!")
print("Lokasi:", html_path.resolve())
print("Ukuran file:", html_path.stat().st_size, "byte")
```

[63] ✓ 0.0s

```
Diagram dependency tree berhasil disimpan!
Lokasi: C:\Users\acer\Documents\NLP\dependency_tree.html
Ukuran file: 4309 byte
```

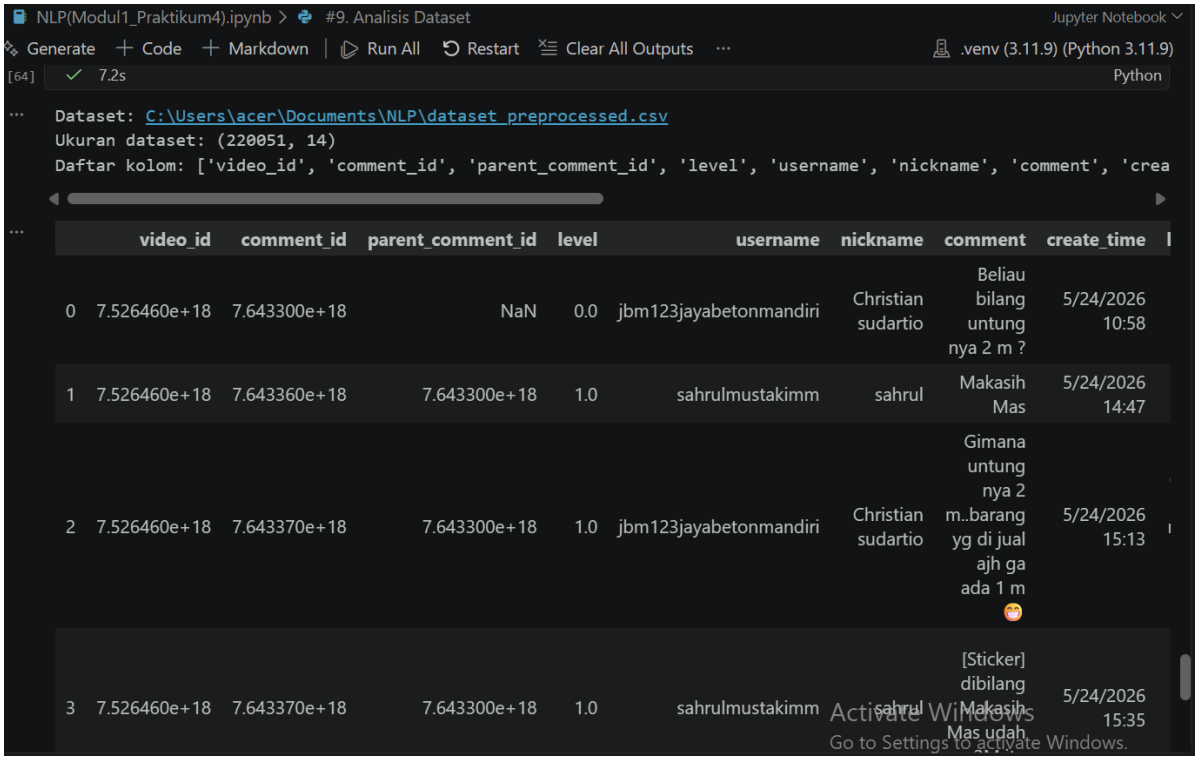
Kode mengatur proses pembuatan atau penyimpanan visualisasi dependency tree. Pengaturan ini digunakan agar struktur hasil parsing dapat ditampilkan dan diperiksa.

Contoh tampilan dependency tree



Gambar memperlihatkan hasil visualisasi pohon dependensi. Setiap kata ditampilkan bersama relasi sintaksisnya sehingga struktur kalimat dapat diamati.

Analisis dataset



Tabel merangkum data kalimat yang dianalisis. Ringkasan ini membantu melihat isi dataset sebelum setiap kalimat diperiksa lebih lanjut.

Kode pengolahan dataset

```
if "comment" in df.columns:
    TEXT_COLUMN = "comment"
elif "text" in df.columns:
    TEXT_COLUMN = "text"
elif "processed_text" in df.columns:
    TEXT_COLUMN = "processed_text"
else:
    raise ValueError("Kolom teks tidak ditemukan: " + str(df.columns.tolist()))

text_df = df[df[TEXT_COLUMN].notna()].copy()
text_df[TEXT_COLUMN] = text_df[TEXT_COLUMN].astype(str).str.strip()
text_df = text_df[text_df[TEXT_COLUMN] != ""]

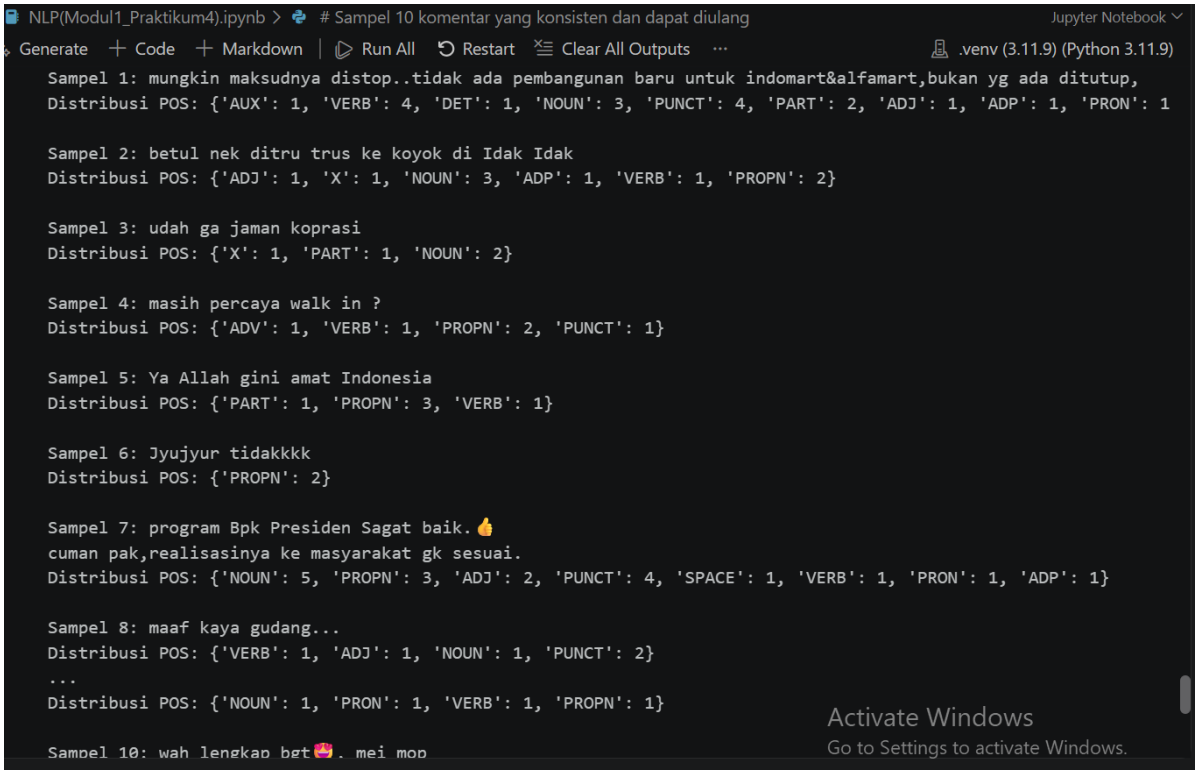
print("Kolom teks:", TEXT_COLUMN)
print("Jumlah baris dataset:", len(df))
print("Jumlah teks valid:", len(text_df))
print("Jumlah teks kosong:", df[TEXT_COLUMN].isna().sum())
```

[65] ✓ 0.5s

```
Kolom teks: comment
Jumlah baris dataset: 220051
Jumlah teks valid: 219747
Jumlah teks kosong: 304
```

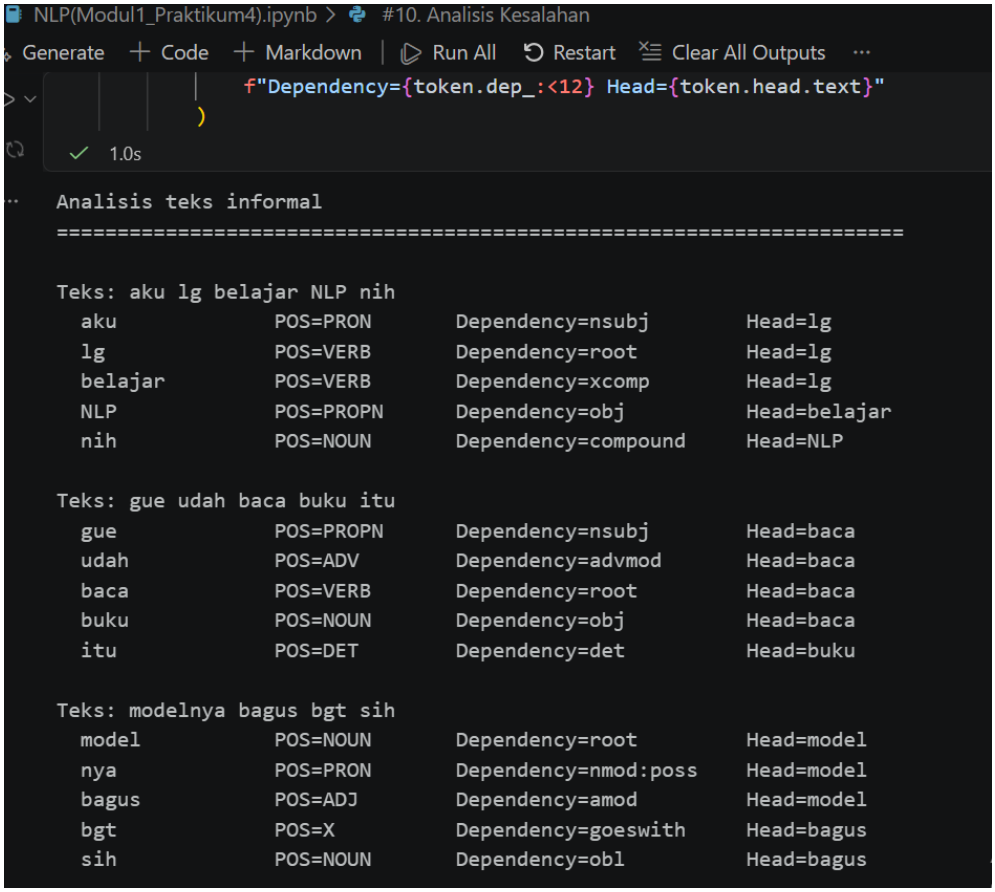
Kode menyiapkan dan memproses data kalimat untuk dianalisis. Hasil pemrosesan kemudian digunakan untuk menghasilkan tabel ringkasan dan analisis linguistik.

Keluaran ringkasan pemrosesan



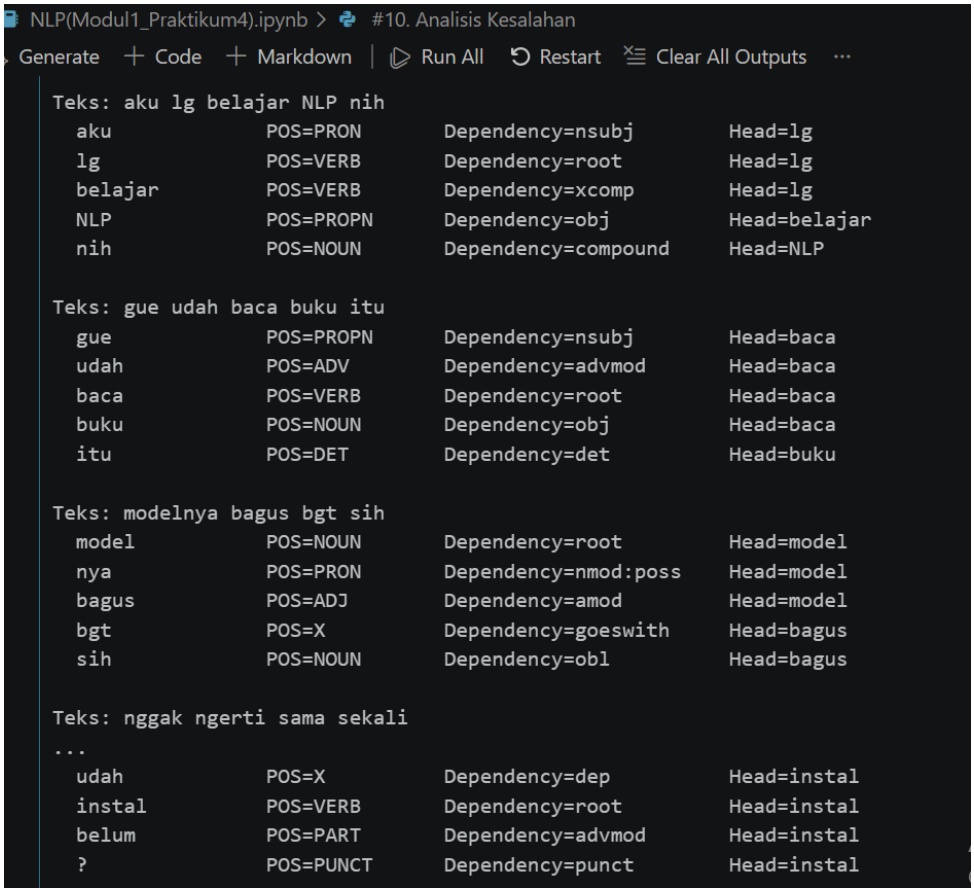
Tampilan ini menunjukkan keluaran proses analisis yang dijalankan pada data. Keluaran tersebut menjadi bukti bahwa tahapan pengolahan telah dilakukan.

Tabel hasil analisis kalimat



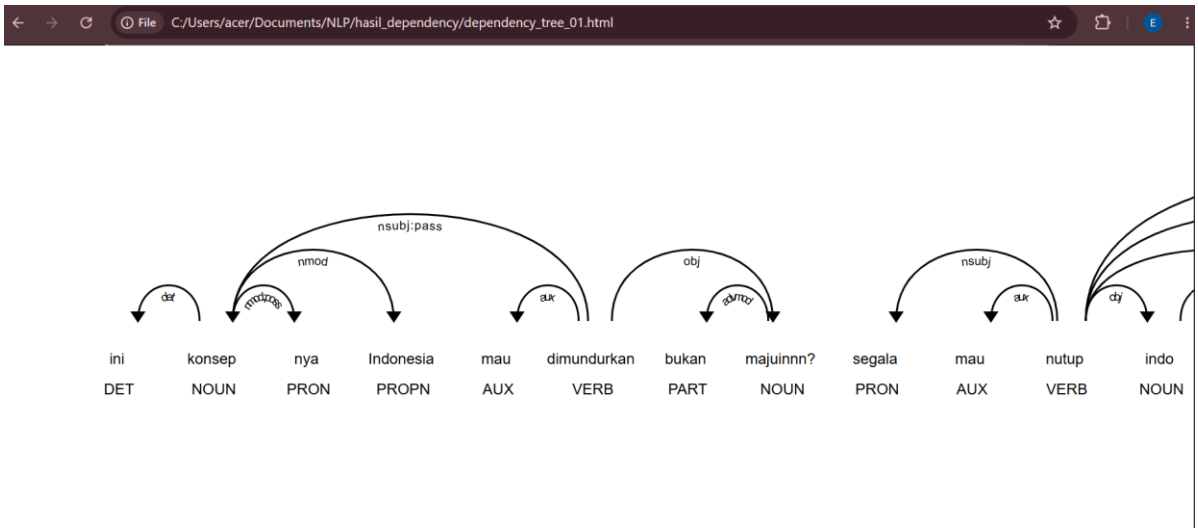
Tabel menampilkan token dan atribut linguistik untuk kalimat yang diproses. Informasi tersebut membantu meninjau kelas kata dan relasi yang diprediksi model.

Hasil analisis kalimat berikutnya



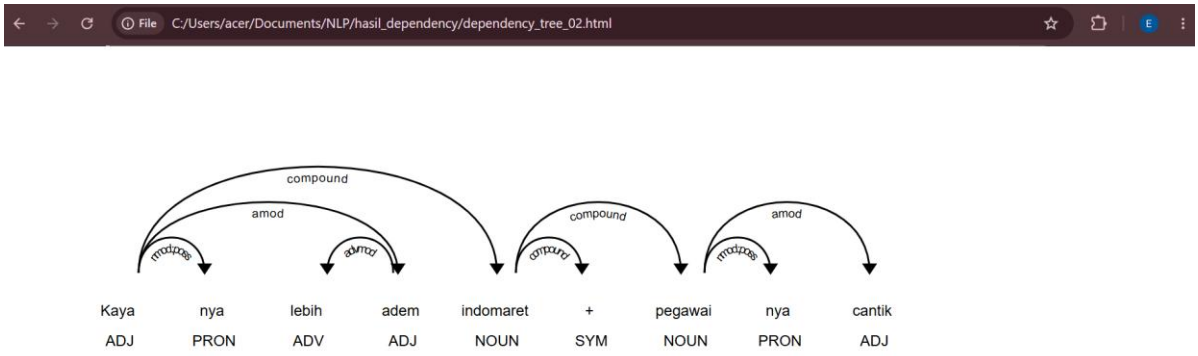
Tabel ini menunjukkan hasil analisis untuk data kalimat lain. Membandingkan beberapa kalimat dapat membantu menemukan bagian yang diprediksi dengan tepat atau perlu diperiksa.

Visualisasi struktur kalimat



Pohon dependensi ditampilkan untuk memperlihatkan hubungan kata secara visual. Panah atau busur pada pohon membantu mengidentifikasi hubungan antar-token.

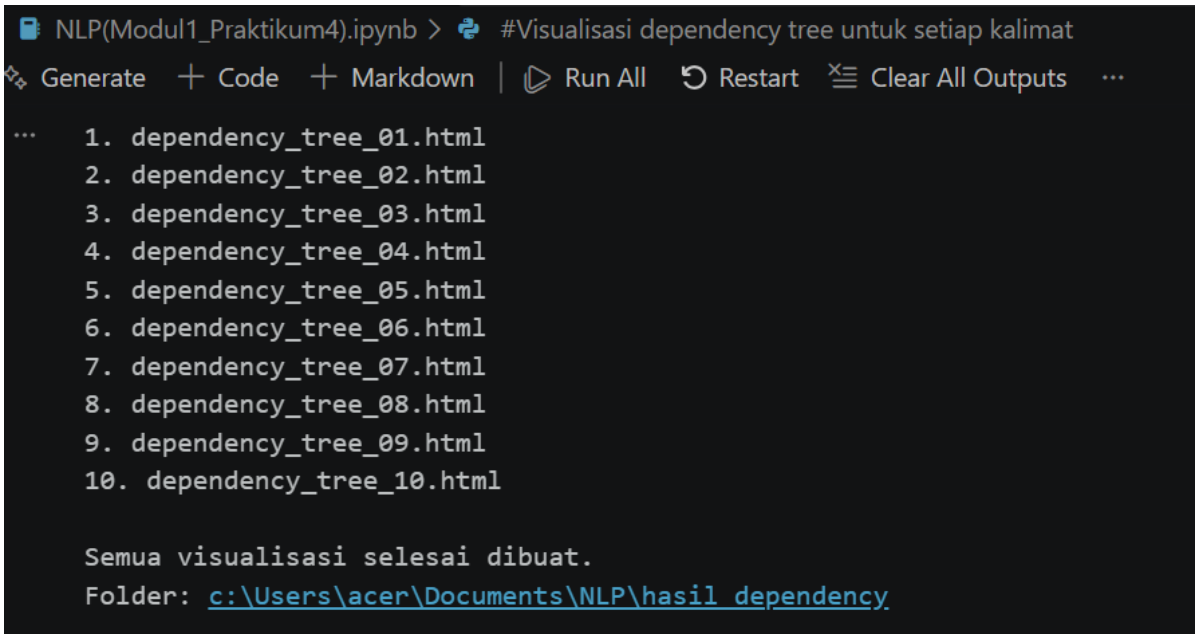
Contoh tambahan dependency tree



Activate Windows

Gambar memberikan contoh lain dari visualisasi struktur dependensi. Setiap pohon merepresentasikan hasil parsing dari kalimat yang berbeda.

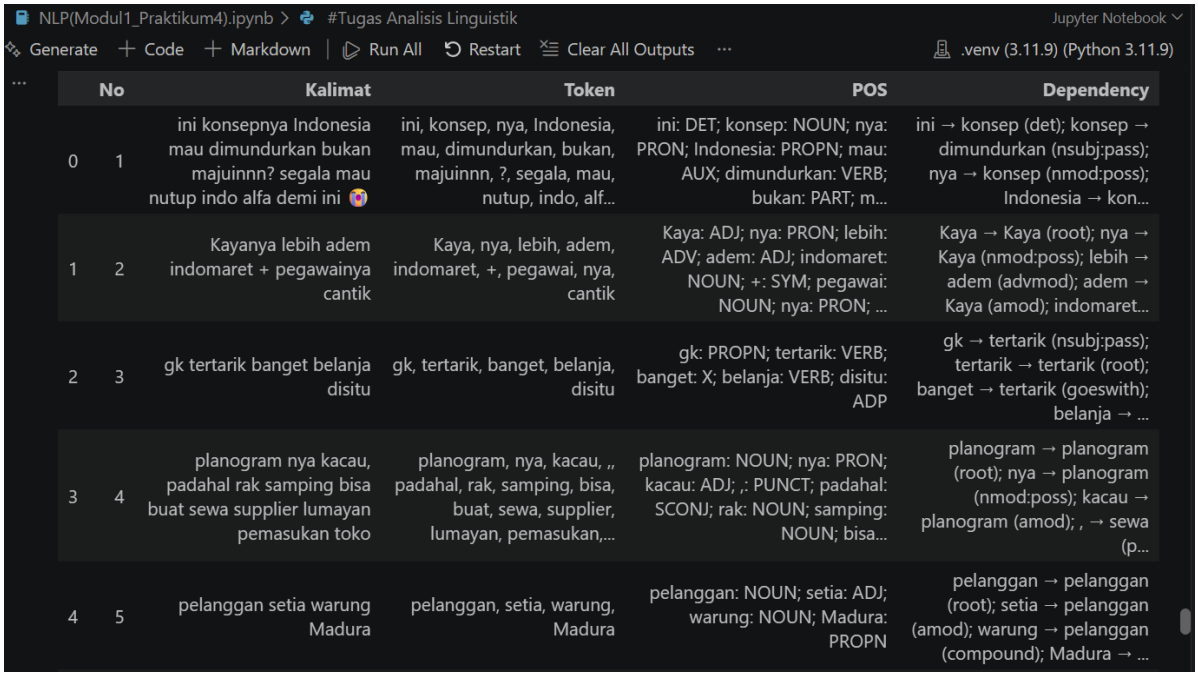
Pemeriksaan berkas hasil visualisasi



Daftar berkas menunjukkan bahwa hasil dependency tree telah disimpan sebagai berkas HTML untuk kalimat-kalimat uji. Berkas ini merupakan keluaran visualisasi yang dapat dibuka kembali.

E. Tugas Analisis Linguistik

Tabel analisis linguistik bagian pertama

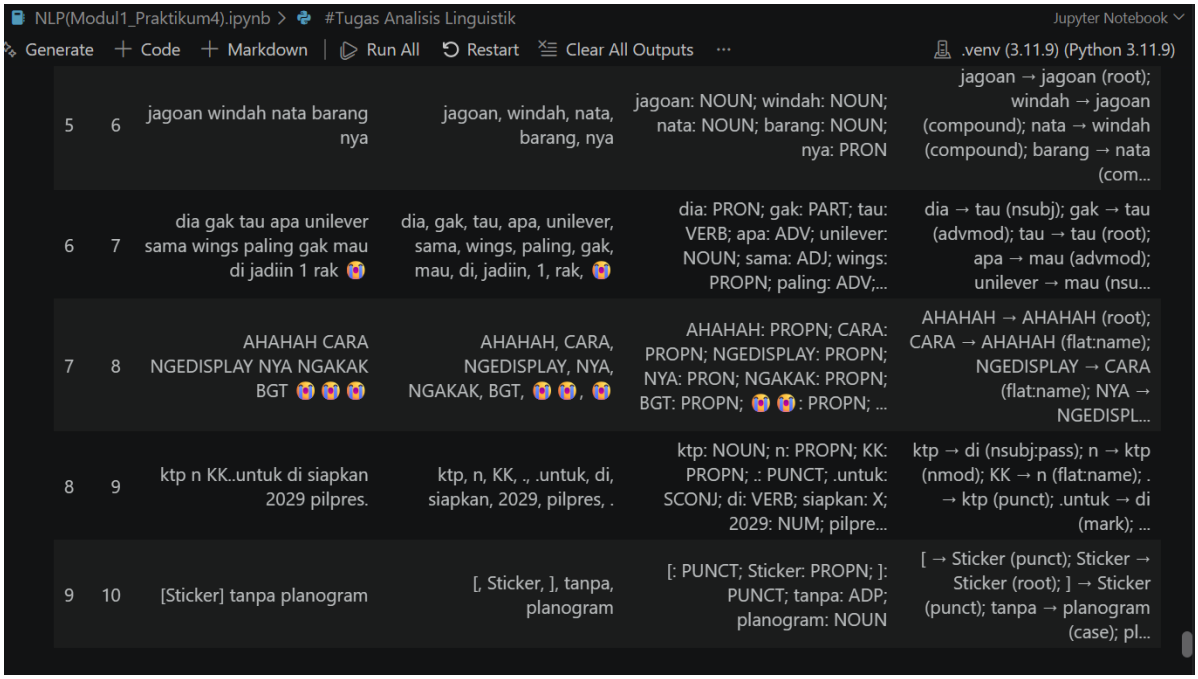

NLP(Modul1_Praktikum4).ipynb > #Tugas Analisis Linguistik — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

	No	Kalimat	Token	POS	Dependency
0	1	ini konsepnya Indonesia mau dimundurkan bukan majuinnn? segala mau nutup indo alfa demi ini	ini, konsep, nya, Indonesia, mau, dimundurkan, bukan, majuinnn, ?, segala, mau, nutup, indo, alf...	ini: DET; konsep: NOUN; nya: PRON; Indonesia: PROPN; mau: AUX; dimundurkan: VERB; bukan: PART; m...	ini → konsep (det); konsep → dimundurkan (nsubj:pass); nya → konsep (nmod:poss); Indonesia → kon...
1	2	Kayanya lebih adem indomaret + pegawainya cantik	Kaya, nya, lebih, adem, indomaret, +, pegawai, nya, cantik	Kaya: ADJ; nya: PRON; lebih: ADV; adem: ADJ; indomaret: NOUN; +: SYM; pegawai: NOUN; nya: PRON; ...	Kaya → Kaya (root); nya → Kaya (nmod:poss); lebih → adem (advmod); adem → Kaya (amod); indomaret...
2	3	gk tertarik banget belanja disitu	gk, tertarik, banget, belanja, disitu	gk: PROPN; tertarik: VERB; banget: X; belanja: VERB; disitu: ADP	gk → tertarik (nsubj:pass); tertarik → tertarik (root); banget → tertarik (goeswith); belanja → ...
3	4	planogram nya kacau, padahal rak samping bisa buat sewa supplier lumayan pemasukan toko	planogram, nya, kacau, ,, padahal, rak, samping, bisa, buat, sewa, supplier, lumayan, pemasukan,...	planogram: NOUN; nya: PRON; kacau: ADJ; ,: PUNCT; padahal: SCONJ; rak: NOUN; samping: NOUN; bisa...	planogram → planogram (root); nya → planogram (nmod:poss); kacau → planogram (amod); , → sewa (p...
4	5	pelanggan setia warung Madura	pelanggan, setia, warung, Madura	pelanggan: NOUN; setia: ADJ; warung: NOUN; Madura: PROPN	pelanggan → pelanggan (root); setia → pelanggan (amod); warung → pelanggan (compound); Madura → ...

Tabel merangkum hasil analisis sejumlah kalimat, termasuk informasi token dan atribut linguistiknya. Data ini menjadi dasar untuk menelaah bagaimana model menangani kalimat Bahasa Indonesia.

Tabel analisis linguistik bagian berikutnya

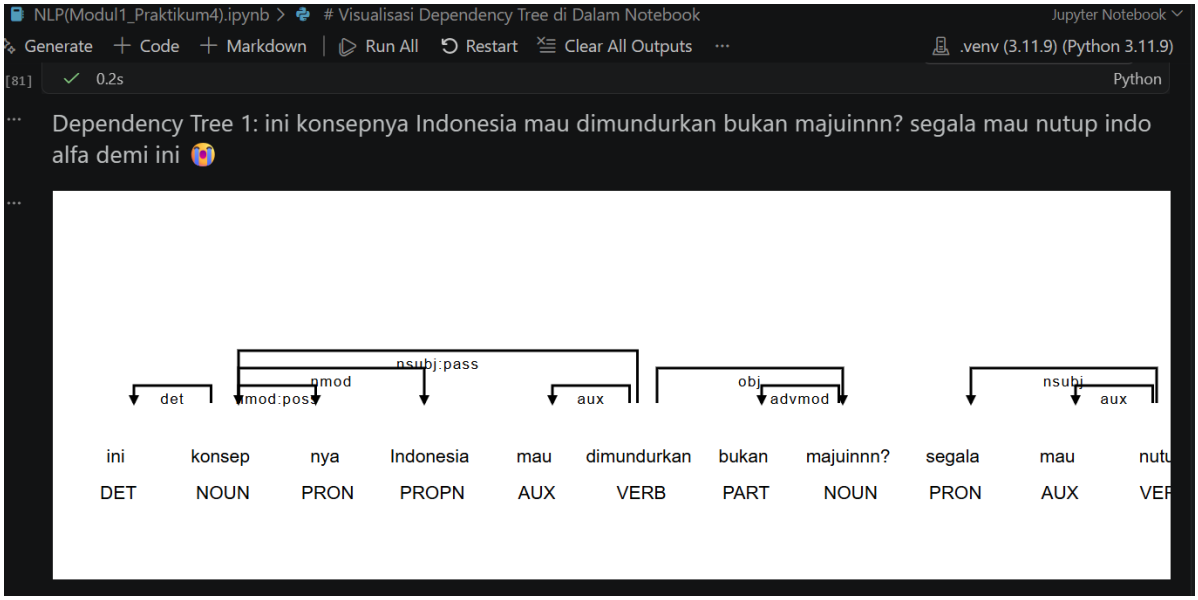

NLP(Modul1_Praktikum4).ipynb > #Tugas Analisis Linguistik — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | .venv (3.11.9) (Python 3.11.9)

5	6	jagoan windah nata barang nya	jagoan, windah, nata, barang, nya	jagoan: NOUN; windah: NOUN; nata: NOUN; barang: NOUN; nya: PRON	jagoan → jagoan (root); windah → jagoan (compound); nata → windah (compound); barang → nata (com...
6	7	dia gak tau apa unilever sama wings paling gak mau di jadiin 1 rak	dia, gak, tau, apa, unilever, sama, wings, paling, gak, mau, di, jadiin, 1, rak,	dia: PRON; gak: PART; tau: VERB; apa: ADV; unilever: NOUN; sama: ADJ; wings: PROPN; paling: ADV;...	dia → tau (nsubj); gak → tau (advmod); tau → tau (root); apa → mau (advmod); unilever → mau (nsu...
7	8	AHAHAH CARA NGEDISPLAY NYA NGAKAK BGT	AHAHAH, CARA, NGEDISPLAY, NYA, NGAKAK, BGT, , ,	AHAHAH: PROPN; CARA: PROPN; NGEDISPLAY: PROPN; NYA: PRON; NGAKAK: PROPN; BGT: PROPN; : PROPN; ...	AHAHAH → AHAHAH (root); CARA → AHAHAH (flat:name); NGEDISPLAY → CARA (flat:name); NYA → NGEDISPL...
8	9	ktp n KK..untuk di siapkan 2029 pilpres.	ktp, n, KK, ., .untuk, di, siapkan, 2029, pilpres, .	ktp: NOUN; n: PROPN; KK: PROPN; .: PUNCT; .untuk: SCONJ; di: VERB; siapkan: X; 2029: NUM; pilpre...	ktp → di (nsubj:pass); n → ktp (nmod); KK → n (flat:name); . → ktp (punct); .untuk → di (mark); ...
9	10	[Sticker] tanpa planogram	[, Sticker,], tanpa, planogram	[: PUNCT; Sticker: PROPN;]: PUNCT; tanpa: ADP; planogram: NOUN	[→ Sticker (punct); Sticker → Sticker (root);] → Sticker (punct); tanpa → planogram (case); pl...

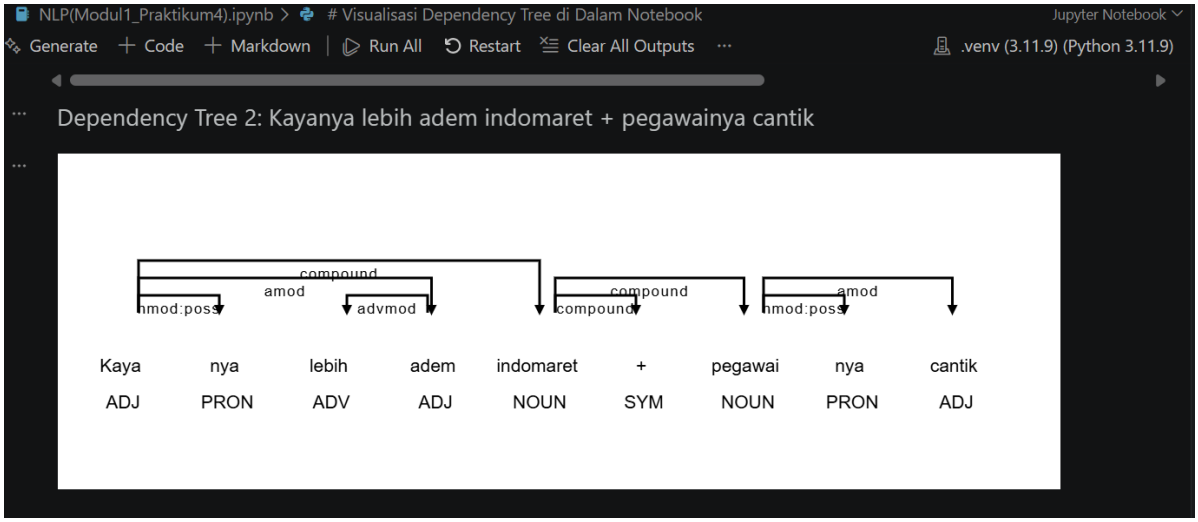
Tabel lanjutan memuat hasil untuk kalimat-kalimat lain. Perbandingan antarkalimat membantu melihat pola label POS dan dependensi yang dihasilkan model.

Dependency Tree 1



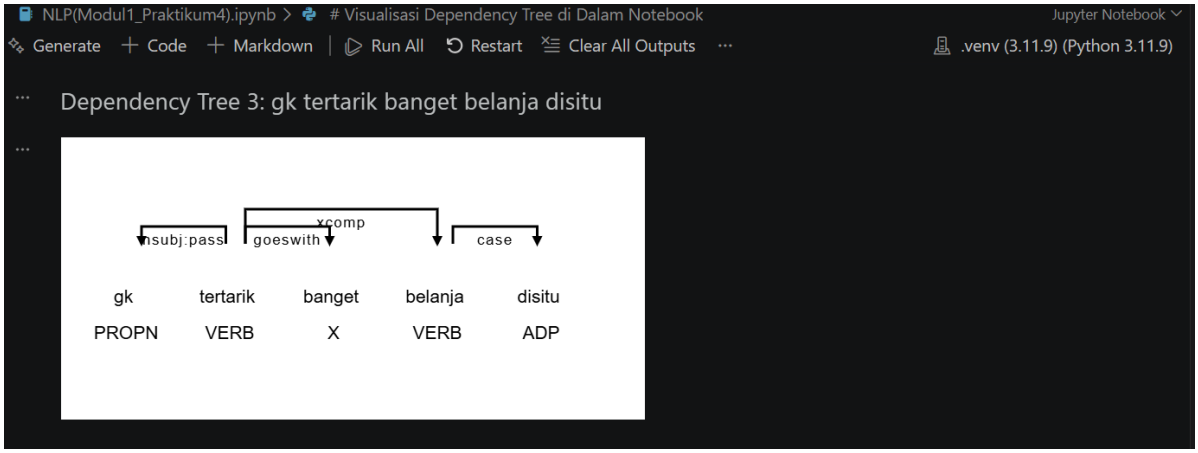
Pohon ini memperlihatkan struktur dependensi kalimat uji pertama. Relasi antarkata dapat dibaca dari kata induk (head) menuju kata yang bergantung padanya.

Dependency Tree 2



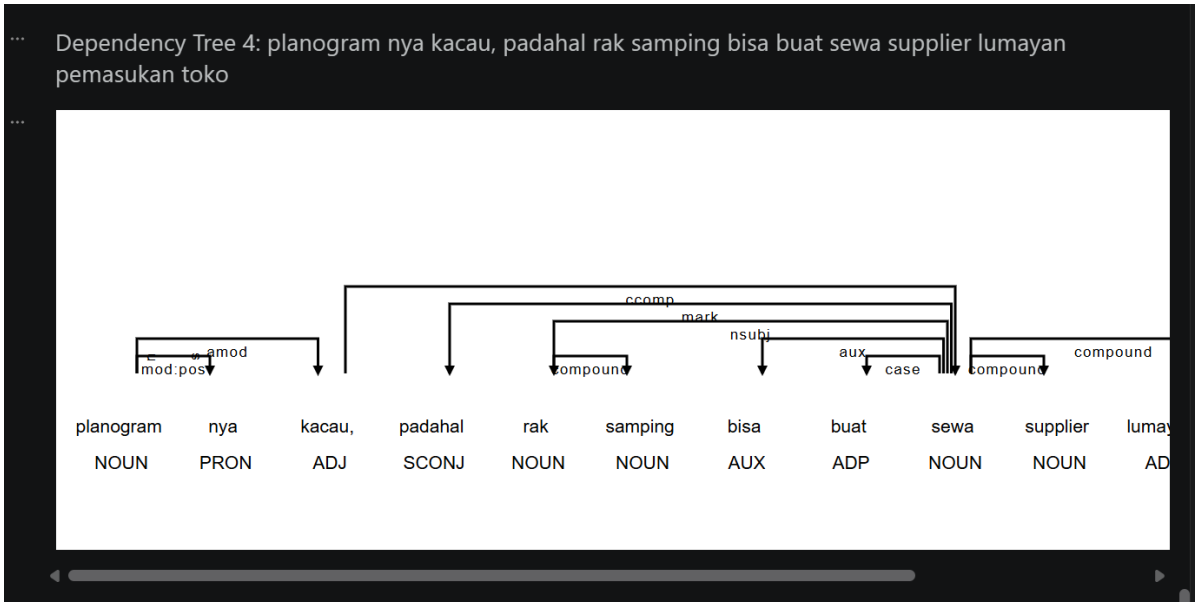
Visualisasi kalimat kedua menunjukkan susunan relasi sintaksis hasil parsing. Struktur ini membantu meninjau apakah kata-kata terhubung sesuai makna kalimat.

Dependency Tree 3



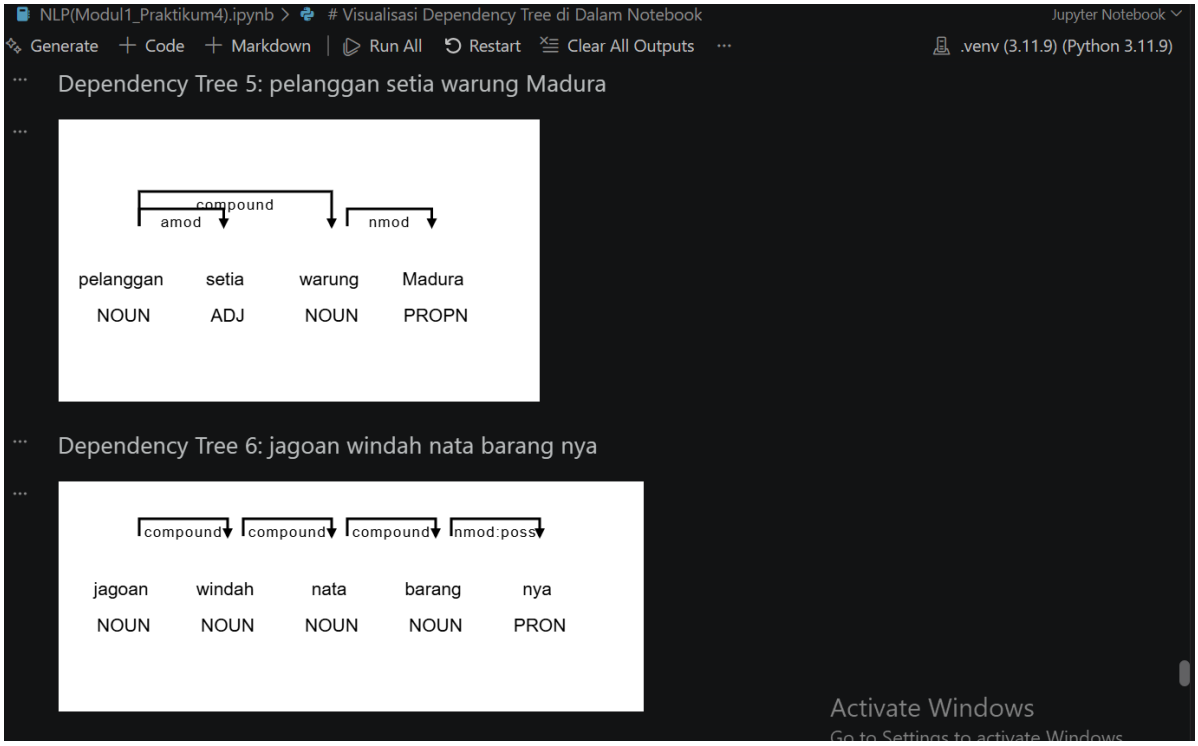
Pohon dependensi kalimat ketiga menampilkan hubungan gramatikal antartoken. Hasil visual perlu dibaca bersama kalimat aslinya karena model dapat keliru pada kata informal atau susunan yang tidak baku.

Dependency Tree 4



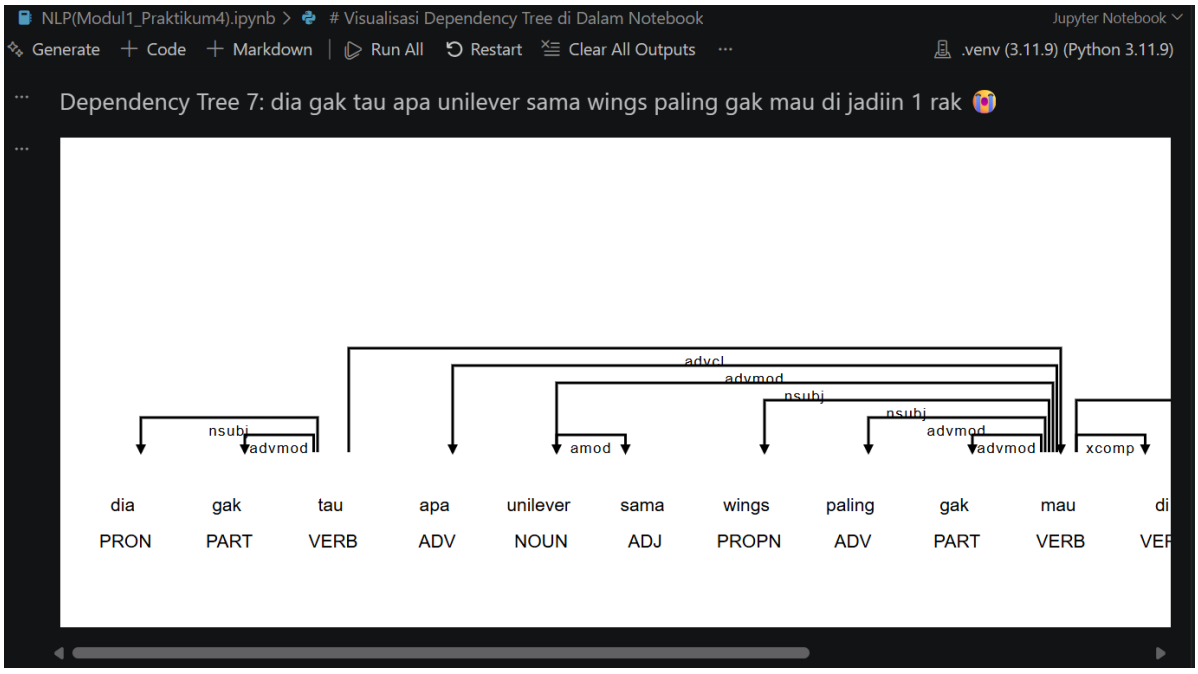
Visualisasi ini menunjukkan relasi kata pada kalimat keempat. Struktur yang panjang atau kompleks dapat membuat beberapa hubungan lebih sulit dibaca, sehingga perlu dibandingkan dengan konteks kalimat.

Dependency Tree 5 dan 6



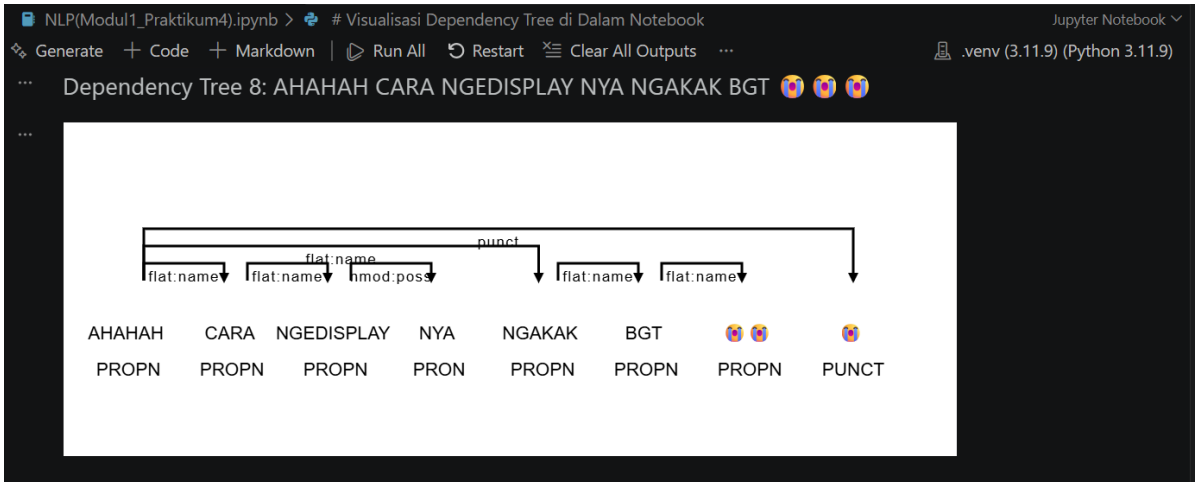
Gambar memuat visualisasi untuk kalimat kelima dan keenam. Masing-masing pohon memperlihatkan prediksi relasi antarkata dari model.

Dependency Tree 7



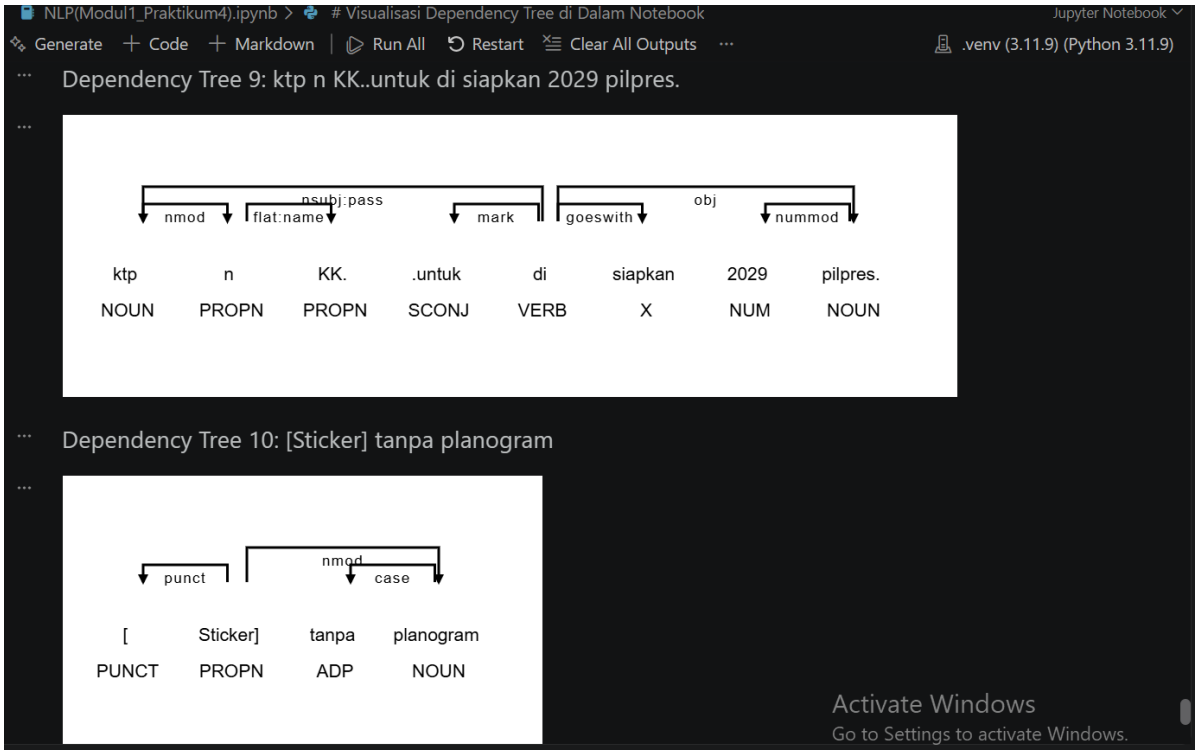
Pohon kalimat ketujuh membantu mengamati hubungan antar-token dalam kalimat uji. Ketepatan relasi tetap perlu diperiksa berdasarkan susunan dan makna kalimat.

Dependency Tree 8



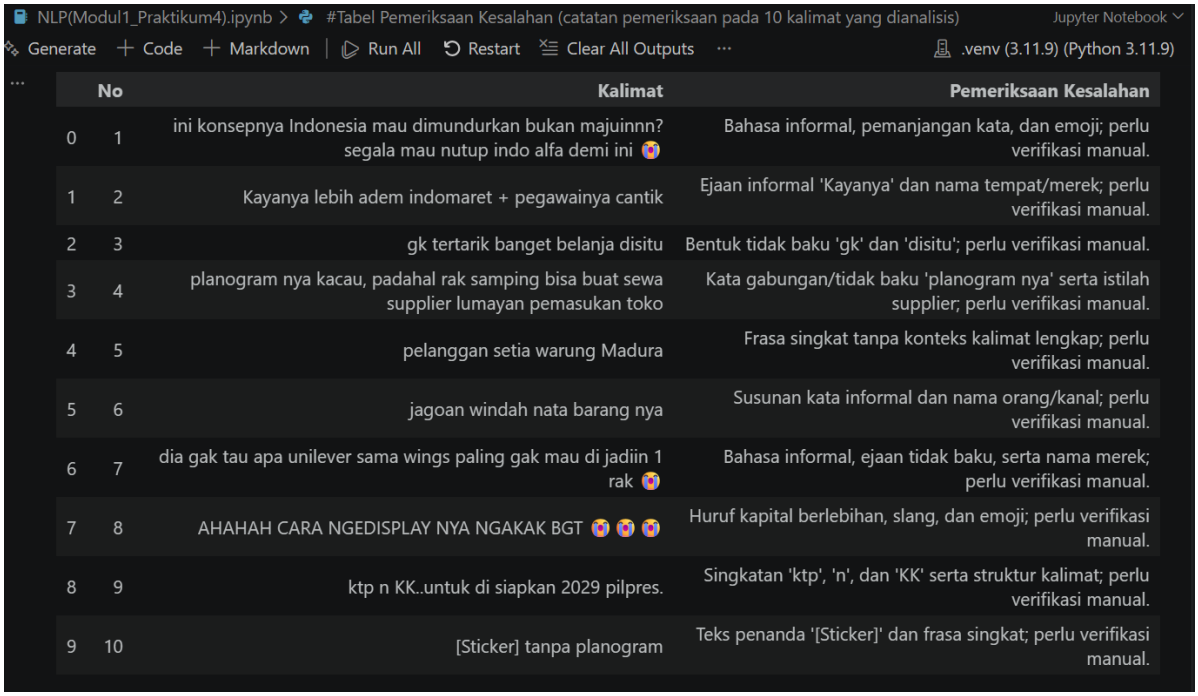
Visualisasi kalimat kedelapan menampilkan struktur dependensi hasil parsing. Pohon ini dapat digunakan untuk mengenali kata utama serta kata-kata yang terkait dengannya.

Dependency Tree 9 dan 10



Gambar menampilkan pohon untuk kalimat kesembilan dan kesepuluh. Dua hasil ini melengkapi pemeriksaan visual terhadap sepuluh kalimat uji.

Rekapitulasi tugas analisis linguistik


NLP(Modul1_Praktikum4).ipynb > #Tabel Pemeriksaan Kesalahan (catatan pemeriksaan pada 10 kalimat yang dianalisis) Jupyter Notebook

Generate + Code + Markdown | Run All Restart Clear All Outputs ... .venv (3.11.9) (Python 3.11.9)

	No	Kalimat	Pemeriksaan Kesalahan
0	1	ini konsepnya Indonesia mau dimundurkan bukan majuinnn? segala mau nutup indo alfa demi ini	Bahasa informal, pemanjangan kata, dan emoji; perlu verifikasi manual.
1	2	Kayanya lebih adem indomaret + pegawainya cantik	Ejaan informal 'Kayanya' dan nama tempat/merek; perlu verifikasi manual.
2	3	gk tertarik banget belanja disitu	Bentuk tidak baku 'gk' dan 'disitu'; perlu verifikasi manual.
3	4	planogram nya kacau, padahal rak samping bisa buat sewa supplier lumayan pemasukan toko	Kata gabungan/tidak baku 'planogram nya' serta istilah supplier; perlu verifikasi manual.
4	5	pelanggan setia warung Madura	Frasa singkat tanpa konteks kalimat lengkap; perlu verifikasi manual.
5	6	jagoan windah nata barang nya	Susunan kata informal dan nama orang/kanal; perlu verifikasi manual.
6	7	dia gak tau apa unilever sama wings paling gak mau di jadiin 1 rak	Bahasa informal, ejaan tidak baku, serta nama merek; perlu verifikasi manual.
7	8	AHAHAH CARA NGEDISPLAY NYA NGAKAK BGT	Huruf kapital berlebihan, slang, dan emoji; perlu verifikasi manual.
8	9	ktp n KK..untuk di siapkan 2029 pilpres.	Singkatan 'ktp', 'n', dan 'KK' serta struktur kalimat; perlu verifikasi manual.
9	10	[Sticker] tanpa planogram	Teks penanda '[Sticker]' dan frasa singkat; perlu verifikasi manual.

Tabel rekapitulasi merangkum analisis untuk kalimat-kalimat uji. Rekap ini dapat dipakai bersama dependency tree untuk membahas prediksi model dan kemungkinan kesalahan analisis.

F. Pertanyaan Diskusi

1. Apakah POS Tagging selalu menghasilkan label yang benar?

   Tidak. POS tagging merupakan prediksi model sehingga hasilnya dapat kurang tepat, terutama pada kalimat ambigu, bahasa informal, singkatan, atau kata yang tidak umum.

2. Apa penyebab utama kesalahan POS Tagging pada teks Bahasa Indonesia informal?

   Penyebabnya dapat berupa singkatan, bahasa gaul, salah ketik, ejaan tidak baku, pemanjangan huruf, emoji, campuran bahasa, dan konteks yang terbatas.

3. Bagaimana slang memengaruhi POS Tagging?

   Model mungkin tidak mengenali slang sebagai bentuk yang lazim. Akibatnya, kata dapat diberi label berdasarkan kemiripan bentuk atau konteks yang tidak sesuai.

4. Bagaimana nama organisasi seperti GOTO dan Tokopedia memengaruhi POS Tagging?

   Model bisa saja tidak mengenali nama organisasi sebagai entitas khusus dan memberikan label kelas kata yang kurang sesuai. Hasilnya perlu diperiksa dalam konteks kalimat.

5. Mengapa parsing penting dalam NLP?

   Parsing membantu menunjukkan hubungan gramatikal antarkata, misalnya kata yang menjadi subjek, objek, atau inti kalimat. Informasi ini berguna untuk memahami struktur kalimat dan mendukung tugas NLP lain.

6. Apa hubungan morfologi dengan stemming?

   Morfologi mempelajari struktur dan pembentukan kata, termasuk imbuhan. Stemming merupakan proses komputasional untuk mengubah kata berimbuhan ke bentuk dasar atau stem.

7. Apakah stemming dan POS Tagging menyelesaikan permasalahan yang sama?

   Tidak. Stemming berfokus pada bentuk kata, sedangkan POS tagging menentukan kelas kata berdasarkan penggunaan dan konteksnya. Keduanya dapat saling melengkapi.

G. Daftar Pustaka

Adriani, M., Asian, J., Nazief, B., Tahaghoghi, S. M. M., & Williams, H. E. (2007). Stemming Indonesian: A confix-stripping approach. *ACM Transactions on Asian Language Information Processing, 6*(4), 1–33.

Alwi, H., Dardjowidjojo, S., Lapoliwa, H., & Moeliono, A. M. (2003). *Tata bahasa baku bahasa Indonesia* (Edisi ketiga). Balai Pustaka.

de Marneffe, M.-C., Manning, C. D., Nivre, J., & Zeman, D. (2021). Universal Dependencies. *Computational Linguistics, 47*(2), 255–308.

Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing* (3rd ed. draft). https://web.stanford.edu/~jurafsky/slp3/

Kübler, S., McDonald, R., & Nivre, J. (2009). *Dependency parsing*. Morgan & Claypool Publishers.

Petrov, S., Das, D., & McDonald, R. (2012). A universal part-of-speech tagset. *Proceedings of LREC 2012*, 2089–2096.

Modul	1 — Fundamental dan Text Processing
Topik	N-Gram dan Language Model
Repository	https://github.com/lian180706/NATURAL-LANGUAGE-PROCESSING
Tanggal	-

A. Tujuan

1. Memahami Konsep Language Model
2. Memahami Probabilitas Kata Dalam Sequence
3. Membuat Unigram, Bigram, Dan Trigram
4. Menghitung Conditional Probability
5. Menghitung Probabilitas Kalimat
6. Memahami Masalah Zero Probability Dan Smoothing
7. Membuat Prediksi Kata Sederhana
8. Menghitung Perplexity
9. Memahami Keterbatasan N-Gram.

B. Requirement

1. Sistem Operasi yang digunakan : Microsoft Windows 11 Pro
2. Browser : Versi 153.0.8010.50 (Build Resmi) (64 bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Language Model

Language Model atau model bahasa merupakan model komputasi yang digunakan untuk memperkirakan probabilitas suatu rangkaian kata dalam bahasa. Model ini dapat menentukan kata yang kemungkinan muncul berikutnya berdasarkan kata-kata sebelumnya. Dalam bidang Natural Language Processing (NLP), language model digunakan dalam berbagai aplikasi, seperti prediksi kata, pemeriksaan ejaan, penerjemahan mesin, dan pembangkitan teks (Jurafsky & Martin, 2026).

Sebagai contoh, kalimat "Saya belajar NLP" cenderung lebih mudah dikenali sebagai rangkaian kata yang wajar dibandingkan rangkaian kata acak. Model bahasa mempelajari pola tersebut berdasarkan data teks atau corpus yang digunakan dalam proses pelatihan.

2. Probabilitas Sequence Kata

Probabilitas sequence merupakan kemungkinan munculnya suatu rangkaian kata secara berurutan. Jika sebuah kalimat terdiri atas kata w1, w2, sampai wn, probabilitas kalimat tersebut dituliskan sebagai P(w1, w2, ..., wn).

Berdasarkan chain rule dalam teori probabilitas, probabilitas rangkaian kata dapat diuraikan menjadi perkalian probabilitas kata pertama dan probabilitas setiap kata berikutnya berdasarkan kata-kata yang mendahuluinya.

Rumus:

$$P(w1, w2, ..., wn) = P(w1) \times P(w2|w1) \times P(w3|w1,w2) \times ... \times P(wn|w1,...,w(n-1))$$

Pendekatan ini menjadi dasar dalam pemodelan bahasa statistik karena memungkinkan model memperkirakan probabilitas kalimat berdasarkan hubungan antarkata (Manning & Schütze, 1999).

3. N-Gram

N-gram merupakan metode pemodelan bahasa yang menggunakan rangkaian token berurutan untuk mempelajari pola bahasa. Token dapat berupa

kata atau unit teks lainnya, bergantung pada proses tokenisasi yang digunakan. Nilai N menunjukkan jumlah token yang membentuk satu rangkaian.

Jenis-jenis N-gram meliputi:

a. Unigram, yaitu rangkaian yang terdiri atas satu token, misalnya “saya”, “belajar”, dan “NLP”.
b. Bigram, yaitu rangkaian yang terdiri atas dua token berurutan, misalnya “saya belajar” dan “belajar NLP”.
c. Trigram, yaitu rangkaian yang terdiri atas tiga token berurutan, misalnya “saya belajar NLP”.

Model N-gram digunakan untuk menyederhanakan perhitungan probabilitas bahasa dengan memanfaatkan frekuensi kemunculan rangkaian token dalam corpus. Semakin besar nilai N, semakin panjang konteks yang dipertimbangkan, tetapi kebutuhan terhadap data pelatihan juga dapat meningkat (Jurafsky & Martin, 2026).

4. Asumsi Markov

Asumsi Markov merupakan pendekatan yang menyatakan bahwa probabilitas suatu kejadian dapat diperkirakan berdasarkan sejumlah kejadian sebelumnya tanpa harus mempertimbangkan seluruh rangkaian kejadian terdahulu. Dalam language model, asumsi ini digunakan untuk menyederhanakan perhitungan probabilitas kata.

Pada model bigram, probabilitas suatu kata diperkirakan berdasarkan satu kata sebelumnya.

Rumus:

$$P(wn \mid w1, ..., w(n-1)) \approx P(wn \mid w(n-1))$$

Sementara itu, pada model trigram, probabilitas suatu kata diperkirakan berdasarkan dua kata sebelumnya.

Rumus:

$$P(wn \mid w1, ..., w(n-1)) \approx P(wn \mid w(n-2), w(n-1))$$

Asumsi tersebut membuat pemodelan bahasa lebih sederhana karena model hanya perlu memperhitungkan konteks dengan panjang tertentu. Namun, hubungan dengan kata-kata yang lebih jauh tidak dipertimbangkan secara langsung (Manning & Schütze, 1999).

5. Probabilitas Unigram

Probabilitas unigram adalah kemungkinan kemunculan sebuah token berdasarkan frekuensinya dalam seluruh corpus. Perhitungan dilakukan dengan membagi jumlah kemunculan token dengan jumlah keseluruhan token.

Rumus:

$$P(w) = Count(w) / N$$

Keterangan:

P(w) adalah probabilitas token w.

Count(w) adalah jumlah kemunculan token tersebut.

N adalah jumlah seluruh token dalam corpus.

Sebagai contoh, apabila kata “belajar” muncul sebanyak 10 kali dalam corpus yang terdiri atas 100 token, probabilitas unigram-nya adalah 10 dibagi 100, yaitu 0,1. Nilai ini menunjukkan bahwa kata tersebut menempati 10 persen dari seluruh token dalam corpus.

6. Probabilitas Bigram

Probabilitas bigram merupakan probabilitas kemunculan suatu kata berdasarkan kata yang muncul tepat sebelumnya. Perhitungan ini menggunakan frekuensi pasangan kata dibandingkan dengan frekuensi kata pertama pasangan tersebut.

Rumus:

$$P(wn \mid w(n-1)) = Count(w(n-1), wn) / Count(w(n-1))$$

Keterangan:

P(wn | w(n-1)) adalah probabilitas kata berikutnya berdasarkan kata sebelumnya.

Count(w(n-1), wn) adalah frekuensi kemunculan pasangan kata.

Count(w(n-1)) adalah frekuensi kemunculan kata sebelumnya.

Sebagai contoh, apabila kata “saya” muncul sebanyak 20 kali dan pasangan “saya belajar” muncul sebanyak 8 kali, probabilitas kata “belajar” setelah kata “saya” adalah 8 dibagi 20, yaitu 0,4.

Probabilitas bigram membantu model mempelajari hubungan lokal antarkata. Metode ini relatif sederhana dan mudah diterapkan, tetapi hasilnya dipengaruhi oleh jumlah serta keragaman data yang digunakan (Jurafsky & Martin, 2026).

7. Zero Probability

Zero probability merupakan kondisi ketika suatu kata atau rangkaian kata tidak pernah muncul dalam data pelatihan sehingga frekuensi yang tercatat bernilai nol. Pada model bigram tanpa smoothing, kondisi tersebut dapat menghasilkan probabilitas nol untuk pasangan kata yang belum pernah ditemukan.

Contohnya, apabila pasangan kata “deep learning” tidak muncul dalam corpus, perhitungan probabilitas bigram dapat menghasilkan nilai P(learning | deep) sebesar nol.

Masalah ini penting karena probabilitas sebuah kalimat dihitung melalui perkalian probabilitas kata-katanya. Jika salah satu komponennya bernilai nol, probabilitas seluruh kalimat juga menjadi nol. Akibatnya, model dapat memberikan penilaian yang kurang tepat terhadap kalimat yang sebenarnya masuk akal. Salah satu pendekatan untuk mengatasi masalah tersebut adalah smoothing (Chen & Goodman, 1999).

8. Laplace Smoothing

Laplace smoothing atau add-one smoothing merupakan teknik untuk mengatasi probabilitas nol dengan menambahkan satu pada frekuensi setiap pasangan kata sebelum probabilitas dihitung. Teknik ini memberikan peluang kepada pasangan kata yang belum pernah ditemukan dalam data pelatihan.

Rumus:

$$P(wi \mid w(i-1)) = [Count(w(i-1), wi) + 1] / [Count(w(i-1)) + V]$$

Keterangan:

Count(w(i-1), wi) adalah frekuensi pasangan kata.

Count(w(i-1)) adalah frekuensi kata sebelumnya.

V adalah ukuran vocabulary, yaitu jumlah token unik dalam kosakata model.

Penambahan satu pada frekuensi pasangan kata dan penambahan V pada penyebut menjaga agar distribusi probabilitas tetap ternormalisasi. Meskipun sederhana, Laplace smoothing dapat memberikan estimasi yang kurang optimal untuk corpus besar atau data yang jarang sehingga terdapat teknik smoothing lain yang lebih canggih (Chen & Goodman, 1999).

9. Perplexity

Perplexity merupakan metrik evaluasi yang digunakan untuk mengukur kemampuan language model dalam memprediksi rangkaian kata pada data evaluasi. Perplexity berkaitan dengan probabilitas yang diberikan model terhadap data tersebut. Secara umum, nilai perplexity yang lebih rendah menunjukkan bahwa model memberikan prediksi yang lebih baik pada data evaluasi yang sama.

Rumus:

$$PP(W) = P(W)^{(-1/N)}$$

Keterangan:

PP(W) adalah nilai perplexity.

P(W) adalah probabilitas seluruh rangkaian kata.

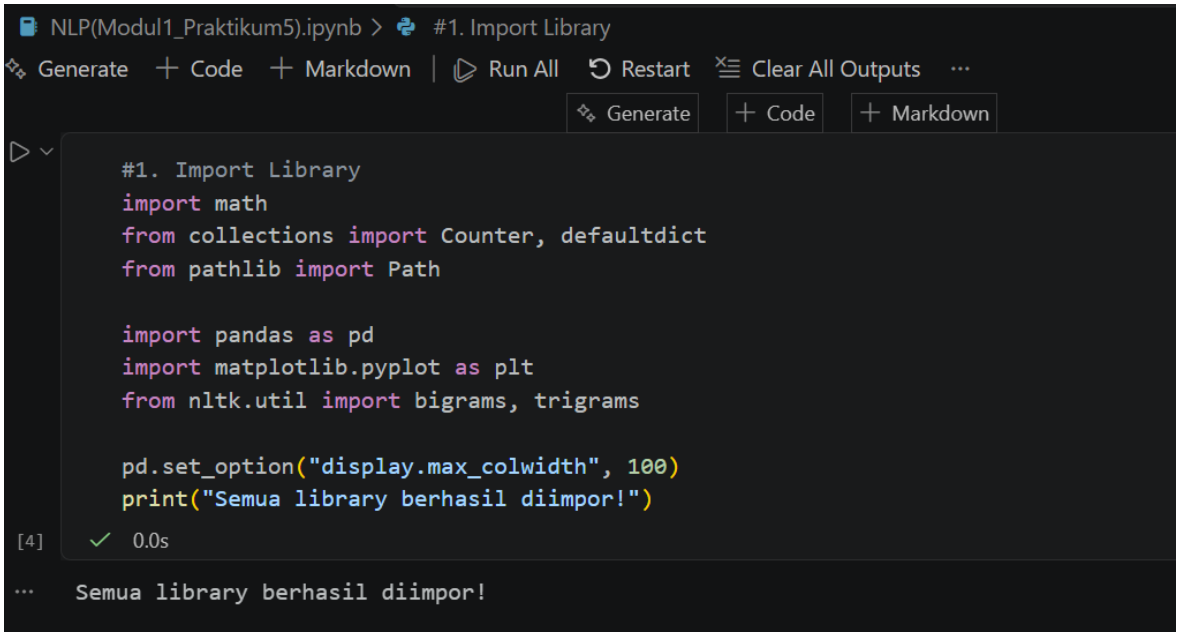
N adalah jumlah token yang dievaluasi.

Dalam praktiknya, perplexity digunakan untuk membandingkan performa model bahasa pada data evaluasi yang sama dengan prosedur tokenisasi dan penghitungan yang sebanding. Nilai yang lebih rendah umumnya menunjukkan model lebih baik dalam memprediksi data tersebut, tetapi tidak secara otomatis

menjamin kualitas bahasa atau kinerja terbaik pada semua tugas NLP (Jurafsky & Martin, 2026).

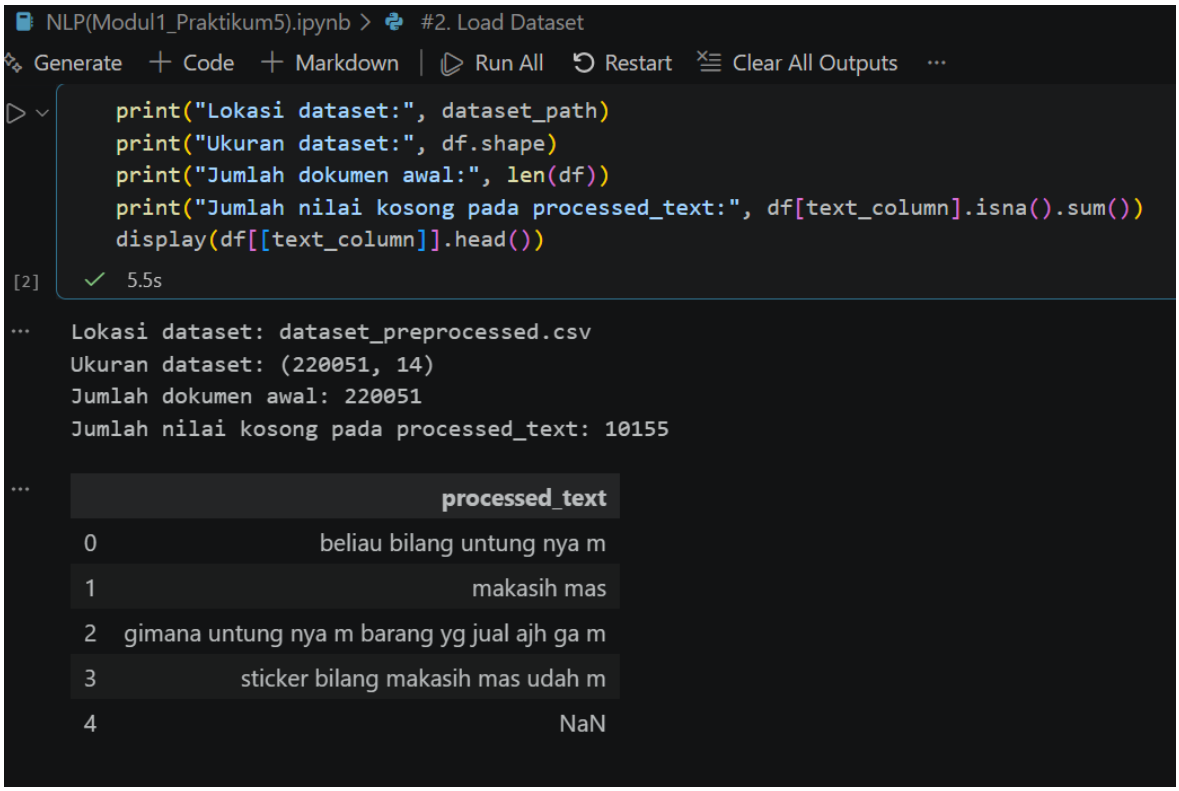
D. Implementasi

Gambar 1. Import library



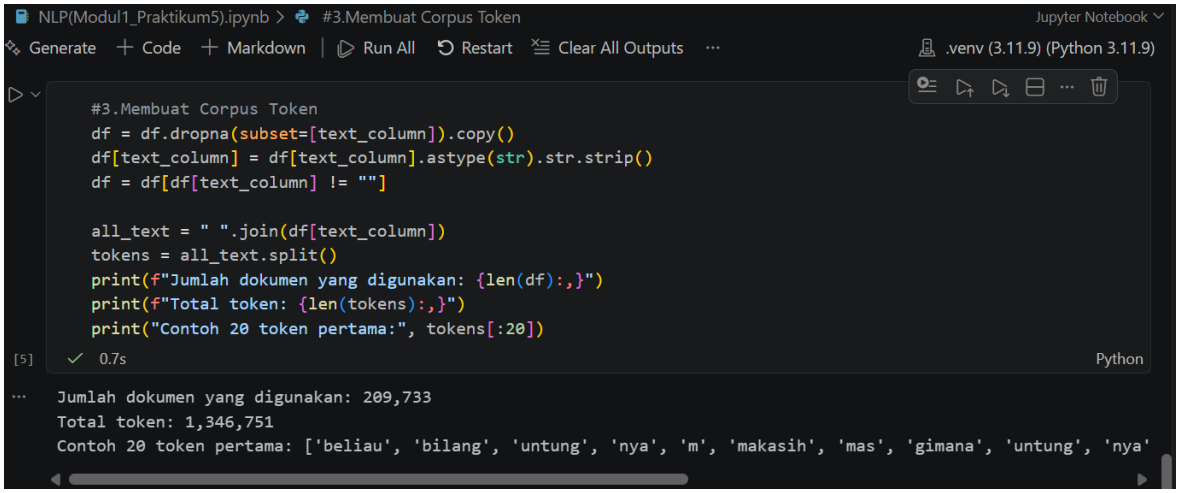
Penjelasan: Kode mengimpor library yang dibutuhkan untuk pengolahan data, perhitungan frekuensi, pembentukan n-gram, dan visualisasi. Counter digunakan untuk menghitung frekuensi token, sedangkan fungsi bigrams dan trigrams membentuk pasangan serta tiga token berurutan.

Gambar 2. Memuat dataset



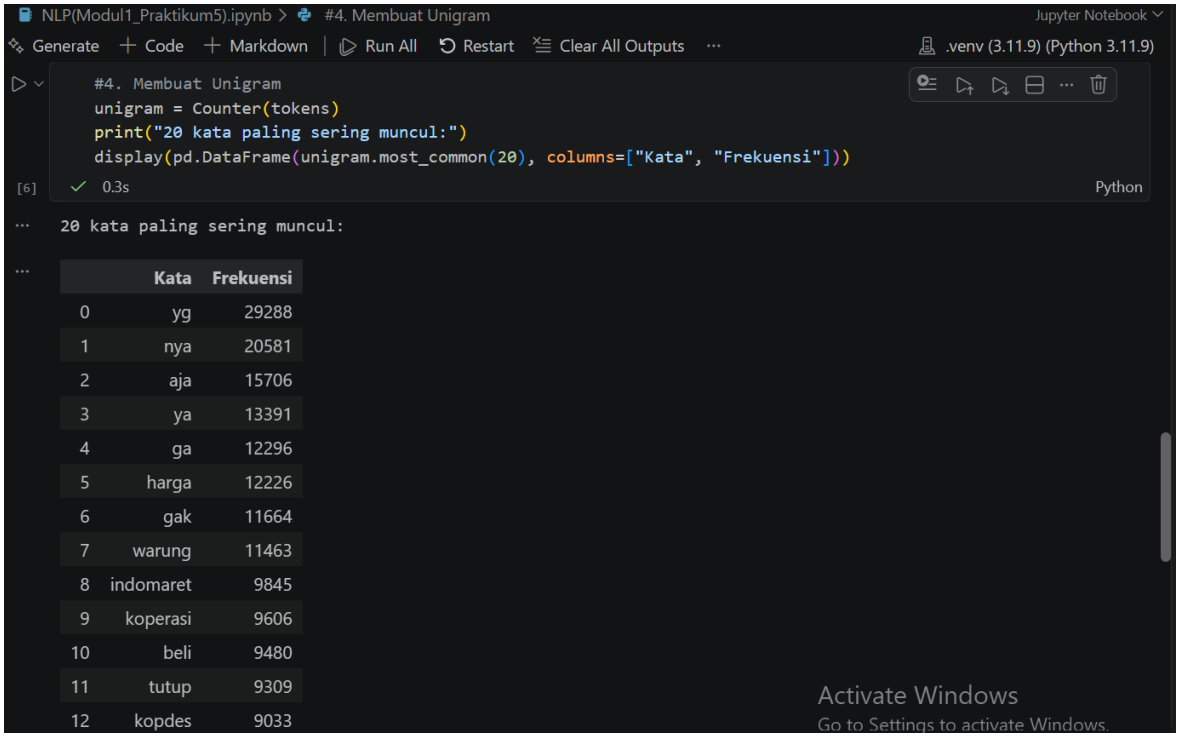
Penjelasan: Dataset dibaca ke dalam DataFrame dan kolom processed_text dipilih sebagai sumber teks. Pemeriksaan jumlah baris dan nilai kosong dilakukan agar kondisi data diketahui sebelum analisis. Baris tanpa teks dipisahkan atau dibersihkan sesuai kode pada notebook.

Gambar 3. Membentuk corpus dan token



Penjelasan: Teks pada kolom processed_text digabung menjadi corpus, lalu dipecah menjadi token. Output menampilkan jumlah dokumen dan token serta contoh token pertama sebagai pemeriksaan bahwa proses pemuatan dan tokenisasi berjalan.

Gambar 4. Frekuensi unigram — bagian pertama



Penjelasan: Counter menghitung berapa kali setiap token muncul. Tabel hasil diurutkan berdasarkan frekuensi sehingga kata yang paling sering muncul dapat dilihat terlebih dahulu.

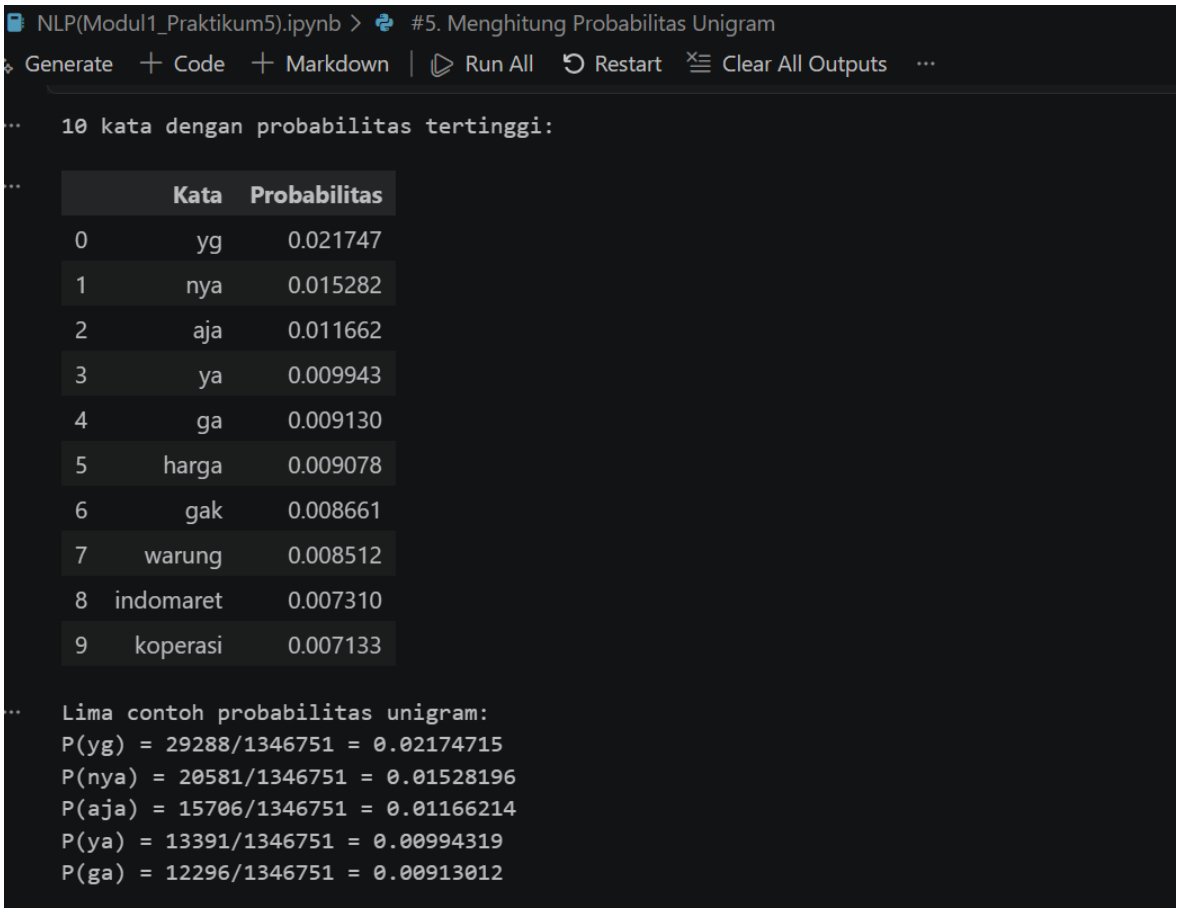
Gambar 5. Frekuensi unigram — lanjutan hasil



13	jual	8958
14	rakyat	8944
15	barang	8602
16	alfa	8544
17	desa	8309
18	toko	8226
19	saing	7793

Penjelasan: Tampilan ini memperlihatkan lanjutan hasil frekuensi kata. Nilai frekuensi menunjukkan jumlah kemunculan kata di seluruh corpus, bukan jumlah dokumen yang mengandung kata tersebut.

Gambar 6. Probabilitas unigram



NLP(Modul1_Praktikum5).ipynb > #5. Menghitung Probabilitas Unigram

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | ...

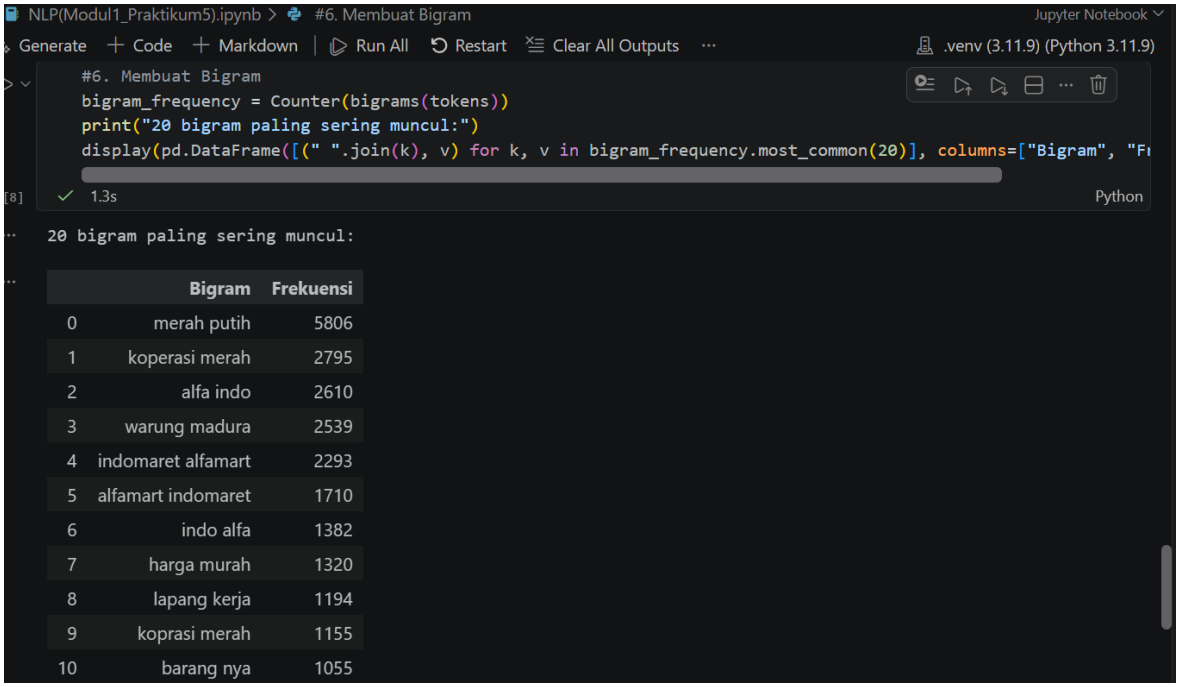
```
10 kata dengan probabilitas tertinggi:
```

	Kata	Probabilitas
0	yg	0.021747
1	nya	0.015282
2	aja	0.011662
3	ya	0.009943
4	ga	0.009130
5	harga	0.009078
6	gak	0.008661
7	warung	0.008512
8	indomaret	0.007310
9	koperasi	0.007133

```
Lima contoh probabilitas unigram:
P(yg) = 29288/1346751 = 0.02174715
P(nya) = 20581/1346751 = 0.01528196
P(aja) = 15706/1346751 = 0.01166214
P(ya) = 13391/1346751 = 0.00994319
P(ga) = 12296/1346751 = 0.00913012
```

Penjelasan: Probabilitas unigram diperoleh dengan membagi frekuensi suatu kata dengan jumlah seluruh token. Hasil mendekati nol untuk kata yang jarang muncul dan lebih besar untuk kata yang lebih sering muncul.

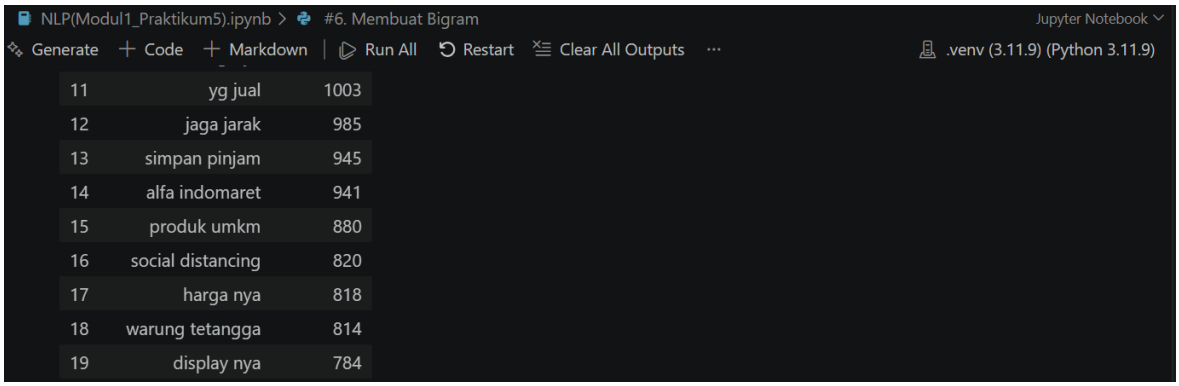
Gambar 7. Frekuensi bigram — bagian pertama



	Bigram	Frekuensi
0	merah putih	5806
1	koperasi merah	2795
2	alfa indo	2610
3	warung madura	2539
4	indomaret alfamart	2293
5	alfamart indomaret	1710
6	indo alfa	1382
7	harga murah	1320
8	lapang kerja	1194
9	koprasi merah	1155
10	barang nya	1055

Penjelasan: Bigram dibentuk dari dua token yang bersebelahan. Tabel menunjukkan pasangan kata beserta jumlah kemunculannya sehingga hubungan lokal antarkata dapat diamati.

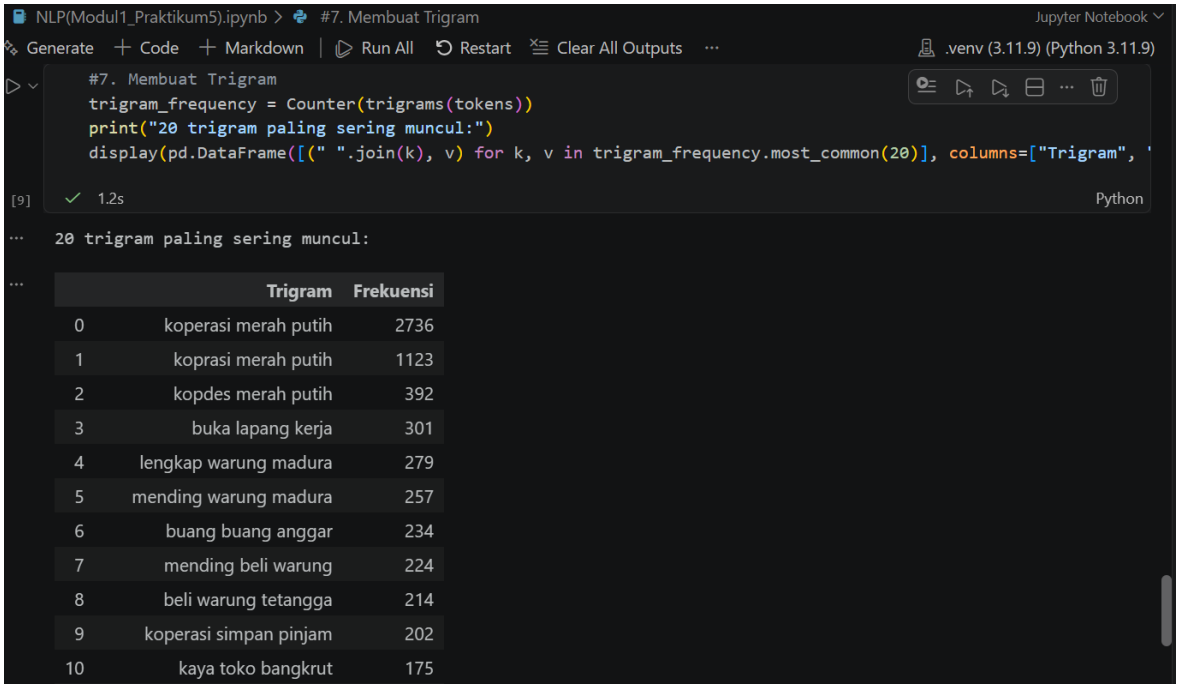
Gambar 8. Frekuensi bigram — lanjutan hasil



11	yg jual	1003
12	jaga jarak	985
13	simpan pinjam	945
14	alfa indomaret	941
15	produk umkm	880
16	social distancing	820
17	harga nya	818
18	warung tetangga	814
19	display nya	784

Penjelasan: Tampilan ini merupakan lanjutan daftar pasangan bigram. Pasangan dengan frekuensi tinggi merupakan pola dua kata yang sering muncul berurutan dalam corpus.

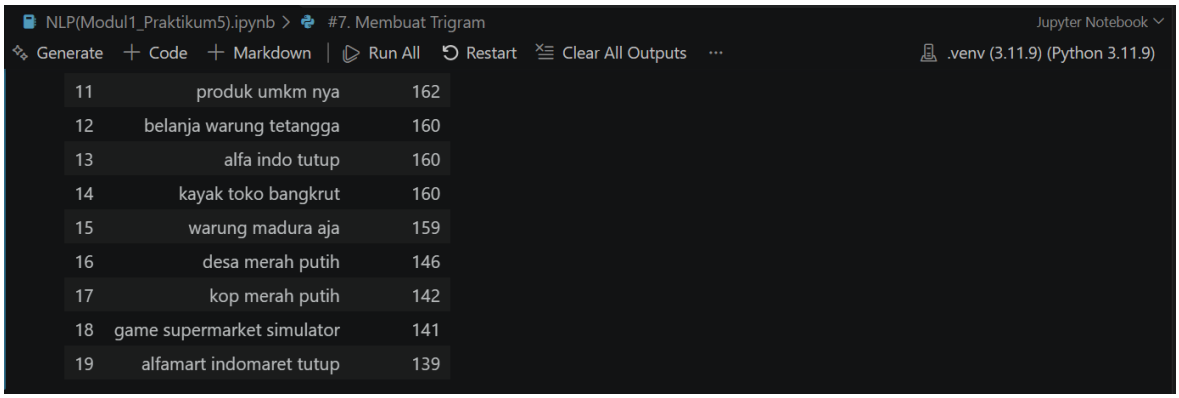
Gambar 9. Frekuensi trigram — bagian pertama



	Trigram	Frekuensi
0	koperasi merah putih	2736
1	koprasi merah putih	1123
2	kopdes merah putih	392
3	buka lapang kerja	301
4	lengkap warung madura	279
5	mending warung madura	257
6	buang buang anggar	234
7	mending beli warung	224
8	beli warung tetangga	214
9	koperasi simpan pinjam	202
10	kaya toko bangkrut	175

Penjelasan: Trigram terdiri atas tiga token berurutan. Hasil frekuensi digunakan untuk melihat pola kata dengan konteks yang lebih panjang dibandingkan unigram dan bigram.

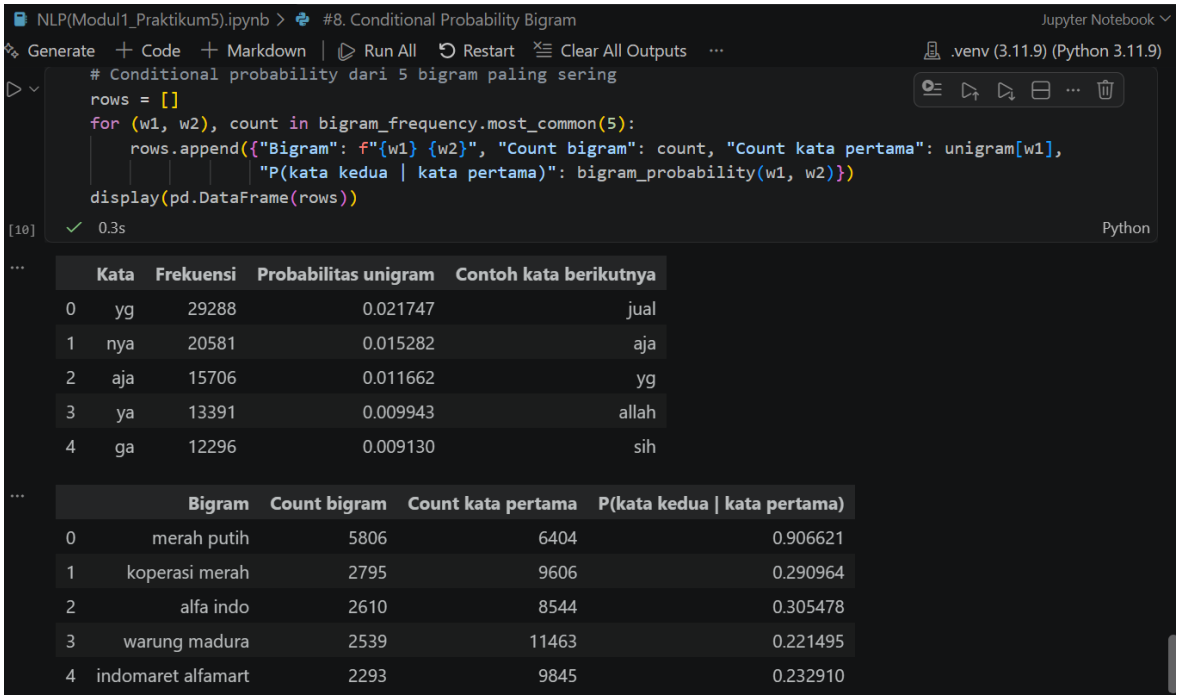
Gambar 10. Frekuensi trigram — lanjutan hasil



11	produk umkm nya	162
12	belanja warung tetangga	160
13	alfa indo tutup	160
14	kayak toko bangkrut	160
15	warung madura aja	159
16	desa merah putih	146
17	kop merah putih	142
18	game supermarket simulator	141
19	alfamart indomaret tutup	139

Penjelasan: Daftar ini melanjutkan hasil penghitungan trigram. Frekuensi trigram cenderung lebih tersebar karena kombinasi tiga kata lebih beragam daripada kombinasi satu atau dua kata.

Gambar 11. Conditional probability dan contoh probabilitas

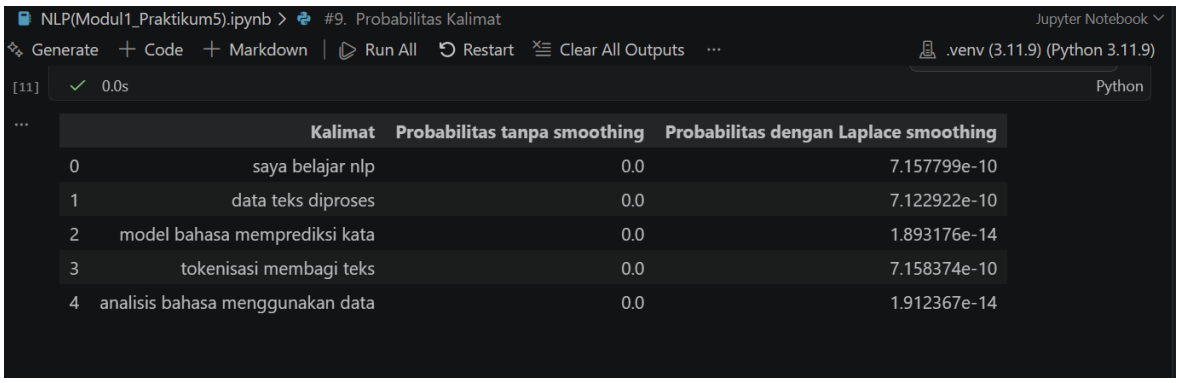


	Kata	Frekuensi	Probabilitas unigram	Contoh kata berikutnya
0	yg	29288	0.021747	jual
1	nya	20581	0.015282	aja
2	aja	15706	0.011662	yg
3	ya	13391	0.009943	allah
4	ga	12296	0.009130	sih

	Bigram	Count bigram	Count kata pertama	P(kata kedua | kata pertama)
0	merah putih	5806	6404	0.906621
1	koperasi merah	2795	9606	0.290964
2	alfa indo	2610	8544	0.305478
3	warung madura	2539	11463	0.221495
4	indomaret alfamart	2293	9845	0.232910

Penjelasan: Kode menghitung probabilitas kata kedua setelah kata pertama dengan membagi frekuensi bigram oleh frekuensi kata pertama. Tabel juga memperlihatkan contoh perhitungan berdasarkan frekuensi yang ditemukan pada corpus.

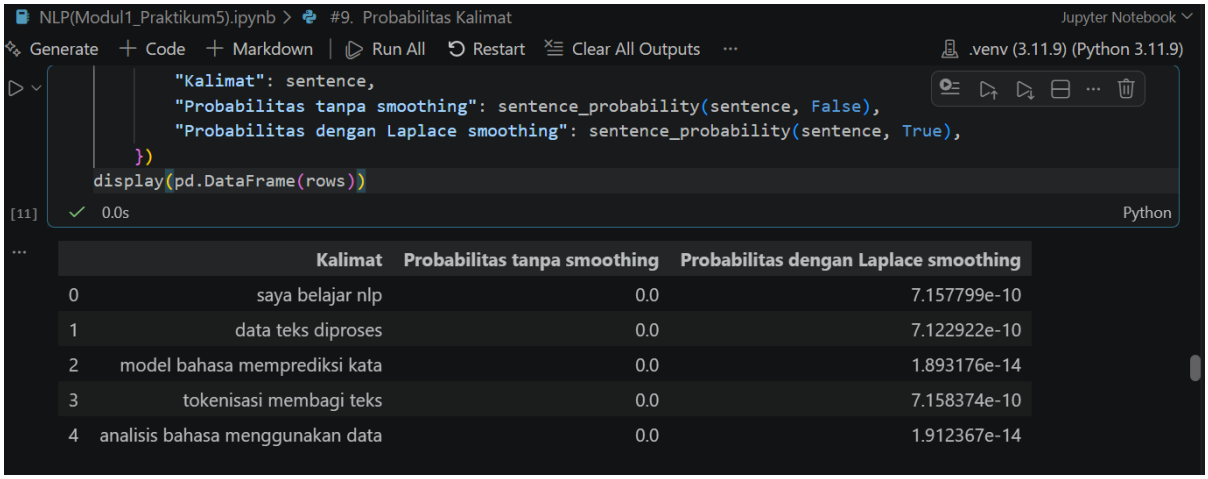
Gambar 12. Perbandingan probabilitas dengan Laplace smoothing



	Kalimat	Probabilitas tanpa smoothing	Probabilitas dengan Laplace smoothing
0	saya belajar nlp	0.0	7.157799e-10
1	data teks diproses	0.0	7.122922e-10
2	model bahasa memprediksi kata	0.0	1.893176e-14
3	tokenisasi membagi teks	0.0	7.158374e-10
4	analisis bahasa menggunakan data	0.0	1.912367e-14

Penjelasan: Output membandingkan probabilitas bigram tanpa smoothing dan dengan Laplace smoothing. Pasangan yang tidak ditemukan dapat memiliki probabilitas nol tanpa smoothing, sedangkan penambahan satu pada frekuensi membuat probabilitas hasil smoothing tetap positif.

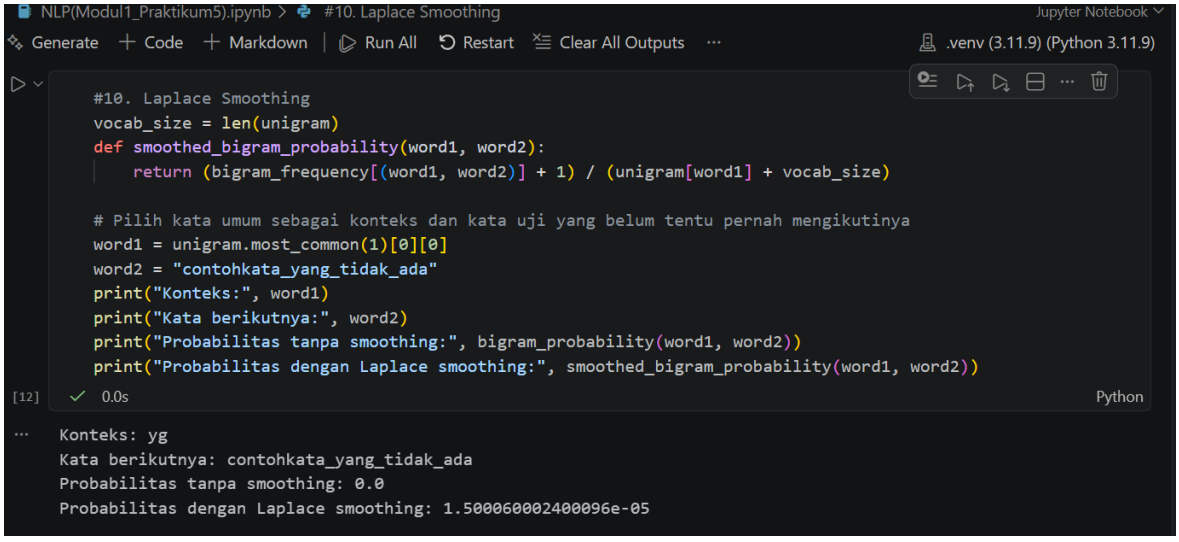
Gambar 13. Probabilitas kalimat



	Kalimat	Probabilitas tanpa smoothing	Probabilitas dengan Laplace smoothing
0	saya belajar nlp	0.0	7.157799e-10
1	data teks diproses	0.0	7.122922e-10
2	model bahasa memprediksi kata	0.0	1.893176e-14
3	tokenisasi membagi teks	0.0	7.158374e-10
4	analisis bahasa menggunakan data	0.0	1.912367e-14

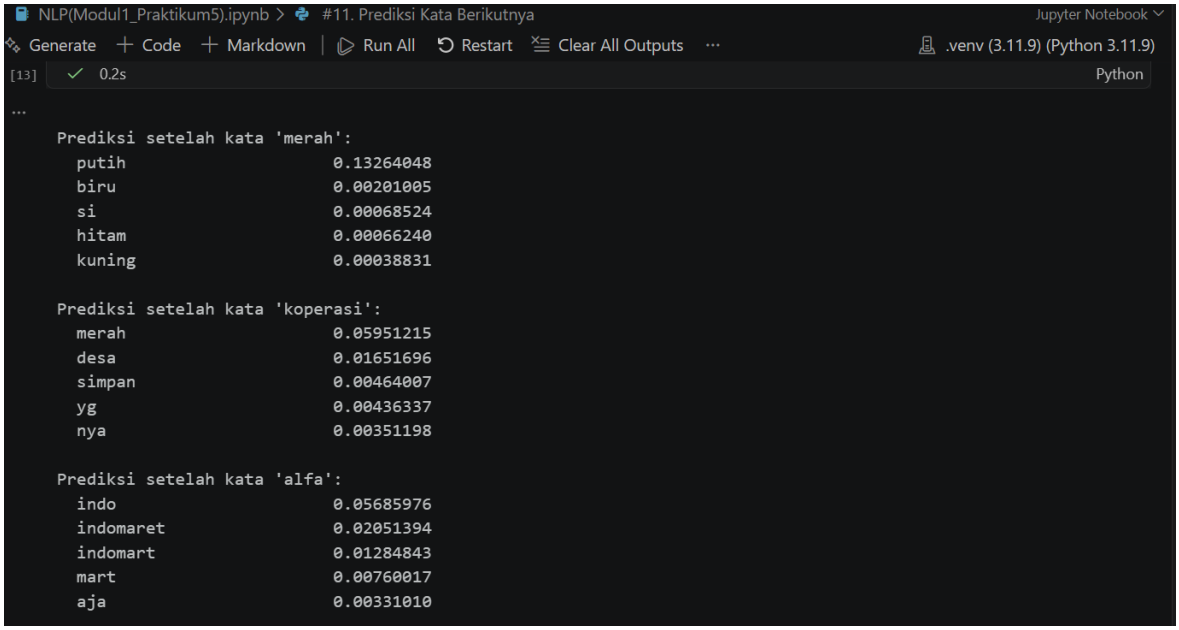
Penjelasan: Output menampilkan probabilitas untuk beberapa kalimat dengan membandingkan metode tanpa smoothing dan dengan Laplace smoothing. Pada contoh yang ditampilkan, probabilitas tanpa smoothing bernilai 0.0, sedangkan Laplace smoothing menghasilkan probabilitas yang sangat kecil tetapi tidak nol. Hal ini menunjukkan bahwa smoothing membantu menghindari probabilitas nol ketika terdapat rangkaian kata yang tidak teramati pada data.

Gambar 14. Laplace Smoothing



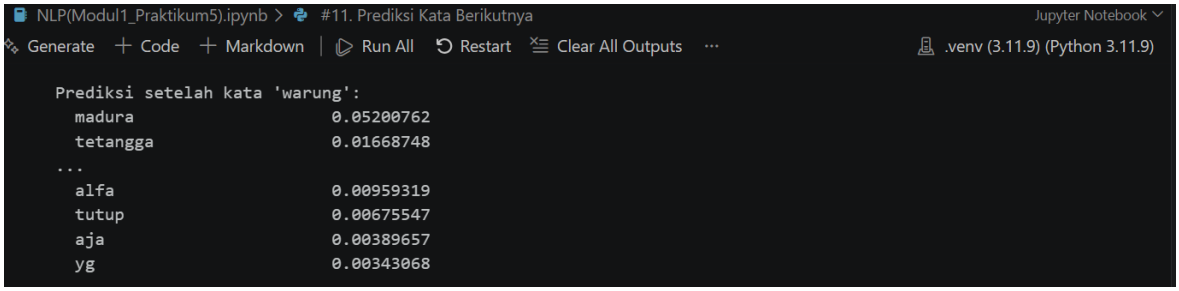
Penjelasan: Fungsi prediksi mencari kata-kata yang pernah muncul setelah kata konteks tertentu, lalu mengurutkan kandidat berdasarkan probabilitas bigram. Kandidat yang ditampilkan berasal dari data corpus.

Gambar 15. Hasil prediksi kata berikutnya



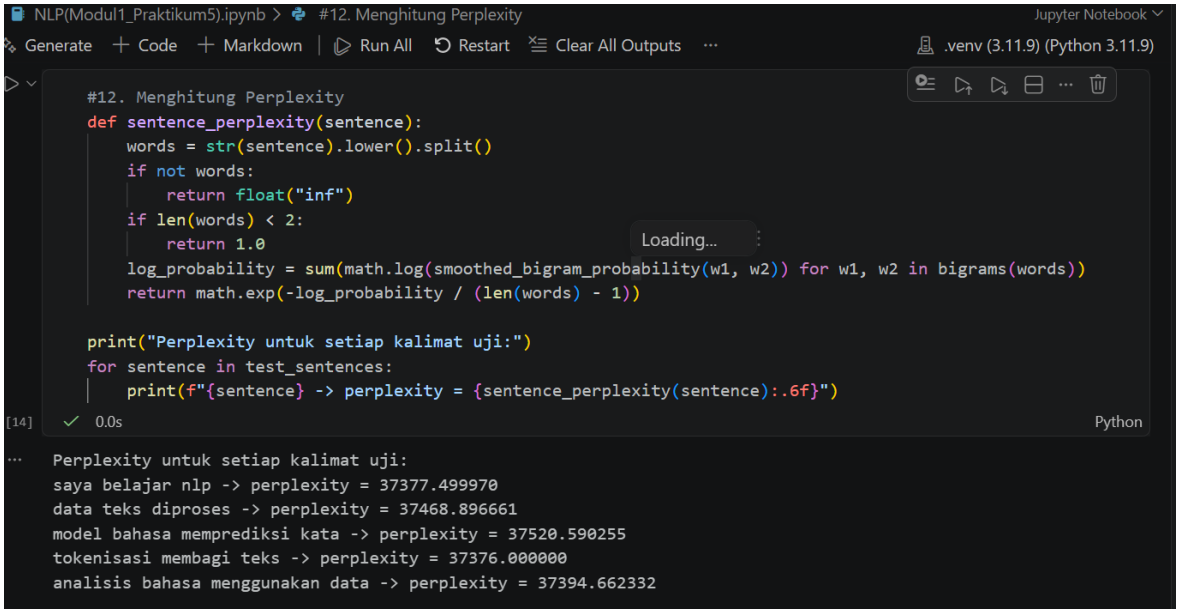
Penjelasan: Output ini menunjukkan kandidat kata setelah beberapa kata konteks. Urutan kandidat mencerminkan skor probabilitas yang dihitung model, sehingga hasilnya merupakan prediksi statistik dan bukan jaminan kata berikutnya.

Gambar 16. Lanjutan hasil prediksi



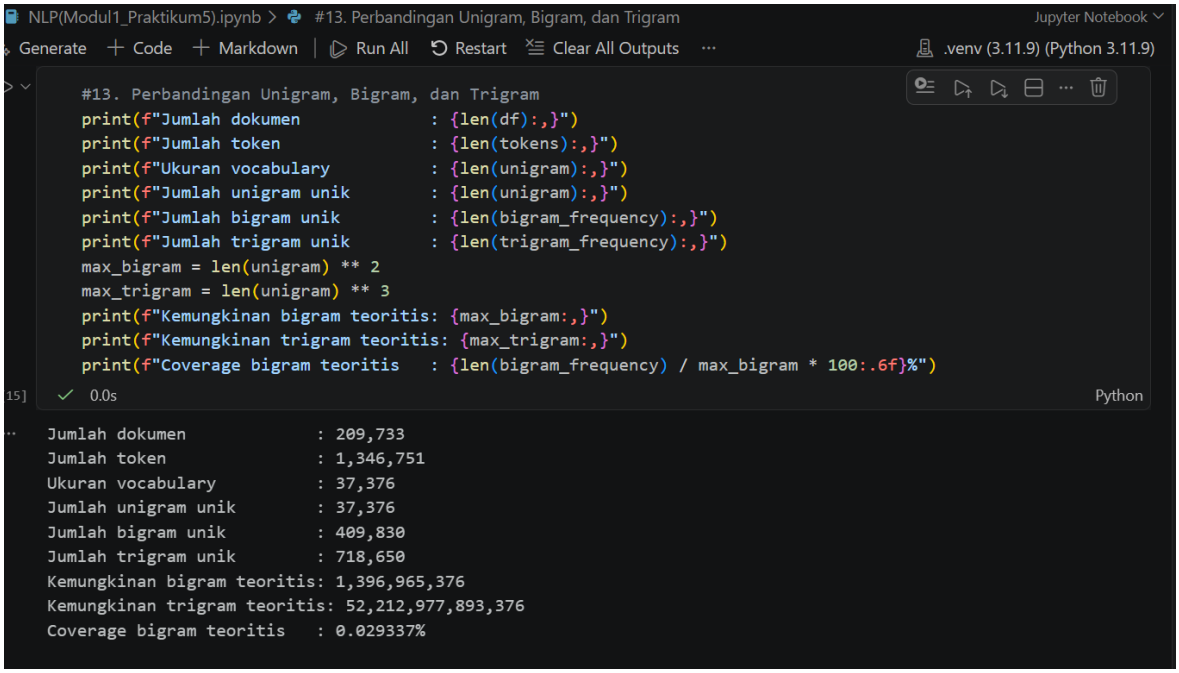
Penjelasan: Tabel ini melanjutkan hasil prediksi kata berikutnya untuk konteks yang diuji. Jika suatu konteks tidak memiliki pasangan yang tercatat, model tidak dapat menampilkan kandidat dari bigram yang ditemukan.

Gambar 17. Menghitung perplexity



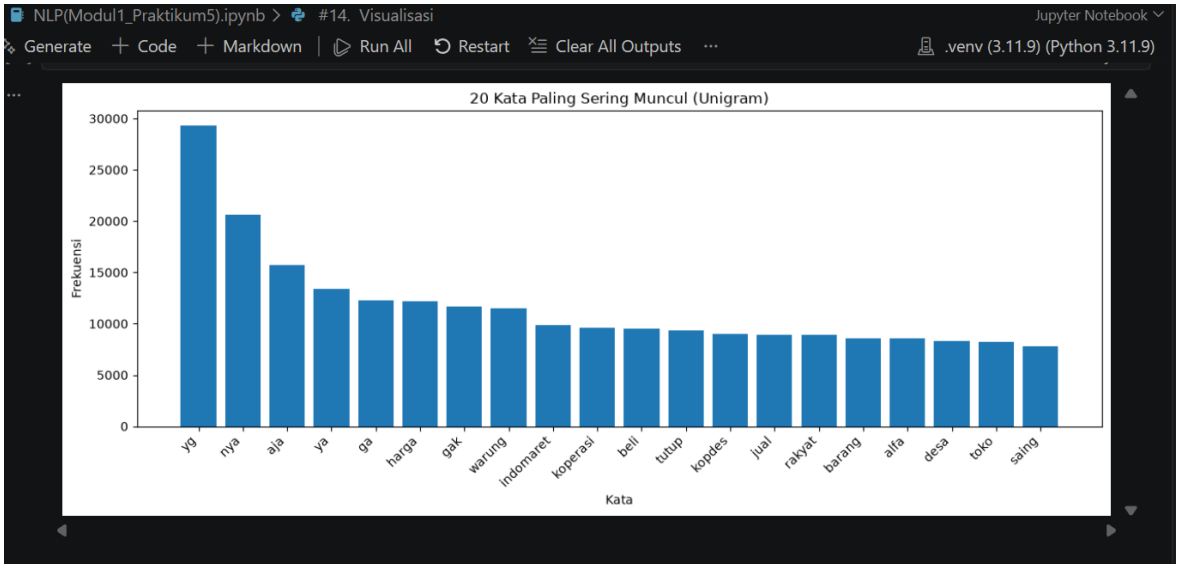
Penjelasan: Fungsi perplexity mengukur seberapa baik model memberikan probabilitas pada sequence uji. Nilai dihitung menggunakan log probabilitas untuk mengurangi masalah underflow. Perbandingan nilai hanya adil apabila data uji dan cara tokenisasi sama.

Gambar 18. Ringkasan perbandingan n-gram dan sparsity



Penjelasan: Ringkasan menampilkan jumlah token, ukuran vocabulary, serta jumlah unigram, bigram, dan trigram unik. Coverage membandingkan kombinasi yang teramati dengan jumlah kombinasi teoritis; nilainya merupakan indikator sederhana sparsity, bukan ukuran lengkap kualitas model.

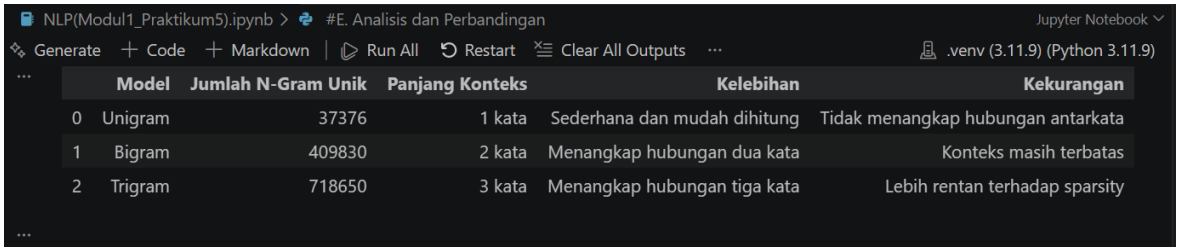
Gambar 19. Visualisasi 20 unigram teratas



Penjelasan: Grafik batang menampilkan 20 kata dengan frekuensi tertinggi. Tinggi batang menunjukkan banyaknya kemunculan kata. Grafik ini membantu mengenali kosakata dominan, tetapi tidak menjelaskan urutan atau hubungan antarkata.

E. Analisis dan Perbandingan

Gambar 20. Tabel perbandingan unigram, bigram, dan trigram


NLP(Modul1_Praktikum5).ipynb > #E. Analisis dan Perbandingan

Jupyter Notebook

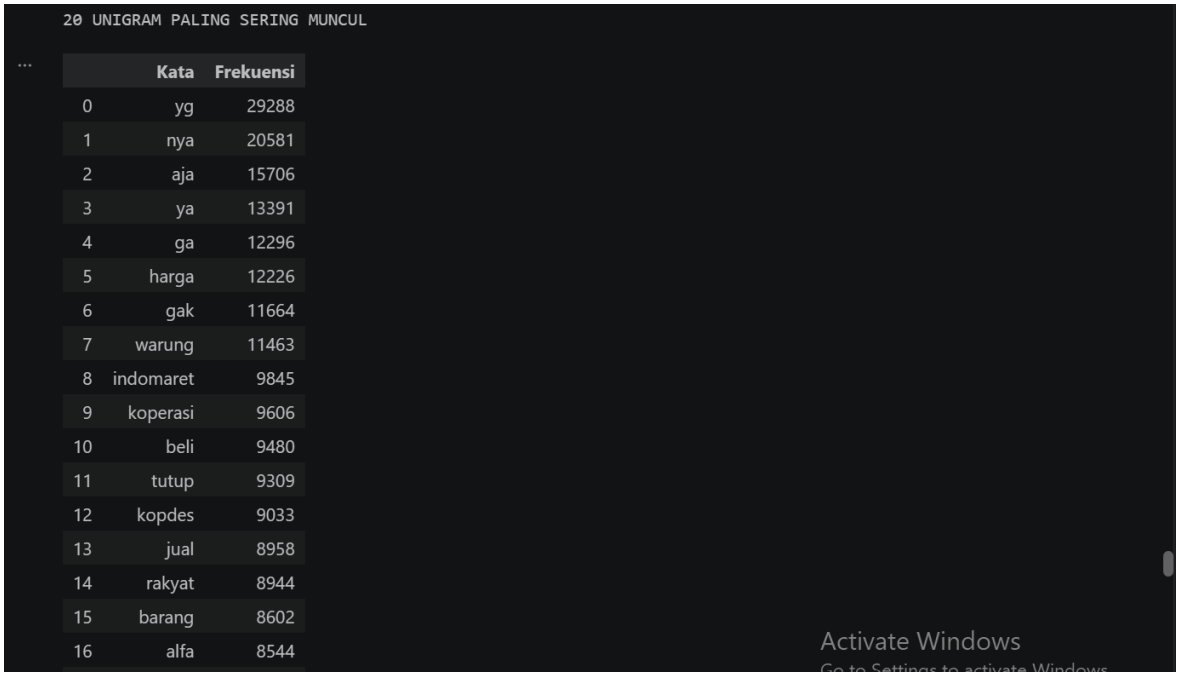
Generate + Code + Markdown | Run All Restart Clear All Outputs ...

.venv (3.11.9) (Python 3.11.9)

	Model	Jumlah N-Gram Unik	Panjang Konteks	Kelebihan	Kekurangan
0	Unigram	37376	1 kata	Sederhana dan mudah dihitung	Tidak menangkap hubungan antarkata
1	Bigram	409830	2 kata	Menangkap hubungan dua kata	Konteks masih terbatas
2	Trigram	718650	3 kata	Menangkap hubungan tiga kata	Lebih rentan terhadap sparsity

Penjelasan: Tabel merangkum perbedaan ketiga model berdasarkan konteks, informasi yang diperoleh, kompleksitas, sparsity, dan penggunaannya. Unigram fokus pada satu kata, bigram pada pasangan kata, sedangkan trigram pada rangkaian tiga kata.

Gambar 21. Hasil frekuensi unigram sebagai bahan analisis

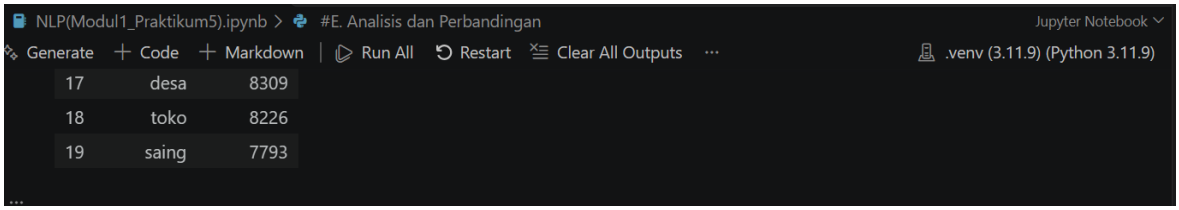

20 UNIGRAM PALING SERING MUNCUL

	Kata	Frekuensi
0	yg	29288
1	nya	20581
2	aja	15706
3	ya	13391
4	ga	12296
5	harga	12226
6	gak	11664
7	warung	11463
8	indomaret	9845
9	koperasi	9606
10	beli	9480
11	tutup	9309
12	kopdes	9033
13	jual	8958
14	rakyat	8944
15	barang	8602
16	alfa	8544

Activate Windows
Go to Settings to activate Windows.

Penjelasan: Tampilan frekuensi unigram menjadi bukti pendukung analisis kata dominan pada bagian E. Kata dengan frekuensi tinggi merupakan kata yang paling banyak muncul setelah preprocessing pada corpus.

Gambar 22. Ringkasan frekuensi pola kata


NLP(Modul1_Praktikum5).ipynb > #E. Analisis dan Perbandingan

Jupyter Notebook

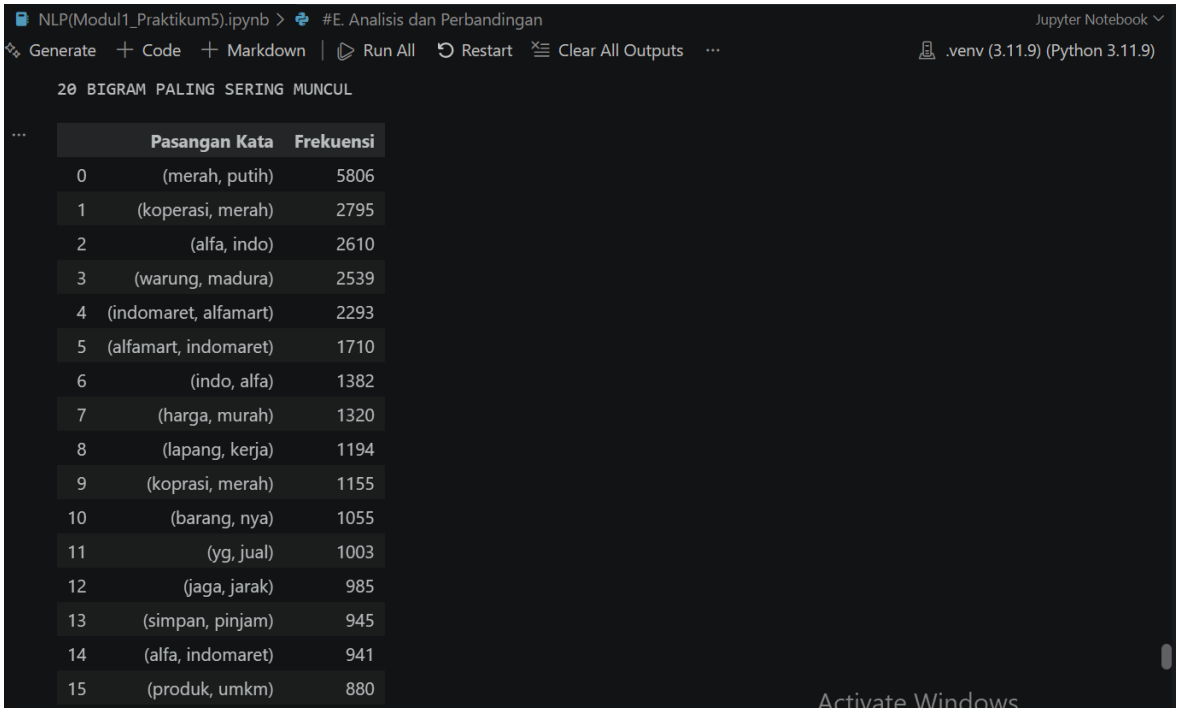
Generate + Code + Markdown | Run All Restart Clear All Outputs ...

.venv (3.11.9) (Python 3.11.9)

17	desa	8309
18	toko	8226
19	saing	7793

Penjelasan: Tampilan hasil ini mendukung pemeriksaan frekuensi pola token. Angka yang muncul merupakan hitungan kemunculan pola di corpus, sehingga pola dengan nilai lebih besar lebih sering ditemukan.

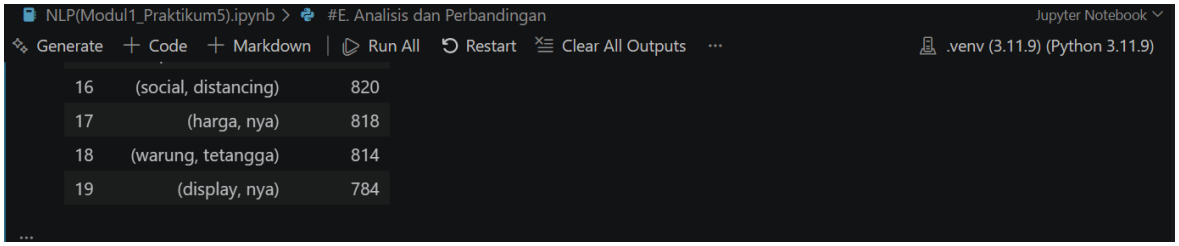
Gambar 23. Hasil frekuensi bigram untuk analisis



	Pasangan Kata	Frekuensi
0	(merah, putih)	5806
1	(koperasi, merah)	2795
2	(alfa, indo)	2610
3	(warung, madura)	2539
4	(indomaret, alfamart)	2293
5	(alfamart, indomaret)	1710
6	(indo, alfa)	1382
7	(harga, murah)	1320
8	(lapang, kerja)	1194
9	(koprasi, merah)	1155
10	(barang, nya)	1055
11	(yg, jual)	1003
12	(jaga, jarak)	985
13	(simpan, pinjam)	945
14	(alfa, indomaret)	941
15	(produk, umkm)	880

Penjelasan: Daftar bigram memperlihatkan pasangan kata yang berulang dalam teks. Hasil ini mendukung analisis bahwa bigram memberikan konteks lebih baik daripada unigram, walaupun hanya melihat satu kata sebelumnya.

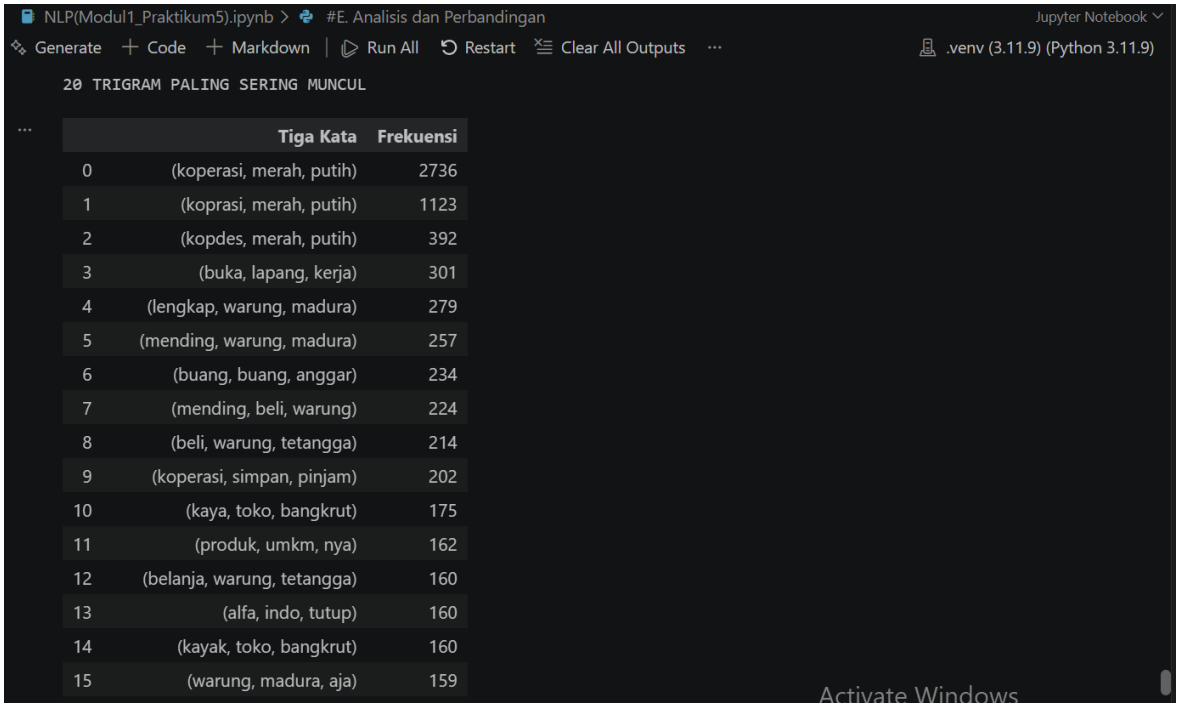
Gambar 24. Lanjutan hasil frekuensi bigram



16	(social, distancing)	820
17	(harga, nya)	818
18	(warung, tetangga)	814
19	(display, nya)	784

Penjelasan: Tampilan ini merupakan lanjutan hasil pola kata berurutan. Pola yang jarang terlihat menunjukkan bahwa tidak semua kombinasi token muncul dengan frekuensi tinggi.

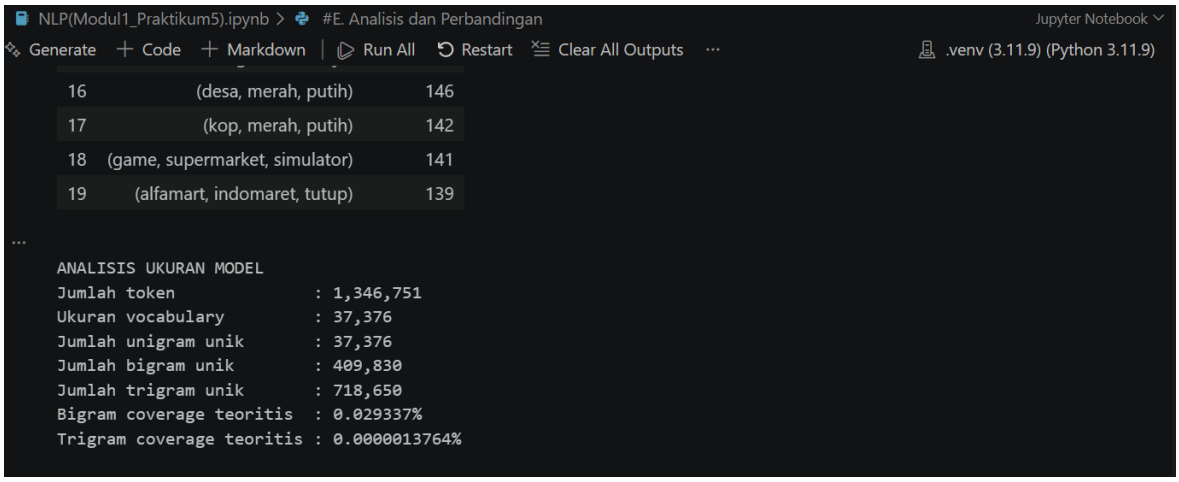
Gambar 25. Hasil frekuensi trigram untuk analisis



	Tiga Kata	Frekuensi
0	(koperasi, merah, putih)	2736
1	(koprasi, merah, putih)	1123
2	(kopdes, merah, putih)	392
3	(buka, lapang, kerja)	301
4	(lengkap, warung, madura)	279
5	(mending, warung, madura)	257
6	(buang, buang, anggar)	234
7	(mending, beli, warung)	224
8	(beli, warung, tetangga)	214
9	(koperasi, simpan, pinjam)	202
10	(kaya, toko, bangkrut)	175
11	(produk, umkm, nya)	162
12	(belanja, warung, tetangga)	160
13	(alfa, indo, tutup)	160
14	(kayak, toko, bangkrut)	160
15	(warung, madura, aja)	159

Penjelasan: Daftar trigram memperlihatkan rangkaian tiga kata yang berulang. Karena konteks lebih panjang, trigram dapat memberi informasi yang lebih spesifik tetapi lebih rentan terhadap sparsity.

Gambar 26. Statistik akhir dan coverage n-gram



Penjelasan: Statistik akhir merangkum ukuran vocabulary dan jumlah pola unik serta coverage. Coverage yang rendah berarti hanya sebagian kecil dari kombinasi teoritis yang teramati pada corpus, sehingga menunjukkan sparsity.

a. Analisis Unigram

Unigram digunakan untuk menghitung frekuensi kemunculan setiap kata dalam kolom processed_text. Melalui hasil 20 kata teratas, dapat diketahui kata-kata

yang paling sering digunakan dalam dataset komentar. Frekuensi tersebut memberikan gambaran mengenai kosakata yang dominan. Namun, unigram belum dapat menjelaskan hubungan suatu kata dengan kata sebelum atau sesudahnya.

b. Analisis Bigram

Bigram menghitung kemunculan pasangan dua kata yang berurutan. Dibandingkan unigram, bigram memberikan informasi konteks yang lebih baik karena dapat menunjukkan pasangan kata yang sering digunakan bersama dalam komentar. Akan tetapi, bigram hanya mempertimbangkan satu hubungan pasangan kata sehingga konteks yang dipahami model masih terbatas.

c. Analisis Trigram

Trigram menghitung urutan tiga kata yang berdekatan. Model ini dapat menangkap konteks yang lebih panjang daripada unigram dan bigram. Hasil trigram dapat digunakan untuk melihat rangkaian kata yang sering muncul bersama. Meskipun demikian, jumlah kombinasi yang mungkin terbentuk semakin besar sehingga banyak rangkaian kata memiliki frekuensi rendah atau tidak ditemukan dalam dataset.

d. Perbandingan Ketiga Model

Aspek	Unigram	Bigram	Trigram
Konteks	Satu kata	Dua kata berurutan	Tiga kata berurutan
Informasi	Frekuensi kata	Hubungan pasangan kata	Hubungan rangkaian tiga kata
Kompleksitas	Rendah	Sedang	Lebih tinggi
Sparsity	Relatif lebih rendah	Lebih tinggi	Cenderung paling tinggi
Penggunaan	Mengetahui kata dominan	Prediksi berdasarkan satu kata sebelumnya	Prediksi berdasarkan dua kata sebelumnya

e. Analisis Sparsity

Sparsity merupakan kondisi ketika banyak kemungkinan kombinasi kata tidak muncul dalam korpus. Pada dataset komentar yang memiliki kosakata beragam, kemungkinan ini perlu diperhatikan. Ketika panjang N-Gram meningkat, kombinasi kata yang mungkin terbentuk juga bertambah. Akibatnya, model trigram cenderung memiliki lebih banyak kombinasi dengan frekuensi rendah dibandingkan model bigram.

Nilai coverage yang dihitung oleh kode menunjukkan persentase kombinasi yang teramati dibandingkan jumlah kombinasi teoritis. Nilai ini merupakan gambaran sederhana tentang kelangkaan kombinasi, bukan ukuran lengkap kualitas model bahasa.

f. Keterbatasan N-Gram

Berdasarkan karakteristik ketiga model, terdapat beberapa keterbatasan yang perlu diperhatikan:

1. Sparsity: banyak kombinasi kata tidak ditemukan dalam korpus sehingga dapat menghasilkan probabilitas nol.
2. Konteks terbatas: bigram hanya melihat satu kata sebelumnya, sedangkan trigram melihat dua kata sebelumnya.
3. Tidak memahami makna: model menghitung pola berdasarkan token dan frekuensi, bukan memahami makna kata secara langsung.
4. Ketergantungan pada dataset: kata atau pola yang jarang muncul sulit diprediksi secara akurat.
5. Kebutuhan smoothing: Laplace smoothing dapat membantu menghindari probabilitas nol pada bigram yang tidak ditemukan.

Kesimpulan Analisis

Berdasarkan perbandingan unigram, bigram, dan trigram, setiap model memiliki fungsi yang berbeda dalam menganalisis teks pada dataset processed_text. Unigram cocok untuk melihat frekuensi kata, bigram memberikan informasi tentang hubungan dua kata, sedangkan trigram menangkap konteks tiga kata yang lebih panjang. Trigram memiliki potensi memberikan konteks yang lebih spesifik, tetapi juga lebih rentan terhadap sparsity. Oleh karena itu, pemilihan model perlu disesuaikan dengan tujuan analisis dan karakteristik

dataset. Penggunaan Laplace smoothing dapat membantu mengatasi probabilitas nol, tetapi tidak menghilangkan seluruh keterbatasan model N-Gram.

F. Pertanyaan Diskusi

1. Mengapa unigram tidak cukup untuk memahami bahasa?
   Unigram menghitung kata secara terpisah dan tidak memperhatikan urutan atau hubungan antarkata.
2. Mengapa bigram lebih baik daripada unigram untuk prediksi?
   Bigram memperhitungkan kata sebelumnya, sehingga prediksi didasarkan pada pasangan kata yang ditemukan pada corpus.
3. Mengapa trigram lebih rentan terhadap sparsity?
   Trigram melibatkan tiga token sehingga kombinasi yang mungkin lebih banyak dan banyak kombinasi tidak muncul.
4. Apa yang terjadi jika bigram belum pernah muncul?
   Frekuensi dan probabilitas bigram tanpa smoothing bernilai nol; probabilitas kalimat yang memuat pasangan itu juga dapat menjadi nol.
5. Mengapa Laplace smoothing diperlukan?
   Smoothing menambahkan satu pada frekuensi bigram agar pasangan yang belum pernah terlihat tetap memiliki probabilitas nonnol.
6. Apa hubungan n-gram dengan language model?
   N-gram adalah salah satu pendekatan untuk membangun language model dengan memperkirakan probabilitas kata berdasarkan kata sebelumnya.
7. Apa keterbatasan n-gram dibandingkan neural language model?
   N-gram memiliki konteks terbatas dan sparsity. Neural language model dapat mempelajari pola yang lebih kompleks, tetapi membutuhkan data dan komputasi yang sesuai.
8. Mengapa embedding diperlukan pada neural NLP?
   Embedding merepresentasikan token sebagai vektor numerik yang dapat menangkap kemiripan pola dari pelatihan. N-gram klasik berbasis frekuensi dan tidak memiliki representasi vektor semantik bawaan.

G. Daftar Pustaka

Chen, S. F., & Goodman, J. (1999). An empirical study of smoothing techniques for language modeling. Computer Speech & Language, 13(4), 359–394. https://doi.org/10.1006/csla.1999.0128

Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models (3rd ed., online manuscript). Stanford University. https://web.stanford.edu/~jurafsky/slp3/

Manning, C. D., & Schütze, H. (1999). Foundations of Statistical Natural Language Processing. MIT Press. https://mitpress.mit.edu/9780262133609/foundations-of-statistical-natural-language-processing/

CEKLIST PENIALAIAN PRAKTIKUM

No.	Modul	NILAI	Catatan
1	Modul 1 sesuaikan dengan CPMK-1		
2	Modul 2 sesuaikan dengan CPMK-2		
3			
…	…	…	…

Tanggal:

Praktikan: Eka Yuliana Rizky	Dosen Pengampu: Ahmad Ashril Rizal, M.Cs.